School of Computing

Department of Computer Science and Engineering

ACADEMIC YEAR: 2025-26

(WINTER SEMESTER)

LAB RECORD

10211CS224 - Full Stack Application Development

NAME : SK. Mansoor

VTU.NO : 26005

REG.NO : 23UECS1202

BRANCH : CSE(AI&DS)

YEAR/SEM : 3rd year, 6th sem

SLOT : E2

10211CS224 - Full Stack Application Development

TABLE OF CONTENTS

23.	21-04-2026	Prompt Engineering for Code Generation Using Generative AI
24.	28-04-2026	Cloud-Based Application Using Vibe Coding and Generative AI
25.	29-04-2026	Use Cases

TASK QUESTION

Task 1

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: student-registration.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration System</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
```

```
    padding: 20px;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
}

.header {
    text-align: center;
    color: white;
    margin-bottom: 30px;
}

.header h1 {
    font-size: 2.5rem;
    margin-bottom: 10px;
    text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}

.content-wrapper {
    display: grid;
    grid-template-columns: 1fr 2fr;
    gap: 30px;
    align-items: start;
}

.form-section, .table-section {
    background: white;
    border-radius: 15px;
    padding: 30px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.2);
}

.form-section h2, .table-section h2 {
    color: #333;
    margin-bottom: 20px;
    font-size: 1.5rem;
    border-bottom: 3px solid #667eea;
    padding-bottom: 10px;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
```

```
    display: block;
    margin-bottom: 5px;
    color: #555;
    font-weight: 600;
}

.form-group input,
.form-group select {
    width: 100%;
    padding: 12px;
    border: 2px solid #e1e1e1;
    border-radius: 8px;
    font-size: 16px;
    transition: border-color 0.3s ease;
}

.form-group input:focus,
.form-group select:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

.form-group input.error {
    border-color: #e74c3c;
}

.error-message {
    color: #e74c3c;
    font-size: 14px;
    margin-top: 5px;
    display: none;
}

.btn {
    padding: 12px 24px;
    border: none;
    border-radius: 8px;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
    margin-right: 10px;
}

.btn-primary {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
```

```
    color: white;
}

.btn-primary:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

.btn-danger {
    background: linear-gradient(135deg, #e74c3c 0%, #c0392b 100%);
    color: white;
}

.btn-danger:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(231, 76, 60, 0.4);
}

.message {
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
    display: none;
    animation: slideIn 0.3s ease;
}

.message.success {
    background: #d4edda;
    color: #155724;
    border: 1px solid #c3e6cb;
}

.message.error {
    background: #f8d7da;
    color: #721c24;
    border: 1px solid #f5c6cb;
}

@keyframes slideIn {
    from {
        opacity: 0;
        transform: translateY(-20px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
```

```
}

.student-table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}

.student-table thead {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
}

.student-table th,
.student-table td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #e1e1e1;
}

.student-table tbody tr:hover {
    background: #f8f9fa;
}

.no-records {
    text-align: center;
    padding: 40px;
    color: #666;
    font-style: italic;
}

.table-controls {
    margin-bottom: 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.record-count {
    color: #666;
    font-weight: 600;
}

@media (max-width: 768px) {
    .content-wrapper {
        grid-template-columns: 1fr;
    }
```

```
        .header h1 {
            font-size: 2rem;
        }

        .form-section, .table-section {
            padding: 20px;
        }
    }

    .loading {
        display: inline-block;
        width: 20px;
        height: 20px;
        border: 3px solid #f3f3f3;
        border-top: 3px solid #667eea;
        border-radius: 50%;
        animation: spin 1s linear infinite;
        margin-left: 10px;
    }

    @keyframes spin {
        0% { transform: rotate(0deg); }
        100% { transform: rotate(360deg); }
    }
  </style>
</head>
<body>
  <div class="container">
    <header class="header">
      <h1>□ Student Registration System</h1>
      <p>Register and manage student records with Web SQL Database</p>
    </header>

    <div class="content-wrapper">
      <section class="form-section">
        <h2>Register New Student</h2>

        <div id="message" class="message"></div>

        <form id="studentForm">
          <div class="form-group">
            <label for="fullName">Full Name *</label>
            <input type="text" id="fullName" name="fullName" placeholder="Enter student's full
name" required>
            <div class="error-message" id="nameError">Please enter a valid name (at least 3
characters)</div>
```

```
            </div>

            <div class="form-group">
              <label for="email">Email Address *</label>
              <input type="email" id="email" name="email" placeholder="student@example.com"
required>
              <div class="error-message" id="emailError">Please enter a valid email address</div>
            </div>

            <div class="form-group">
              <label for="dob">Date of Birth *</label>
              <input type="date" id="dob" name="dob" required>
              <div class="error-message" id="dobError">Please select a valid date of birth</div>
            </div>

            <div class="form-group">
              <label for="department">Department *</label>
              <select id="department" name="department" required>
                <option value="">Select Department</option>
                <option value="CSE">Computer Science Engineering</option>
                <option value="ECE">Electronics and Communication Engineering</option>
                <option value="EEE">Electrical and Electronics Engineering</option>
                <option value="Mechanical">Mechanical Engineering</option>
                <option value="Civil">Civil Engineering</option>
              </select>
              <div class="error-message" id="departmentError">Please select a department</div>
            </div>

            <div class="form-group">
              <label for="phone">Phone Number *</label>
              <input type="tel" id="phone" name="phone" placeholder="10-digit phone number"
maxlength="10" required>
              <div class="error-message" id="phoneError">Please enter a valid 10-digit phone
number</div>
            </div>

            <button type="submit" class="btn btn-primary">
              Register Student
            </button>
            <button type="button" class="btn btn-secondary" onclick="resetForm()">
              Clear Form
            </button>
          </form>
        </section>

        <section class="table-section">
          <h2>Registered Students</h2>
```

```
        <div class="table-controls">
          <span class="record-count" id="recordCount">Total Records: 0</span>
          <button class="btn btn-danger" onclick="deleteAllRecords()">
            Delete All Records
          </button>
        </div>

        <div id="tableContainer">
          <div class="no-records">No student records found. Register your first student!</div>
        </div>
      </section>
    </div>
  </div>

  <script>
    // Database variables
    let db;
    const DB_NAME = 'StudentDB';
    const DB_VERSION = '1.0';
    const DB_DISPLAY_NAME = 'Student Registration Database';
    const DB_SIZE = 2 * 1024 * 1024; // 2MB

    // Initialize database
    function initDatabase() {
      try {
        db = openDatabase(DB_NAME, DB_VERSION, DB_DISPLAY_NAME, DB_SIZE);

        db.transaction(function(tx) {
          // Create students table if it doesn't exist
          tx.executeSql(
            `CREATE TABLE IF NOT EXISTS students (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              name TEXT NOT NULL,
              email TEXT UNIQUE NOT NULL,
              dob TEXT NOT NULL,
              department TEXT NOT NULL,
              phone TEXT NOT NULL,
              created_at DATETIME DEFAULT CURRENT_TIMESTAMP
            )`,
            [],
            function(tx, result) {
              console.log('Table created successfully');
              loadStudents();
            },
            function(tx, error) {
              console.error('Error creating table:', error);
```

```
                showMessage('Error initializing database: ' + error.message, 'error');
              }
            );
          });
        } catch (error) {
          console.error('Error opening database:', error);
          showMessage('Web SQL is not supported in this browser. Please use a compatible browser.',
'error');
        }
      }

      // Form validation
      function validateForm() {
        let isValid = true;

        // Reset error states
        document.querySelectorAll('.error-message').forEach(msg => msg.style.display = 'none');
        document.querySelectorAll('input, select').forEach(field => field.classList.remove('error'));

        // Validate name
        const name = document.getElementById('fullName').value.trim();
        if (name.length < 3) {
          document.getElementById('nameError').style.display = 'block';
          document.getElementById('fullName').classList.add('error');
          isValid = false;
        }

        // Validate email
        const email = document.getElementById('email').value.trim();
        const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailRegex.test(email)) {
          document.getElementById('emailError').style.display = 'block';
          document.getElementById('email').classList.add('error');
          isValid = false;
        }

        // Validate date of birth
        const dob = document.getElementById('dob').value;
        if (!dob) {
          document.getElementById('dobError').style.display = 'block';
          document.getElementById('dob').classList.add('error');
          isValid = false;
        } else {
          // Check if age is reasonable (between 16 and 60 years)
          const dobDate = new Date(dob);
          const today = new Date();
          const age = Math.floor((today - dobDate) / (365.25 * 24 * 60 * 60 * 1000));
```

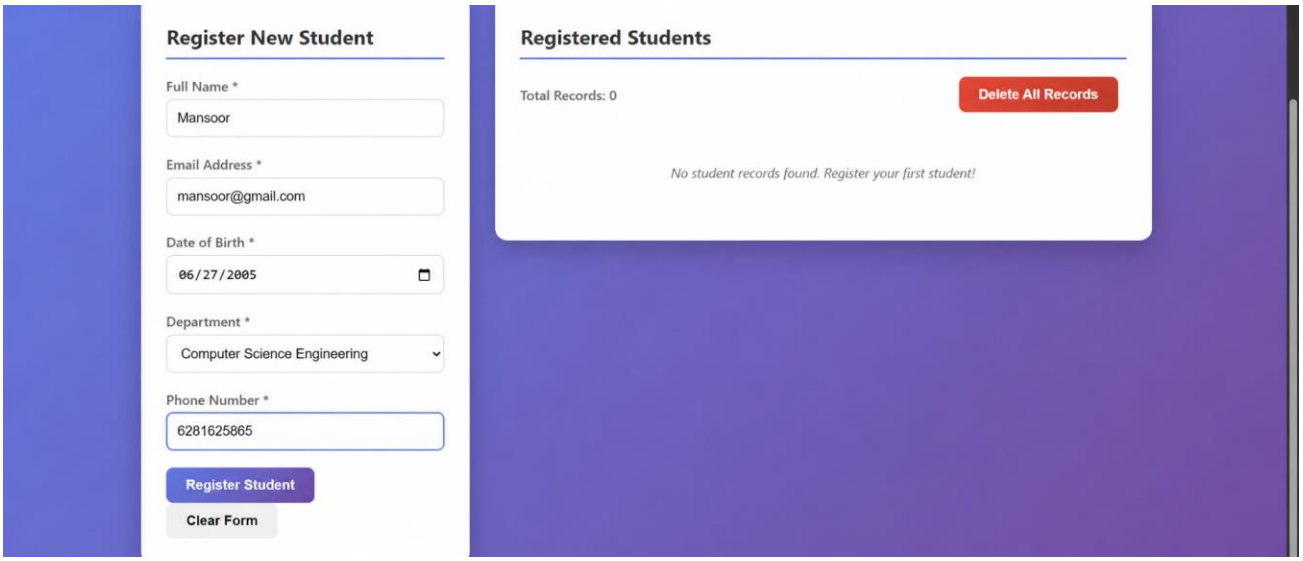
```
            if (age < 16 || age > 60) {
                document.getElementById('dobError').textContent = 'Age should be between 16 and 60
years';
                document.getElementById('dobError').style.display = 'block';
                document.getElementById('dob').classList.add('error');
                isValid = false;
            }
        }

        // Validate department
        const department = document.getElementById('department').value;
        if (!department) {
            document.getElementById('departmentError').style.display = 'block';
            document.getElementById('department').classList.add('error');
            isValid = false;
        }

        // Validate phone
        const phone = document.getElementById('phone').value.trim();
        const phoneRegex = /^[6-9]\d{9}$/;
        if (!phoneRegex.test(phone)) {
            document.getElementById('phoneError').style.display = 'block';
            document.getElementById('phone').classList.add('error');
            isValid = false;
        }

        return isValid;
    }

    // Check for duplicate email
    function checkDuplicateEmail(email, callback) {
        db.transaction(function(tx) {
            tx.executeSql(
                'SELECT id FROM students WHERE email = ?',
                [email],
                function(tx, result) {
                    callback(result.rows.length > 0);
                },
                function(tx, error) {
                    console.error('Error checking duplicate email:', error);
                    callback(false);
                }
            );
        });
    }

    // Insert student record
```

```
    function insertStudent(studentData) {
        db.transaction(function(tx) {
            tx.executeSql(
                'INSERT INTO students (name, email, dob, department, phone) VALUES (?, ?, ?, ?, ?)',
                [studentData.name, studentData.email, studentData.dob, studentData.department,
studentData.phone],
                function(tx, result) {
                    showMessage('Student registered successfully!', 'success');
                    resetForm();
                    loadStudents();
                },
                function(tx, error) {
                    console.error('Error inserting student:', error);
                    if (error.message.includes('UNIQUE constraint failed')) {
                        showMessage('This email address is already registered!', 'error');
                    } else {
                        showMessage('Error registering student: ' + error.message, 'error');
                    }
                }
            );
        });
    }

    // Load all students
    function loadStudents() {
        db.transaction(function(tx) {
            tx.executeSql(
                'SELECT * FROM students ORDER BY created_at DESC',
                [],
                function(tx, result) {
                    displayStudents(result.rows);
                },
                function(tx, error) {
                    console.error('Error loading students:', error);
                    showMessage('Error loading student data: ' + error.message, 'error');
                }
            );
        });
    }

    // Display students in table
    function displayStudents(rows) {
        const container = document.getElementById('tableContainer');
        const recordCount = document.getElementById('recordCount');

        recordCount.textContent = `Total Records: ${rows.length}`;
```

```
        if (rows.length === 0) {
            container.innerHTML = '<div class="no-records">No student records found. Register your first student!</div>';
            return;
        }

        let tableHTML = `
            <table class="student-table">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Date of Birth</th>
                        <th>Department</th>
                        <th>Phone</th>
                    </tr>
                </thead>
                <tbody>
        `;

        for (let i = 0; i < rows.length; i++) {
            const student = rows[i];
            const formattedDate = new Date(student.dob).toLocaleDateString('en-US', {
                year: 'numeric',
                month: 'short',
                day: 'numeric'
            });

            tableHTML += `
                <tr>
                    <td>${student.id}</td>
                    <td>${student.name}</td>
                    <td>${student.email}</td>
                    <td>${formattedDate}</td>
                    <td>${student.department}</td>
                    <td>${student.phone}</td>
                </tr>
            `;
        }

        tableHTML += '</tbody></table>';
        container.innerHTML = tableHTML;
    }

    // Delete all records
    function deleteAllRecords() {
```

```
    if (confirm('Are you sure you want to delete all student records? This action cannot be undone.'))
{
        db.transaction(function(tx) {
            tx.executeSql(
                'DELETE FROM students',
                [],
                function(tx, result) {
                    showMessage('All records deleted successfully!', 'success');
                    loadStudents();
                },
                function(tx, error) {
                    console.error('Error deleting records:', error);
                    showMessage('Error deleting records: ' + error.message, 'error');
                }
            );
        });
    }
}

// Reset form
function resetForm() {
    document.getElementById('studentForm').reset();
    document.querySelectorAll('.error-message').forEach(msg => msg.style.display = 'none');
    document.querySelectorAll('input, select').forEach(field => field.classList.remove('error'));
}

// Show message
function showMessage(text, type) {
    const messageEl = document.getElementById('message');
    messageEl.textContent = text;
    messageEl.className = `message ${type}`;
    messageEl.style.display = 'block';

    // Auto-hide after 5 seconds
    setTimeout(() => {
        messageEl.style.display = 'none';
    }, 5000);
}

// Form submission handler
document.getElementById('studentForm').addEventListene
```

OUTPUT


Register New Student
Full Name *
Mansoor
Email Address *
mansoor@gmail.com
Date of Birth *
06/27/2005
Department *
Computer Science Engineering
Phone Number *
6281625865
Register Student
Clear Form
Registered Students
Total Records: 0
Delete All Records
No student records found. Register your first student!

RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 2

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: dashboard.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Retrieval & Sorting Dashboard</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: white;
      color: #333;
      line-height: 1.6;
    }

    .container {
```

```
    max-width: 1400px;
    margin: 0 auto;
    padding: 20px;
}

header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 30px 0;
    text-align: center;
    border-radius: 10px;
    margin-bottom: 30px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

h1 {
    font-size: 2.5em;
    margin-bottom: 10px;
}

.subtitle {
    font-size: 1.1em;
    opacity: 0.9;
}

.dashboard-grid {
    display: grid;
    grid-template-columns: 1fr 300px;
    gap: 30px;
    margin-bottom: 30px;
}

.main-content {
    background: white;
    border-radius: 10px;
    padding: 25px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.sidebar {
    background: white;
    border-radius: 10px;
    padding: 25px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    height: fit-content;
}
```

```
.controls-section {
    background: #f8f9fa;
    border-radius: 8px;
    padding: 20px;
    margin-bottom: 25px;
}

.controls-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 15px;
    margin-bottom: 15px;
}

.control-group {
    display: flex;
    flex-direction: column;
}

label {
    font-weight: 600;
    margin-bottom: 5px;
    color: #555;
    font-size: 0.9em;
}

input[type="text"], select {
    padding: 10px;
    border: 2px solid #e1e5e9;
    border-radius: 5px;
    font-size: 14px;
    transition: border-color 0.3s ease;
}

input[type="text"]:focus, select:focus {
    outline: none;
    border-color: #667eea;
}

.button-group {
    display: flex;
    gap: 10px;
    flex-wrap: wrap;
}

button {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
```

```
    color: white;
    border: none;
    padding: 10px 20px;
    border-radius: 5px;
    cursor: pointer;
    font-size: 14px;
    font-weight: 600;
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}

button:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
}

button:active {
    transform: translateY(0);
}

.secondary-btn {
    background: #6c757d;
}

.table-container {
    overflow-x: auto;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

table {
    width: 100%;
    border-collapse: collapse;
    background: white;
}

th, td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #e1e5e9;
}

th {
    background: #f8f9fa;
    font-weight: 600;
    color: #495057;
    position: sticky;
    top: 0;
```

```
}

tr:hover {
    background-color: #f8f9fa;
}

.highlight {
    background-color: #fff3cd !important;
    animation: highlight 0.5s ease;
}

@keyframes highlight {
    0% { background-color: #ffeaa7; }
    100% { background-color: #fff3cd; }
}

.no-records {
    text-align: center;
    padding: 40px;
    color: #6c757d;
    font-size: 1.1em;
}

.analytics-card {
    background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
    color: white;
    border-radius: 8px;
    padding: 20px;
    margin-bottom: 20px;
}

.analytics-card h3 {
    margin-bottom: 15px;
    font-size: 1.2em;
}

.department-count {
    display: flex;
    justify-content: space-between;
    padding: 8px 0;
    border-bottom: 1px solid rgba(255,255,255,0.2);
}

.department-count:last-child {
    border-bottom: none;
}
```

```
.department-name {
    font-weight: 600;
}

.count-badge {
    background: rgba(255,255,255,0.2);
    padding: 2px 8px;
    border-radius: 12px;
    font-size: 0.9em;
}

.stats-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 15px;
    margin-top: 20px;
}

.stat-item {
    background: rgba(255,255,255,0.1);
    padding: 15px;
    border-radius: 8px;
    text-align: center;
}

.stat-number {
    font-size: 2em;
    font-weight: bold;
    margin-bottom: 5px;
}

.stat-label {
    font-size: 0.9em;
    opacity: 0.9;
}

@media (max-width: 768px) {
    .dashboard-grid {
        grid-template-columns: 1fr;
    }

    .controls-grid {
        grid-template-columns: 1fr;
    }

    .stats-grid {
        grid-template-columns: 1fr;
```

```
      }

      h1 {
        font-size: 2em;
      }
    }

    .loading {
      display: inline-block;
      width: 20px;
      height: 20px;
      border: 3px solid #f3f3f3;
      border-top: 3px solid #667eea;
      border-radius: 50%;
      animation: spin 1s linear infinite;
    }

    @keyframes spin {
      0% { transform: rotate(0deg); }
      100% { transform: rotate(360deg); }
    }
  </style>
</head>
<body>
  <div class="container">
    <header>
      <h1>📊 Data Retrieval & Sorting Dashboard</h1>
      <p class="subtitle">Student/Employee Records Management System</p>
    </header>

    <div class="dashboard-grid">
      <main class="main-content">
        <section class="controls-section">
          <div class="controls-grid">
            <div class="control-group">
              <label for="searchInput">🔍 Search by Name</label>
              <input type="text" id="searchInput" placeholder="Enter name to search...">
            </div>

            <div class="control-group">
              <label for="departmentFilter">🏢 Department Filter</label>
              <select id="departmentFilter">
                <option value="all">All Departments</option>
                <option value="CSE">Computer Science</option>
                <option value="ECE">Electronics & Communication</option>
                <option value="EEE">Electrical & Electronics</option>
                <option value="Mechanical">Mechanical Engineering</option>
```

```
            <option value="Civil">Civil Engineering</option>
          </select>
        </div>
      </div>

      <div class="button-group">
        <button onclick="sortByName('asc')">🔤 Name A→Z</button>
        <button onclick="sortByName('desc')">🔤 Name Z→A</button>
        <button onclick="sortByDOB('asc')">🎂 DOB Youngest→Oldest</button>
        <button onclick="sortByDOB('desc')">🎂 DOB Oldest→Youngest</button>
        <button class="secondary-btn" onclick="resetFilters()">🔄 Reset All</button>
      </div>
    </section>

    <section class="table-container">
      <table id="dataTable">
        <thead>
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Date of Birth</th>
            <th>Department</th>
            <th>Phone</th>
          </tr>
        </thead>
        <tbody id="tableBody">
        </tbody>
      </table>
      <div id="noRecords" class="no-records" style="display: none;">
        <p>📭 No records found</p>
        <p>Try adjusting your filters or search criteria</p>
      </div>
    </section>
  </main>

  <aside class="sidebar">
    <div class="analytics-card">
      <h3>📊 Department Analytics</h3>
      <div id="departmentCounts"></div>
    </div>

    <div class="stats-grid">
      <div class="stat-item">
        <div class="stat-number" id="totalRecords">0</div>
        <div class="stat-label">Total Records</div>
      </div>
      <div class="stat-item">
```

```
                <div class="stat-number" id="filteredRecords">0</div>
                <div class="stat-label">Filtered Records</div>
              </div>
            </div>
          </aside>
        </div>
      </div>

      <script>
        class DataDashboard {
          constructor() {
            this.originalData = [];
            this.filteredData = [];
            this.initializeDatabase();
            this.attachEventListeners();
          }

          initializeDatabase() {
            if (!localStorage.getItem('studentData')) {
              const sampleData = [
                { name: "Alice Johnson", email: "alice@example.com", dob: "2000-05-15", department:
"CSE", phone: "123-456-7890" },
                { name: "Bob Smith", email: "bob@example.com", dob: "1999-08-22", department: "ECE",
phone: "234-567-8901" },
                { name: "Charlie Brown", email: "charlie@example.com", dob: "2001-02-10", department:
"EEE", phone: "345-678-9012" },
                { name: "Diana Prince", email: "diana@example.com", dob: "2000-11-30", department:
"Mechanical", phone: "456-789-0123" },
                { name: "Edward Norton", email: "edward@example.com", dob: "1999-04-18", department:
"Civil", phone: "567-890-1234" },
                { name: "Fiona Green", email: "fiona@example.com", dob: "2001-07-25", department:
"CSE", phone: "678-901-2345" },
                { name: "George Wilson", email: "george@example.com", dob: "2000-09-12", department:
"ECE", phone: "789-012-3456" },
                { name: "Hannah Baker", email: "hannah@example.com", dob: "1999-12-05", department:
"EEE", phone: "890-123-4567" },
                { name: "Ian McKellen", email: "ian@example.com", dob: "2001-03-28", department:
"Mechanical", phone: "901-234-5678" },
                { name: "Julia Roberts", email: "julia@example.com", dob: "2000-06-14", department:
"Civil", phone: "012-345-6789" },
                { name: "Kevin Hart", email: "kevin@example.com", dob: "1999-10-08", department: "CSE",
phone: "123-456-7891" },
                { name: "Laura Palmer", email: "laura@example.com", dob: "2001-01-20", department:
"ECE", phone: "234-567-8902" },
                { name: "Michael Scott", email: "michael@example.com", dob: "2000-08-03", department:
"EEE", phone: "345-678-9013" },
                { name: "Nancy Drew", email: "nancy@example.com", dob: "1999-05-17", department:
```

```
"Mechanical", phone: "456-789-0124" },
                { name: "Oscar Wilde", email: "oscar@example.com", dob: "2001-12-01", department:
"Civil", phone: "567-890-1235" }
            ];
            localStorage.setItem('studentData', JSON.stringify(sampleData));
        }

        this.loadData();
    }

    loadData() {
        const storedData = localStorage.getItem('studentData');
        this.originalData = JSON.parse(storedData);
        this.filteredData = [...this.originalData];
        this.renderTable();
        this.updateAnalytics();
    }

    attachEventListeners() {
        document.getElementById('searchInput').addEventListener('input', (e) => {
            this.searchByName(e.target.value);
        });

        document.getElementById('departmentFilter').addEventListener('change', (e) => {
            this.filterByDepartment(e.target.value);
        });
    }

    searchByName(searchTerm) {
        const term = searchTerm.toLowerCase();
        if (term === '') {
            this.filteredData = [...this.originalData];
        } else {
            this.filteredData = this.originalData.filter(person =>
                person.name.toLowerCase().includes(term)
            );
        }

        const currentDept = document.getElementById('departmentFilter').value;
        if (currentDept !== 'all') {
            this.filteredData = this.filteredData.filter(person =>
                person.department === currentDept
            );
        }

        this.renderTable();
        this.updateAnalytics();
```

```
    this.highlightSearchResults(term);
}

filterByDepartment(department) {
    if (department === 'all') {
        this.filteredData = [...this.originalData];
    } else {
        this.filteredData = this.originalData.filter(person =>
            person.department === department
        );
    }

    const searchTerm = document.getElementById('searchInput').value;
    if (searchTerm !== '') {
        this.filteredData = this.filteredData.filter(person =>
            person.name.toLowerCase().includes(searchTerm.toLowerCase())
        );
    }

    this.renderTable();
    this.updateAnalytics();
}

sortByName(order) {
    this.filteredData.sort((a, b) => {
        if (order === 'asc') {
            return a.name.localeCompare(b.name);
        } else {
            return b.name.localeCompare(a.name);
        }
    });
    this.renderTable();
    this.highlightSortedRows();
}

sortByDOB(order) {
    this.filteredData.sort((a, b) => {
        const dateA = new Date(a.dob);
        const dateB = new Date(b.dob);
        if (order === 'asc') {
            return dateB - dateA;
        } else {
            return dateA - dateB;
        }
    });
    this.renderTable();
    this.highlightSortedRows();
```

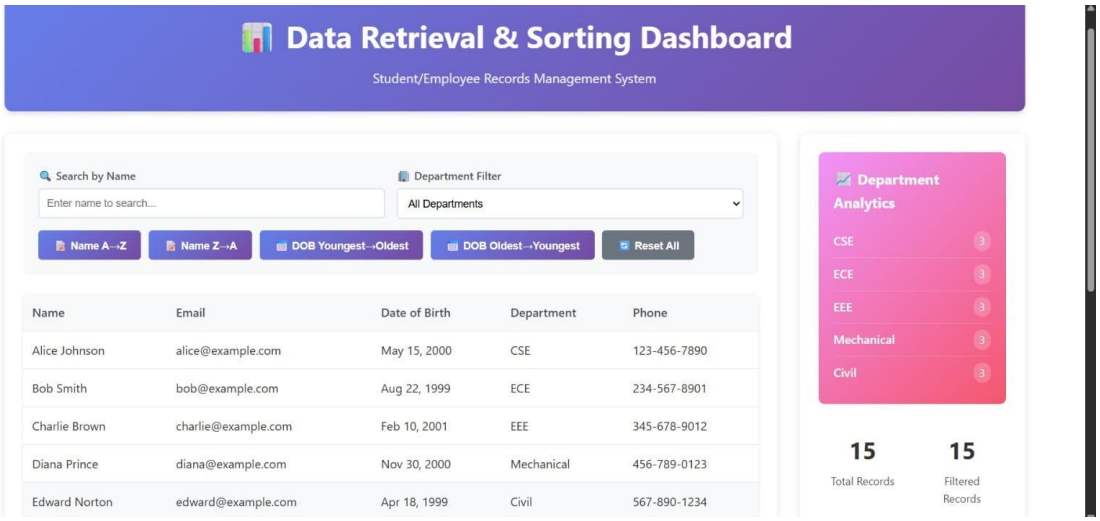
```
}

renderTable() {
    const tableBody = document.getElementById('tableBody');
    const noRecords = document.getElementById('noRecords');

    if (this.filteredData.length === 0) {
        tableBody.innerHTML = '';
        noRecords.style.display = 'block';
        return;
    }

    noRecords.style.display = 'none';
    tableBody.innerHTML = this.filteredData.map((person, index) => `
        <tr data-index="${index}">
            <td>${person.name}</td>
            <td>${person.email}</td>
            <td>${this.formatDate(person.dob)}</td>
            <td>${person.department}</td>
            <td>${person.phone}</td>
        </tr>
    `).join('');
}

formatDate(dateString) {
    const date = new Date(dateString);
    return date.toLocaleDateString('en-US', {
        year: 'numeric',
        month: 'short',
        day: 'numeric'
    });
}

updateAnalytics() {
    const departmentCounts = {};
    const departments = ['CSE', 'ECE', 'EEE', 'Mechanical', 'Civil'];

    departments.forEach(dept => {
        departmentCounts[dept] = this.originalData.filter(person =>
            person.department === dept
        ).length;
    });

    const countsHtml = Object.entries(departmentCounts).map(([dept, count]) => `
        <div class="department-count">
            <span class="department-name">${dept}</span>
            <span class="count-badge">${count}</span>
```

```
            </div>
        `).join('');

        document.getElementById('departmentCounts').innerHTML = countsHtml;
        document.getElementById('totalRecords').textContent = this.originalData.length;
        document.getElementById('filteredRecords').textContent = this.filteredData.length;
    }

    highlightSearchResults(searchTerm) {
        const rows = document.querySelectorAll('#tableBody tr');
        rows.forEach(row => {
            row.classList.remove('highlight');
            const nameCell = row.cells[0];
            if (searchTerm &&
nameCell.textContent.toLowerCase().includes(searchTerm.toLowerCase())) {
                row.classList.add('highlight');
            }
        });
    }

    highlightSortedRows() {
        const rows = document.querySelectorAll('#tableBody tr');
        rows.forEach((row, index) => {
            setTimeout(() => {
                row.classList.add('highlight');
                setTimeout(() => {
                    row.classList.remove('highlight');
                }, 1000);
            }, index * 50);
        });
    }

    resetFilters() {
        document.getElementById('searchInput').value = '';
        document.getElementById('departmentFilter').value = 'all';
        this.filteredData = [...this.originalData];
        this.renderTable();
        this.updateAnalytics();
    }
}

function sortByName(order) {
    dashboard.sortByName(order);
}

function sortByDOB(order) {
    dashboard.sortByDOB(order);
```

}

function

OUTPUT



RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 3

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login - Premium Experience</title>
  <style>
    @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

    :root {
      --primary: #6366f1;
      --primary-hover: #4f46e5;
      --bg-start: #0f172a;
      --bg-end: #1e1b4b;
      --surface: rgba(30, 41, 59, 0.7);
      --border: rgba(255, 255, 255, 0.1);
      --text-main: #f8fafc;
      --text-muted: #94a3b8;
      --error: #ef4444;
      --success: #22c55e;
```

```
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: 'Inter', sans-serif;
}

body {
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    background: linear-gradient(135deg, var(--bg-start), var(--bg-end));
    color: var(--text-main);
    padding: 1rem;
}

.login-container {
    background: var(--surface);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
    border: 1px solid var(--border);
    border-radius: 20px;
    padding: 3rem;
    width: 100%;
    max-width: 420px;
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
    transform: translateY(0);
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.login-container:hover {
    transform: translateY(-5px);
    box-shadow: 0 30px 60px -12px rgba(0, 0, 0, 0.6);
}

.header {
    text-align: center;
    margin-bottom: 2.5rem;
}

.header h1 {
    font-size: 2rem;
    font-weight: 700;
    margin-bottom: 0.5rem;
```

```
    background: linear-gradient(to right, #a5b4fc, #818cf8);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}

.header p {
    color: var(--text-muted);
    font-size: 0.95rem;
}

.input-group {
    margin-bottom: 1.5rem;
    position: relative;
}

.input-group label {
    display: block;
    margin-bottom: 0.5rem;
    font-size: 0.9rem;
    font-weight: 500;
    color: var(--text-muted);
    transition: color 0.3s;
}

.input-field {
    width: 100%;
    padding: 1rem;
    background: rgba(15, 23, 42, 0.6);
    border: 1px solid var(--border);
    border-radius: 10px;
    color: var(--text-main);
    font-size: 1rem;
    outline: none;
    transition: all 0.3s ease;
}

.input-field:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.2);
}

.input-field:focus + label,
.input-group:focus-within label {
    color: var(--primary);
}

.error-message {
```

```
    color: var(--error);
    font-size: 0.8rem;
    margin-top: 0.5rem;
    min-height: 1rem;
    display: none;
    opacity: 0;
    transition: opacity 0.3s;
}

.error-message.visible {
    display: block;
    opacity: 1;
    animation: slideDown 0.3s ease forwards;
}

.submit-btn {
    width: 100%;
    padding: 1rem;
    background: var(--primary);
    color: white;
    border: none;
    border-radius: 10px;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
    margin-top: 1rem;
}

.submit-btn:hover {
    background: var(--primary-hover);
    transform: translateY(-2px);
    box-shadow: 0 10px 20px -10px var(--primary);
}

.submit-btn:active {
    transform: translateY(0);
}

.status-message {
    margin-top: 1.5rem;
    text-align: center;
    font-size: 0.95rem;
    font-weight: 500;
    padding: 1rem;
    border-radius: 8px;
    display: none;
```

```
            opacity: 0;
            transition: all 0.3s;
        }

        .status-message.visible {
            display: block;
            opacity: 1;
            animation: slideDown 0.3s ease forwards;
        }

        .status-success {
            background: rgba(34, 197, 94, 0.1);
            color: var(--success);
            border: 1px solid rgba(34, 197, 94, 0.2);
        }

        .status-error {
            background: rgba(239, 68, 68, 0.1);
            color: var(--error);
            border: 1px solid rgba(239, 68, 68, 0.2);
        }

        @keyframes slideDown {
            from { transform: translateY(-10px); opacity: 0; }
            to { transform: translateY(0); opacity: 1; }
        }

        /* Input shaking animation for errors */
        @keyframes shake {
            0%, 100% { transform: translateX(0); }
            25% { transform: translateX(-5px); }
            75% { transform: translateX(5px); }
        }
        .input-error {
            border-color: var(--error);
            animation: shake 0.4s ease-in-out;
        }
    </style>
</head>
<body>

<div class="login-container">
    <div class="header">
        <h1>Welcome Back</h1>
        <p>Please enter your details to sign in</p>
    </div>
```

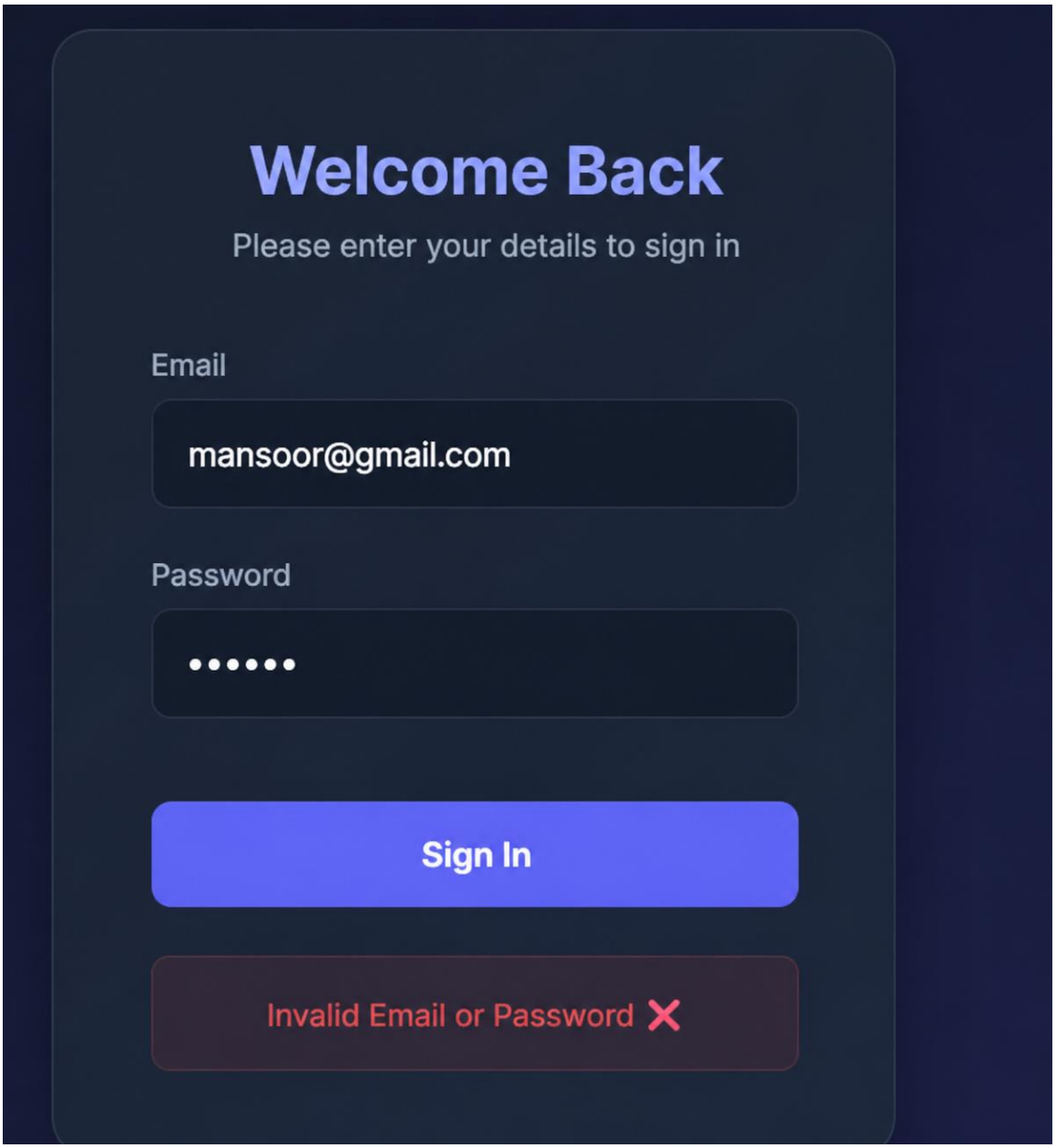
```
  <form id="loginForm" novalidate>
    <div class="input-group">
      <label for="email">Email</label>
      <input type="email" id="email" class="input-field" placeholder="Enter your email" required
autocomplete="off">
      <div id="emailError" class="error-message"></div>
    </div>

    <div class="input-group">
      <label for="password">Password</label>
      <input type="password" id="password" class="input-field" placeholder="Enter your password"
required>
      <div id="passwordError" class="error-message"></div>
    </div>

    <button type="submit" class="submit-btn" id="loginBtn">Sign In</button>
  </form>

  <div id="statusMessage" class="status-message"></div>
</div>

<script>
  // 1. Initialize Database Simulation (LocalStorage)
  // Pre-populate some dummy user credentials if not present
  if (!localStorage.getItem('users')) {
    const dummyUsers = [
      { email: 'user@example.com', password: 'password123' },
      { email: 'admin@test.com', password: 'adminpass' }
    ];
    localStorage.setItem('users', JSON.stringify(dummyUsers));
  }

  // 2. DOM Elements
  const form = document.getElementById('loginForm');
  const emailInput = document.getElementById('email');
  const passwordInput = document.getElementById('password');
  const emailError = document.getElementById('emailError');
  const passwordError = document.getElementById('passwordError');
  const statusMessage = document.getElementById('statusMessage');
  const loginBtn = document.getElementById('loginBtn');

  // 3. Validation Helpers
  const validateEmailFormat = (email) => {
    const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return regex.test(email);
  };
```

```
const showError = (inputElement, errorElement, message) => {
    errorElement.textContent = message;
    errorElement.classList.add('visible');
    inputElement.classList.add('input-error');
    // Remove animation class after it completes to allow re-triggering
    setTimeout(() => inputElement.classList.remove('input-error'), 400);
};

const clearErrors = () => {
    [emailError, passwordError].forEach(el => {
        el.textContent = '';
        el.classList.remove('visible');
    });
    [emailInput, passwordInput].forEach(el => el.classList.remove('input-error'));
    statusMessage.classList.remove('visible', 'status-success', 'status-error');
};

const showStatus = (message, isSuccess) => {
    statusMessage.textContent = message;
    statusMessage.className = `status-message visible ${isSuccess ? 'status-success' : 'status-error'}`;
};

// 4. Form Submission Handler
form.addEventListener('submit', (e) => {
    e.preventDefault(); // Prevent page reload

    // Reset errors
    clearErrors();

    const email = emailInput.value.trim();
    const password = passwordInput.value.trim();
    let isValid = true;

    // Front-end Validation
    if (!email) {
        showError(emailInput, emailError, 'Email field cannot be empty');
        isValid = false;
    } else if (!validateEmailFormat(email)) {
        showError(emailInput, emailError, 'Please enter a valid email format');
        isValid = false;
    }

    if (!password) {
        showError(passwordInput, passwordError, 'Password field cannot be empty');
        isValid = false;
    } else if (password.length < 6) {
        showError(passwordInput, passwordError, 'Password must be at least 6 characters');
```

```
        isValid = false;
    }

    // If validation fails, stop execution
    if (!isValid) {
        // Optional: Haptic feedback or visual cue on button
        loginBtn.style.transform = 'scale(0.98)';
        setTimeout(() => loginBtn.style.transform = '', 150);
        return;
    }

    // Database Check (LocalStorage)
    // Add loading state
    loginBtn.textContent = 'Verifying...';
    loginBtn.style.opacity = '0.8';
    loginBtn.disabled = true;

    // Simulate slight network delay for better UX feel
    setTimeout(() => {
        const users = JSON.parse(localStorage.getItem('users')) || [];

        const matchedUser = users.find(
            u => u.email.toLowerCase() === email.toLowerCase() && u.password === password
        );

        if (matchedUser) {
            showStatus('Login Successful! ✓', true);
            // Reset form on success
            form.reset();
        } else {
            showStatus('Invalid Email or Password ✕', false);
        }

        // Restore button state
        loginBtn.textContent = 'Sign In';
        loginBtn.style.opacity = '1';
        loginBtn.disabled = false;
    }, 600); // 600ms delay simulation
});

// Clear errors on typing
emailInput.addEventListener('input', () => {
    if(emailError.classList.contains('visible')) {
        emailError.classList.remove('visible');
        emailInput.classList.remove('input-error');
    }
});
```

```
  passwordInput.addEventListener('input', () => {
    if(passwordError.classList.contains('visible')) {
      passwordError.classList.remove('visible');
      passwordInput.classList.remove('input-error');
    }
  });

</script>
</body>
</html>
```

OUTPUT


Welcome Back
Please enter your details to sign in
Email
mansoor@gmail.com
Password
••••••
Sign In
Invalid Email or Password ❌

RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 4

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task 4: Order Management</title>
  <style>
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: #f4f7fb;
      color: #333;
      margin: 0;
      padding: 20px;
    }

    .container {
      max-width: 900px;
      margin: 0 auto;
      background: #fff;
      padding: 30px;
      border-radius: 8px;
```

```
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

h1 {
    text-align: center;
    color: #2c3e50;
    font-size: 2.2rem;
    margin-bottom: 20px;
}

h2 {
    color: #34495e;
    border-bottom: 2px solid #3498db;
    padding-bottom: 5px;
    margin-top: 40px;
    font-size: 1.5rem;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
    margin-bottom: 20px;
}

th, td {
    text-align: left;
    padding: 12px 15px;
    border-bottom: 1px solid #e1e8ee;
}

th {
    background-color: #3498db;
    color: white;
    text-transform: uppercase;
    font-size: 0.9rem;
}

tr:hover {
    background-color: #f1f5f8;
}

.badge {
    background-color: #e74c3c;
    color: white;
    padding: 5px 10px;
    border-radius: 20px;
```

```
        font-weight: bold;
        font-size: 0.8rem;
      }
    </style>
</head>
<body>
    <div class="container">
        <h1>🛒 E-commerce Order Management</h1>
        <p style="text-align: center; color: #7f8c8d; font-size: 1.1rem;">
            Demonstrating relational data with Join queries, Subqueries, and CSS layout.
        </p>

        <!-- Section 1 -->
        <h2>1. Customer Order History</h2>

        <table>
            <thead>
                <tr>
                    <th>Order ID</th>
                    <th>Customer Name</th>
                    <th>Product</th>
                    <th>Quantity</th>
                    <th>Total Price</th>
                    <th>Order Date</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>4</td>
                    <td>Charlie</td>
                    <td>Laptop</td>
                    <td>2</td>
                    <td>$2,000.00</td>
                    <td>2026-04-04</td>
                </tr>
                <tr>
                    <td>3</td>
                    <td>Alice</td>
                    <td>Headphones</td>
                    <td>1</td>
                    <td>$100.00</td>
                    <td>2026-04-03</td>
                </tr>
                <tr>
                    <td>2</td>
                    <td>Bob</td>
                    <td>Phone</td>
```

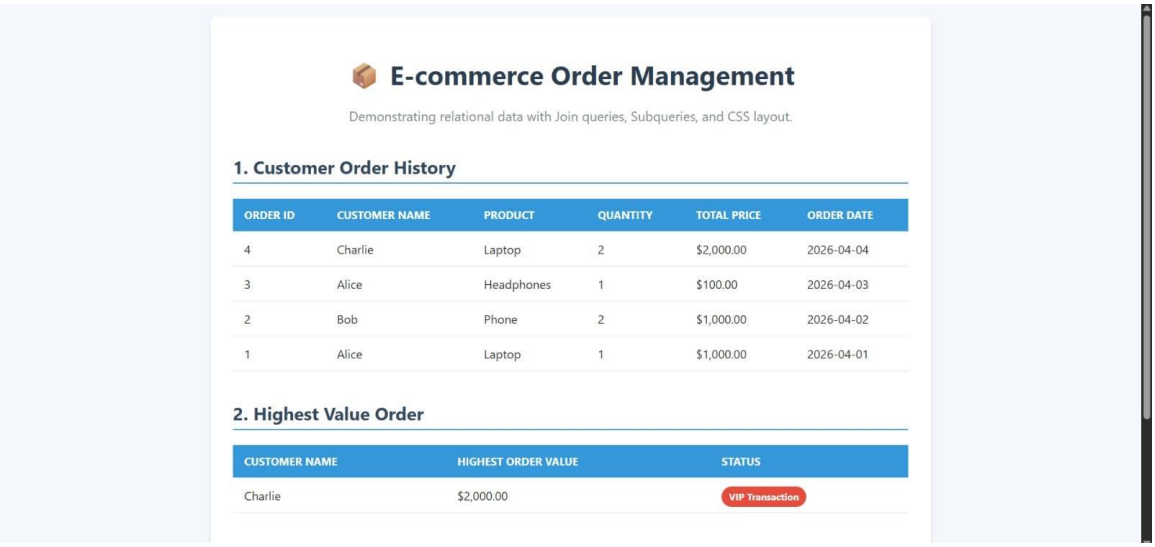
```
        <td>2</td>
        <td>$1,000.00</td>
        <td>2026-04-02</td>
      </tr>
      <tr>
        <td>1</td>
        <td>Alice</td>
        <td>Laptop</td>
        <td>1</td>
        <td>$1,000.00</td>
        <td>2026-04-01</td>
      </tr>
    </tbody>
</table>

<!-- Section 2 -->
<h2>2. Highest Value Order</h2>

<table>
    <thead>
      <tr>
        <th>Customer Name</th>
        <th>Highest Order Value</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Charlie</td>
        <td>$2,000.00</td>
        <td><span class="badge">VIP Transaction</span></td>
      </tr>
    </tbody>
</table>

<!-- Section 3 -->
<h2>3. Most Active Customer</h2>

<table>
    <thead>
      <tr>
        <th>Most Active Customer</th>
        <th>Total Number of Orders</th>
      </tr>
    </thead>
    <tbody>
      <tr>
```

```
            <td>Alice</td>
            <td>2 orders</td>
          </tr>
        </tbody>
      </table>

  </div>
</body>
</html>


// File: queries.sql
-- Create Tables

CREATE TABLE Customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);

CREATE TABLE Products (
  product_id INT PRIMARY KEY,
  name VARCHAR(100),
  price DECIMAL(10, 2)
);

CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  product_id INT,
  quantity INT,
  order_date DATE,
  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
  FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- Insert Sample Data

INSERT INTO Customers (customer_id, name, email) VALUES
(1, 'Alice', 'alice@example.com'),
(2, 'Bob', 'bob@example.com'),
(3, 'Charlie', 'charlie@example.com');

INSERT INTO Products (product_id, name, price) VALUES
(101, 'Laptop', 1000.00),
(102, 'Phone', 500.00),
(103, 'Headphones', 100.00);
```

```
INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date) VALUES
(1, 1, 101, 1, '2026-04-01'),
(2, 2, 102, 2, '2026-04-02'),
(3, 1, 103, 1, '2026-04-03'),
(4, 3, 101, 2, '2026-04-04');

-- 1. Display Customer Order History using JOINs and ORDER BY
-- Real-Time Usage: Viewing recent activity in user's profile
SELECT
   o.order_id,
   c.name AS customer_name,
   p.name AS product_name,
   o.quantity,
   p.price,
   (o.quantity * p.price) AS total_value,
   o.order_date
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id
ORDER BY o.order_date DESC;

-- 2. Find Highest Value Order using a Subquery
-- Real-Time Usage: Finding the biggest sales logic / VIP orders
SELECT
   o.order_id,
   c.name AS customer_name,
   (o.quantity * p.price) AS order_value
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id
WHERE (o.quantity * p.price) = (
   SELECT MAX(o2.quantity * p2.price)
   FROM Orders o2
   JOIN Products p2 ON o2.product_id = p2.product_id
);

-- 3. Find Most Active Customer using a Subquery and ORDER BY
-- Real-Time Usage: Finding the most engaged user for rewards
SELECT
   name,
   order_count
FROM (
   SELECT c.name, COUNT(o.order_id) AS order_count
   FROM Customers c
   JOIN Orders o ON c.customer_id = o.customer_id
   GROUP BY c.customer_id
```

) AS customer_orders
ORDER BY order_count DESC
LIMIT 1;

OUTPUT



RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 5

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Transaction Simulation</title>
  <style>
    @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&display=swap');

    :root {
      --bg-color: #f8fafc;
      --card-bg: #ffffff;
      --primary: #3b82f6;
      --success: #10b981;
      --danger: #ef4444;
      --text-main: #0f172a;
      --text-muted: #64748b;
    }

    body {
```

```
    font-family: 'Inter', sans-serif;
    background-color: var(--bg-color);
    color: var(--text-main);
    margin: 0;
    padding: 2rem;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
    align-items: center;
}

.header {
    text-align: center;
    margin-bottom: 3rem;
    margin-top: 2rem;
}

.header h1 {
    font-size: 2.5rem;
    margin: 0 0 0.5rem 0;
    color: var(--text-main);
}

.header p {
    color: var(--text-muted);
    margin: 0;
    font-size: 1.1rem;
}

.accounts-container {
    display: flex;
    gap: 2rem;
    margin-bottom: 2rem;
    width: 100%;
    max-width: 800px;
}

.account-card {
    flex: 1;
    background: var(--card-bg);
    border: 1px solid #e2e8f0;
    border-radius: 1rem;
    padding: 2rem;
    text-align: center;
    transition: transform 0.3s ease;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
}
```

```
.icon {
    font-size: 3rem;
    margin-bottom: 1rem;
}

h2 {
    margin: 0 0 1rem 0;
    font-size: 1.25rem;
    color: var(--text-main);
}

.balance {
    font-size: 2.5rem;
    font-weight: 700;
    margin: 0;
    color: var(--primary);
}

.actions-panel {
    background: var(--card-bg);
    border: 1px solid #e2e8f0;
    border-radius: 1rem;
    padding: 2rem;
    width: 100%;
    max-width: 800px;
    display: flex;
    gap: 1rem;
    align-items: flex-end;
    margin-bottom: 2rem;
    box-sizing: border-box;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
}

.input-group {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
    text-align: left;
}

label {
    color: var(--text-muted);
    font-size: 0.875rem;
    font-weight: 600;
}
```

```
input {
  background: #f1f5f9;
  border: 1px solid #cbd5e1;
  color: var(--text-main);
  padding: 1rem;
  border-radius: 0.5rem;
  font-size: 1.125rem;
  font-family: inherit;
  outline: none;
  transition: border-color 0.3s ease;
}

input:focus {
  border-color: var(--primary);
  background: #ffffff;
}

button {
  background: var(--primary);
  color: white;
  border: none;
  padding: 1rem 2rem;
  border-radius: 0.5rem;
  font-size: 1.125rem;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
  box-shadow: 0 4px 15px rgba(59, 130, 246, 0.4);
  height: 3.5rem;
}

button:hover {
  background: #2563eb;
  transform: translateY(-2px);
}

button:active {
  transform: translateY(0);
}

.logs {
  width: 100%;
  max-width: 800px;
  background: #1e293b;
  color: #f8fafc;
  border-radius: 0.5rem;
```

```
      padding: 1.5rem;
      font-family: monospace;
      height: 250px;
      overflow-y: auto;
      border: 1px solid #334155;
      box-sizing: border-box;
    }

    .log-entry {
      margin-bottom: 0.5rem;
      border-bottom: 1px solid rgba(255,255,255,0.05);
      padding-bottom: 0.25rem;
    }

    .log-success { color: #34d399; }
    .log-error { color: #f87171; }
    .log-info { color: #60a5fa; }
    .log-query { color: #c084fc; }

    @keyframes pulse-success {
      0% { box-shadow: 0 0 0 0 rgba(16, 185, 129, 0.4); }
      70% { box-shadow: 0 0 0 20px rgba(16, 185, 129, 0); }
      100% { box-shadow: 0 0 0 0 rgba(16, 185, 129, 0); }
    }

    @keyframes pulse-error {
      0% { box-shadow: 0 0 0 0 rgba(239, 68, 68, 0.4); }
      70% { box-shadow: 0 0 0 20px rgba(239, 68, 68, 0); }
      100% { box-shadow: 0 0 0 0 rgba(239, 68, 68, 0); }
    }

    .anim-success { animation: pulse-success 1s ease-out; }
    .anim-error { animation: pulse-error 1s ease-out; }
  </style>
</head>
<body>

  <div class="header">
    <h1>Transaction Simulator</h1>
    <p>Interactive web representation of SQL COMMIT and ROLLBACK</p>
  </div>

  <div class="accounts-container">
    <div class="account-card" id="user-card">
      <div class="icon">□□</div>
      <h2>User Account</h2>
      <div class="balance">₹<span id="user-balance">1000.00</span></div>
```

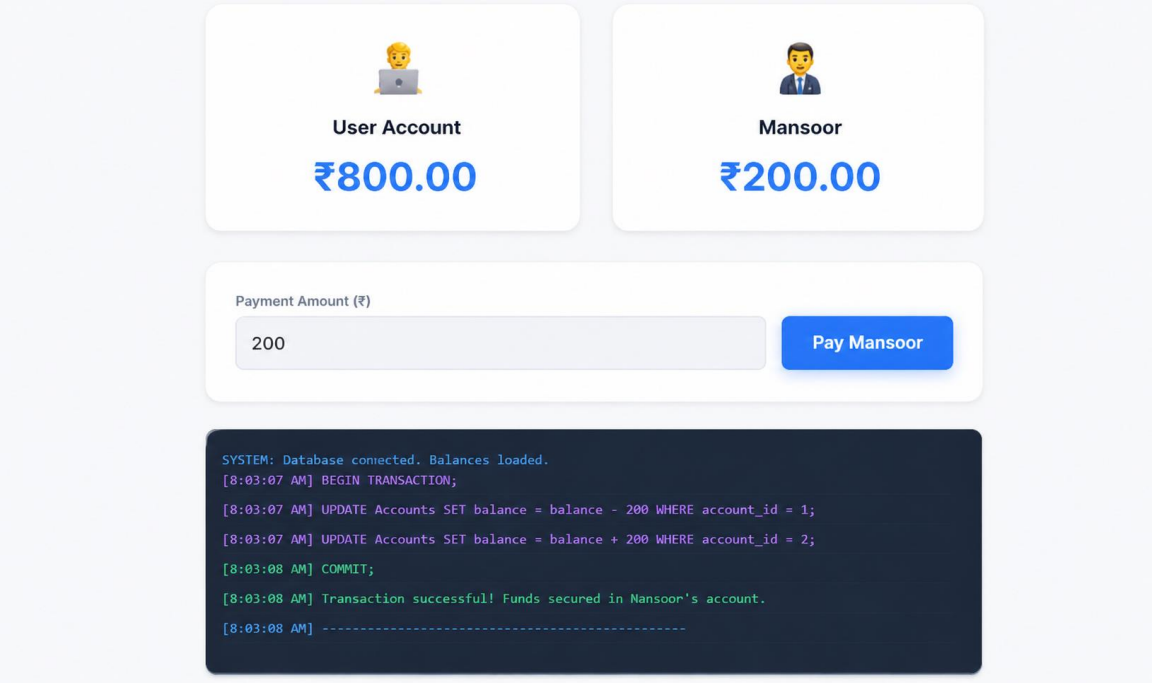
```
  </div>
  <div class="account-card" id="merchant-card">
    <div class="icon">🏪</div>
    <h2>Vandana</h2>
    <div class="balance">₹<span id="merchant-balance">0.00</span></div>
  </div>
</div>

<div class="actions-panel">
  <div class="input-group">
    <label>Payment Amount (₹)</label>
    <input type="number" id="amount" placeholder="e.g. 200" min="1" step="0.01">
  </div>
  <button onclick="processTransaction()">Pay Vandana</button>
</div>

<div class="logs" id="logs">
  <div class="log-info">SYSTEM: Database connected. Balances loaded.</div>
</div>

<script>
  // Internal state
  let db = {
    userBalance: 1000.00,
    merchantBalance: 0.00
  };

  function addLog(message, type = 'info') {
    const logs = document.getElementById('logs');
    const entry = document.createElement('div');
    entry.className = `log-entry log-${type}`;
    const time = new Date().toLocaleTimeString();
    entry.innerText = `[${time}] ${message}`;
    logs.appendChild(entry);
    logs.scrollTop = logs.scrollHeight;
  }

  function updateUI() {
    document.getElementById('user-balance').innerText = db.userBalance.toFixed(2);
    document.getElementById('merchant-balance').innerText = db.merchantBalance.toFixed(2);
  }

  function processTransaction() {
    const inputField = document.getElementById('amount');
    const inputAmount = parseFloat(inputField.value);

    if (isNaN(inputAmount) || inputAmount <= 0) {
```

```
            addLog('Please enter a valid amount.', 'error');
            return;
        }

        const userCard = document.getElementById('user-card');

        addLog('BEGIN TRANSACTION;', 'query');

        let tempUserBalance = db.userBalance - inputAmount;
        let tempMerchantBalance = db.merchantBalance + inputAmount;

        addLog(`UPDATE Accounts SET balance = balance - ${inputAmount} WHERE account_id = 1;`,
'query');
        addLog(`UPDATE Accounts SET balance = balance + ${inputAmount} WHERE account_id = 2;`,
'query');

        // Timeout to simulate network processing
        setTimeout(() => {
            if (tempUserBalance < 0) {
                addLog(`ERROR: Insufficient funds. Balance would be ₹${tempUserBalance.toFixed(2)}.`,
'error');
                addLog('ROLLBACK;', 'error');
                addLog('Transaction aborted. Values reverted to initial state.', 'info');

                userCard.classList.remove('anim-error', 'anim-success');
                void userCard.offsetWidth; // trigger reflow
                userCard.classList.add('anim-error');
            } else {
                db.userBalance = tempUserBalance;
                db.merchantBalance = tempMerchantBalance;

                addLog('COMMIT;', 'success');
                // Fixed syntax error here
                addLog("Transaction successful! Funds secured in Vandana's account.", 'success');

                userCard.classList.remove('anim-error', 'anim-success');
                void userCard.offsetWidth;
                userCard.classList.add('anim-success');

                updateUI();
            }

            addLog('------------------------------------------------', 'info');
        }, 800);
    }
  </script>
</body>
```

```
</html>


// File: payment_simulation.sql
-- Step 1: Create a table for accounts
CREATE TABLE Accounts (
   account_id INT PRIMARY KEY,
   account_name VARCHAR(100),
   balance DECIMAL(10, 2)
);

-- Step 2: Insert initial balances for the User and Merchant
INSERT INTO Accounts (account_id, account_name, balance)
VALUES
   (1, 'User Account', 1000.00),  -- User starts with ₹1000
   (2, 'Vandana', 0.00);         -- Vandana starts with ₹0

-- ========================================
-- SCENARIO 1: SUCCESSFUL PAYMENT (COMMIT)
-- ========================================
-- Simulating a ₹200 payment from the User to Vandana

BEGIN TRANSACTION; -- Start the transaction

   -- 1. Deduct balance from User account
   UPDATE Accounts
   SET balance = balance - 200.00
   WHERE account_id = 1;

   -- 2. Add amount to Vandana's account
   UPDATE Accounts
   SET balance = balance + 200.00
   WHERE account_id = 2;

-- Both steps succeeded without issues.
-- Save the changes permanently:
COMMIT;


-- ========================================
-- SCENARIO 2: FAILED PAYMENT (ROLLBACK)
-- ========================================
-- Simulating a ₹5000 payment, but the user doesn't have enough funds.
-- Let's pretend an error occurs mid-way.

BEGIN TRANSACTION; -- Start the transaction
```

```
   -- 1. Deduct balance from User account
   UPDATE Accounts
   SET balance = balance - 5000.00
   WHERE account_id = 1;

   -- 2. Add amount to Vandana's account
   UPDATE Accounts
   SET balance = balance + 5000.00
   WHERE account_id = 2;

-- Error detected! The user balance would go negative (-₹4200).
-- This violates business logic (insufficient funds).
-- Revert all changes made during this transaction:
ROLLBACK;


-- ========================================
-- View Final Balances
-- ========================================
-- In the end, only the successful ₹200 transaction was saved.
-- User should have ₹800, and Vandana should have ₹200.
SELECT * FROM Accounts;
```

OUTPUT



RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 6

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: 01_database_schema.sql
-- Task 6: Automated Logging using Triggers & Views
-- Database Schema for Audit Logging Demonstration

-- Create sample database (if using MySQL/MariaDB)
-- CREATE DATABASE audit_logging_demo;
-- USE audit_logging_demo;

-- Sample Employees table for demonstration
CREATE TABLE IF NOT EXISTS employees (
   employee_id INT PRIMARY KEY AUTO_INCREMENT,
   first_name VARCHAR(50) NOT NULL,
   last_name VARCHAR(50) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   department VARCHAR(50),
   salary DECIMAL(10,2),
   hire_date DATE,
   is_active BOOLEAN DEFAULT TRUE,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

```
-- Sample Products table for demonstration
CREATE TABLE IF NOT EXISTS products (
   product_id INT PRIMARY KEY AUTO_INCREMENT,
   product_name VARCHAR(100) NOT NULL,
   category VARCHAR(50),
   price DECIMAL(10,2),
   stock_quantity INT DEFAULT 0,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Sample Orders table for demonstration
CREATE TABLE IF NOT EXISTS orders (
   order_id INT PRIMARY KEY AUTO_INCREMENT,
   employee_id INT,
   product_id INT,
   quantity INT NOT NULL,
   order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   total_amount DECIMAL(10,2),
   status VARCHAR(20) DEFAULT 'pending',
   FOREIGN KEY (employee_id) REFERENCES employees(employee_id),
   FOREIGN KEY (product_id) REFERENCES products(product_id)
);



// File: 02_audit_log_table.sql
-- Audit Log Table Structure
-- This table will capture all INSERT and UPDATE operations on monitored tables

CREATE TABLE IF NOT EXISTS audit_log (
   log_id INT PRIMARY KEY AUTO_INCREMENT,
   table_name VARCHAR(50) NOT NULL,
   operation_type VARCHAR(10) NOT NULL, -- 'INSERT' or 'UPDATE'
   record_id INT NOT NULL,
   old_data JSON, -- For UPDATE operations, stores the old values
   new_data JSON, -- Stores the new values (for both INSERT and UPDATE)
   changed_fields JSON, -- Lists which fields were changed (for UPDATE operations)
   user_name VARCHAR(100), -- Current user performing the operation
   client_ip VARCHAR(45), -- IP address of the client
   operation_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   affected_rows INT DEFAULT 1,
   transaction_id VARCHAR(50), -- Transaction identifier for tracking
   additional_info JSON -- Extra metadata about the operation
);

-- Create indexes for better performance
CREATE INDEX idx_audit_table_name ON audit_log(table_name);
```

```
CREATE INDEX idx_audit_operation_type ON audit_log(operation_type);
CREATE INDEX idx_audit_timestamp ON audit_log(operation_timestamp);
CREATE INDEX idx_audit_record_id ON audit_log(table_name, record_id);

-- Create a table to track which tables have audit triggers
CREATE TABLE IF NOT EXISTS audit_tracked_tables (
   table_name VARCHAR(50) PRIMARY KEY,
   is_active BOOLEAN DEFAULT TRUE,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Insert the tables we want to track
INSERT IGNORE INTO audit_tracked_tables (table_name) VALUES
('employees'),
('products'),
('orders');


// File: 03_insert_trigger.sql
-- INSERT Trigger for Audit Logging
-- This trigger fires on INSERT operations and logs the new data

-- Employees table INSERT trigger
DELIMITER //
CREATE TRIGGER tr_employees_insert_audit
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
   DECLARE current_user VARCHAR(100);
   DECLARE client_ip VARCHAR(45);

   -- Get current user and client IP (MySQL specific)
   SET current_user = CURRENT_USER();
   SET client_ip = CONNECTION_ID();

   INSERT INTO audit_log (
      table_name,
      operation_type,
      record_id,
      old_data,
      new_data,
      changed_fields,
      user_name,
      client_ip,
      transaction_id,
      additional_info
```

```
  ) VALUES (
    'employees',
    'INSERT',
    NEW.employee_id,
    NULL, -- No old data for INSERT
    JSON_OBJECT(
      'employee_id', NEW.employee_id,
      'first_name', NEW.first_name,
      'last_name', NEW.last_name,
      'email', NEW.email,
      'department', NEW.department,
      'salary', NEW.salary,
      'hire_date', NEW.hire_date,
      'is_active', NEW.is_active,
      'created_at', NEW.created_at,
      'updated_at', NEW.updated_at
    ),
    NULL, -- No changed fields for INSERT
    current_user,
    client_ip,
    CONNECTION_ID(),
    JSON_OBJECT('trigger_type', 'INSERT', 'table', 'employees')
  );
END//
DELIMITER ;

-- Products table INSERT trigger
DELIMITER //
CREATE TRIGGER tr_products_insert_audit
AFTER INSERT ON products
FOR EACH ROW
BEGIN
  DECLARE current_user VARCHAR(100);
  DECLARE client_ip VARCHAR(45);

  SET current_user = CURRENT_USER();
  SET client_ip = CONNECTION_ID();

  INSERT INTO audit_log (
    table_name,
    operation_type,
    record_id,
    old_data,
    new_data,
    changed_fields,
    user_name,
    client_ip,
```

```
    transaction_id,
    additional_info
  ) VALUES (
    'products',
    'INSERT',
    NEW.product_id,
    NULL,
    JSON_OBJECT(
      'product_id', NEW.product_id,
      'product_name', NEW.product_name,
      'category', NEW.category,
      'price', NEW.price,
      'stock_quantity', NEW.stock_quantity,
      'created_at', NEW.created_at,
      'updated_at', NEW.updated_at
    ),
    NULL,
    current_user,
    client_ip,
    CONNECTION_ID(),
    JSON_OBJECT('trigger_type', 'INSERT', 'table', 'products')
  );
END//
DELIMITER ;

-- Orders table INSERT trigger
DELIMITER //
CREATE TRIGGER tr_orders_insert_audit
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
  DECLARE current_user VARCHAR(100);
  DECLARE client_ip VARCHAR(45);

  SET current_user = CURRENT_USER();
  SET client_ip = CONNECTION_ID();

  INSERT INTO audit_log (
    table_name,
    operation_type,
    record_id,
    old_data,
    new_data,
    changed_fields,
    user_name,
    client_ip,
    transaction_id,
```

```
    additional_info
  ) VALUES (
    'orders',
    'INSERT',
    NEW.order_id,
    NULL,
    JSON_OBJECT(
      'order_id', NEW.order_id,
      'employee_id', NEW.employee_id,
      'product_id', NEW.product_id,
      'quantity', NEW.quantity,
      'order_date', NEW.order_date,
      'total_amount', NEW.total_amount,
      'status', NEW.status
    ),
    NULL,
    current_user,
    client_ip,
    CONNECTION_ID(),
    JSON_OBJECT('trigger_type', 'INSERT', 'table', 'orders')
  );
END//
DELIMITER ;


// File: 04_update_trigger.sql
-- UPDATE Trigger for Audit Logging
-- This trigger fires on UPDATE operations and logs the changes

-- Employees table UPDATE trigger
DELIMITER //
CREATE TRIGGER tr_employees_update_audit
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
  DECLARE current_user VARCHAR(100);
  DECLARE client_ip VARCHAR(45);
  DECLARE changed_fields JSON;

  SET current_user = CURRENT_USER();
  SET client_ip = CONNECTION_ID();

  -- Determine which fields have changed
  SET changed_fields = JSON_ARRAY(
    IF(NEW.first_name <> OLD.first_name, 'first_name', NULL),
    IF(NEW.last_name <> OLD.last_name, 'last_name', NULL),
    IF(NEW.email <> OLD.email, 'email', NULL),
```

```
    IF(NEW.department <> OLD.department, 'department', NULL),
    IF(NEW.salary <> OLD.salary, 'salary', NULL),
    IF(NEW.hire_date <> OLD.hire_date, 'hire_date', NULL),
    IF(NEW.is_active <> OLD.is_active, 'is_active', NULL)
);

-- Remove NULL values from the array
SET changed_fields = JSON_REMOVE(changed_fields, '$[0]', '$[1]', '$[2]', '$[3]', '$[4]', '$[5]', '$[6]');

INSERT INTO audit_log (
    table_name,
    operation_type,
    record_id,
    old_data,
    new_data,
    changed_fields,
    user_name,
    client_ip,
    transaction_id,
    additional_info
) VALUES (
    'employees',
    'UPDATE',
    NEW.employee_id,
    JSON_OBJECT(
        'employee_id', OLD.employee_id,
        'first_name', OLD.first_name,
        'last_name', OLD.last_name,
        'email', OLD.email,
        'department', OLD.department,
        'salary', OLD.salary,
        'hire_date', OLD.hire_date,
        'is_active', OLD.is_active,
        'created_at', OLD.created_at,
        'updated_at', OLD.updated_at
    ),
    JSON_OBJECT(
        'employee_id', NEW.employee_id,
        'first_name', NEW.first_name,
        'last_name', NEW.last_name,
        'email', NEW.email,
        'department', NEW.department,
        'salary', NEW.salary,
        'hire_date', NEW.hire_date,
        'is_active', NEW.is_active,
        'created_at', NEW.created_at,
        'updated_at', NEW.updated_at
```

```
    ),
    changed_fields,
    current_user,
    client_ip,
    CONNECTION_ID(),
    JSON_OBJECT('trigger_type', 'UPDATE', 'table', 'employees')
  );
END//
DELIMITER ;

-- Products table UPDATE trigger
DELIMITER //
CREATE TRIGGER tr_products_update_audit
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
  DECLARE current_user VARCHAR(100);
  DECLARE client_ip VARCHAR(45);
  DECLARE changed_fields JSON;

  SET current_user = CURRENT_USER();
  SET client_ip = CONNECTION_ID();

  -- Determine which fields have changed
  SET changed_fields = JSON_ARRAY(
    IF(NEW.product_name <> OLD.product_name, 'product_name', NULL),
    IF(NEW.category <> OLD.category, 'category', NULL),
    IF(NEW.price <> OLD.price, 'price', NULL),
    IF(NEW.stock_quantity <> OLD.stock_quantity, 'stock_quantity', NULL)
  );

  -- Remove NULL values from the array
  SET changed_fields = JSON_REMOVE(changed_fields, '$[0]', '$[1]', '$[2]', '$[3]');

  INSERT INTO audit_log (
    table_name,
    operation_type,
    record_id,
    old_data,
    new_data,
    changed_fields,
    user_name,
    client_ip,
    transaction_id,
    additional_info
  ) VALUES (
    'products',
```

```
      'UPDATE',
      NEW.product_id,
      JSON_OBJECT(
        'product_id', OLD.product_id,
        'product_name', OLD.product_name,
        'category', OLD.category,
        'price', OLD.price,
        'stock_quantity', OLD.stock_quantity,
        'created_at', OLD.created_at,
        'updated_at', OLD.updated_at
      ),
      JSON_OBJECT(
        'product_id', NEW.product_id,
        'product_name', NEW.product_name,
        'category', NEW.category,
        'price', NEW.price,
        'stock_quantity', NEW.stock_quantity,
        'created_at', NEW.created_at,
        'updated_at', NEW.updated_at
      ),
      changed_fields,
      current_user,
      client_ip,
      CONNECTION_ID(),
      JSON_OBJECT('trigger_type', 'UPDATE', 'table', 'products')
    );
END//
DELIMITER ;

-- Orders table UPDATE trigger
DELIMITER //
CREATE TRIGGER tr_orders_update_audit
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    DECLARE current_user VARCHAR(100);
    DECLARE client_ip VARCHAR(45);
    DECLARE changed_fields JSON;

    SET current_user = CURRENT_USER();
    SET client_ip = CONNECTION_ID();

    -- Determine which fields have changed
    SET changed_fields = JSON_ARRAY(
      IF(NEW.employee_id <> OLD.employee_id, 'employee_id', NULL),
      IF(NEW.product_id <> OLD.product_id, 'product_id', NULL),
      IF(NEW.quantity <> OLD.quantity, 'quantity', NULL),
```

```
    IF(NEW.total_amount <> OLD.total_amount, 'total_amount', NULL),
    IF(NEW.status <> OLD.status, 'status', NULL)
  );

  -- Remove NULL values from the array
  SET changed_fields = JSON_REMOVE(changed_fields, '$[0]', '$[1]', '$[2]', '$[3]', '$[4]');

  INSERT INTO audit_log (
    table_name,
    operation_type,
    record_id,
    old_data,
    new_data,
    changed_fields,
    user_name,
    client_ip,
    transaction_id,
    additional_info
  ) VALUES (
    'orders',
    'UPDATE',
    NEW.order_id,
    JSON_OBJECT(
      'order_id', OLD.order_id,
      'employee_id', OLD.employee_id,
      'product_id', OLD.product_id,
      'quantity', OLD.quantity,
      'order_date', OLD.order_date,
      'total_amount', OLD.total_amount,
      'status', OLD.status
    ),
    JSON_OBJECT(
      'order_id', NEW.order_id,
      'employee_id', NEW.employee_id,
      'product_id', NEW.product_id,
      'quantity', NEW.quantity,
      'order_date', NEW.order_date,
      'total_amount', NEW.total_amount,
      'status', NEW.status
    ),
    changed_fields,
    current_user,
    client_ip,
    CONNECTION_ID(),
    JSON_OBJECT('trigger_type', 'UPDATE', 'table', 'orders')
  );
END//
```

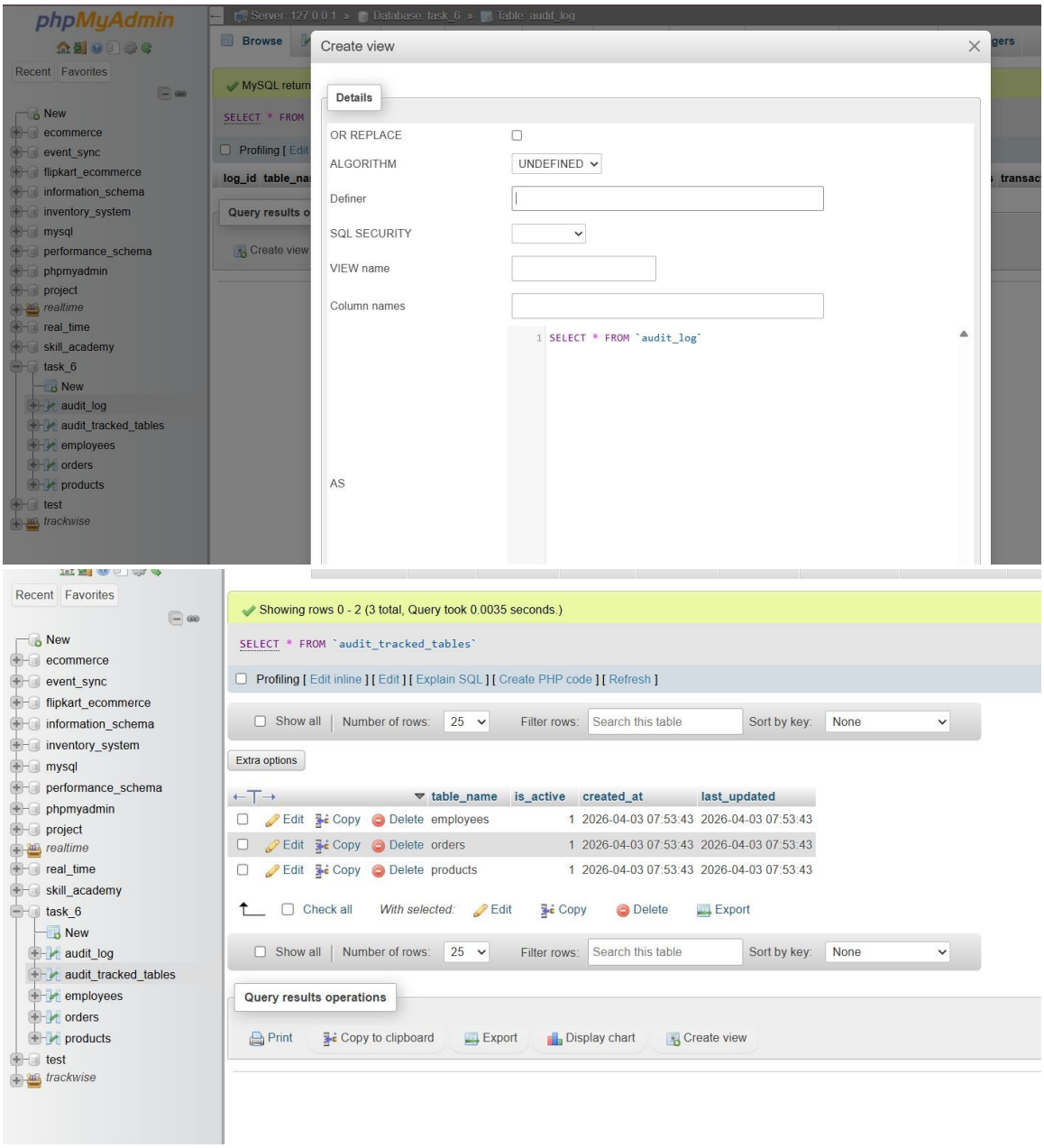
```
DELIMITER ;


// File: 05_daily_activity_view.sql
-- Daily Activity Report Views
-- These views provide comprehensive audit reporting capabilities

-- View 1: Daily Activity Summary
CREATE OR REPLACE VIEW daily_activity_summary AS
SELECT
   DATE(operation_timestamp) as activity_date,
   table_name,
   operation_type,
   COUNT(*) as operation_count,
   COUNT(DISTINCT user_name) as unique_users,
   MIN(operation_timestamp) as first_activity,
   MAX(operation_timestamp) as last_activity
FROM audit_log
GROUP BY DATE(operation_timestamp), table_name, operation_type
ORDER BY activity_date DESC, table_name, operation_type;

-- View 2: Detailed Daily Activity Log
CREATE OR REPLACE VIEW daily_activity_detail AS
SELECT
   DATE(operation_timestamp) as activity_date,
   operation_timestamp,
   table_name,
   operation_type,
   record_id,
   user_name,
   client_ip,
   CASE
      WHEN operation_type = 'INSERT' THEN
         JSON_EXTRACT(new_data, '$.first_name') or
         JSON_EXTRACT(new_data, '$.product_name') or
         CONCAT('Order #', JSON_EXTRACT(new_data, '$.order_id'))
      WHEN operation_type = 'UPDATE' THEN
         CONCAT('Updated fields: ',
              IFNULL(JSON_UNQUOTE(changed_fields), 'None'))
   END as description,
   changed_fields,
   transaction_id
FROM audit_log
ORDER BY activity_date DESC, operation_timestamp DESC;

-- View 3: User Activity Analysis
CREATE OR REPLACE VIEW user_activity_analysis AS
```

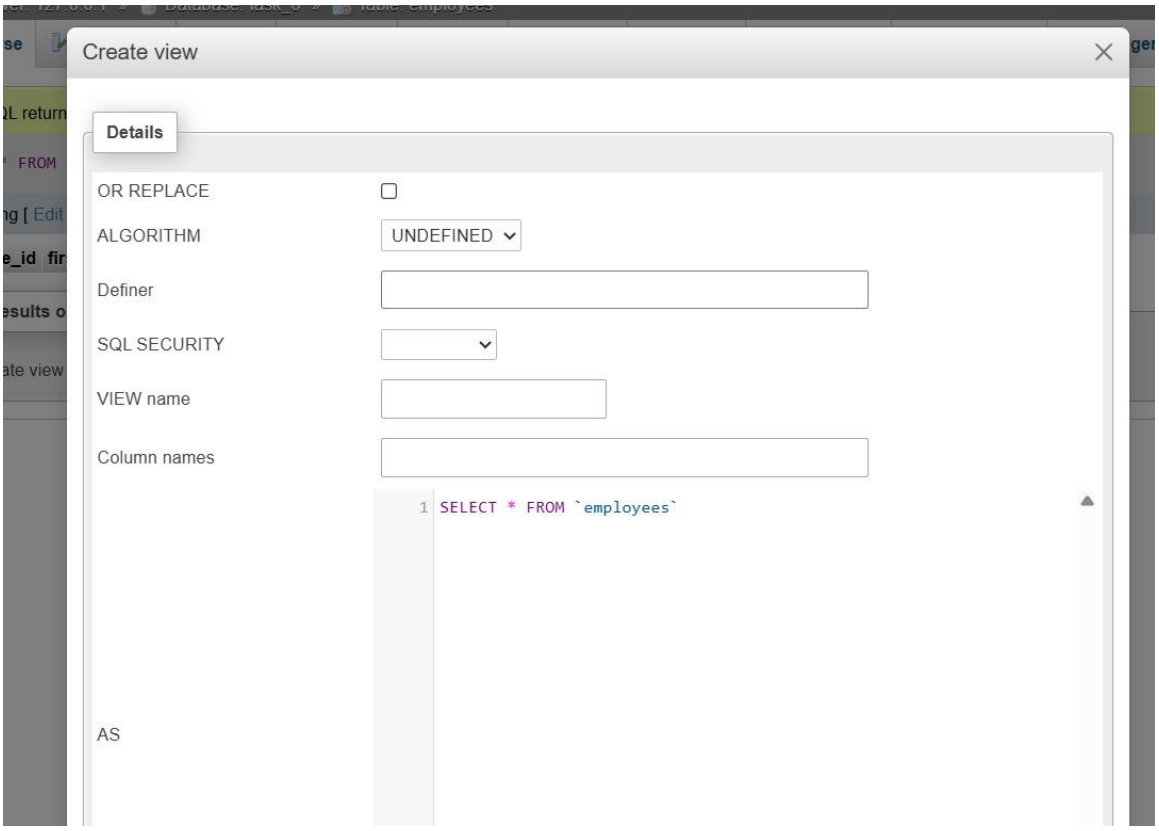
```
SELECT
  user_name,
  DATE(operation_timestamp) as activity_date,
  table_name,
  operation_type,
  COUNT(*) as operations_count,
  COUNT(DISTINCT record_id) as records_affected,
  GROUP_CONCAT(DISTINCT record_id) as affected_record_ids
FROM audit_log
WHERE user_name IS NOT NULL
GROUP BY user_name, DATE(operation_timestamp), table_name, operation_type
ORDER BY activity_date DESC, operations_count DESC;

-- View 4: Table Activity Heatmap
CREATE OR REPLACE VIEW table_activity_heatmap AS
SELECT
  table_name,
  DATE(operation_timestamp) as activity_date,
  operation_type,
  COUNT(*) as operation_count,
  ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (PARTITION BY DATE(operation_timestamp)), 2)
as percentage_of_daily_activity
FROM audit_log
GROUP BY table_name, DATE(operation_timestamp), operation_type
ORDER BY activity_date DESC, operation_count DESC;

-- View 5: Field Change Analysis (for UPDATE operations)
CREATE OR REPLACE VIEW field_change_analysis AS
SELECT
  DATE(operation_timestamp) as activity_date,
  table_name,
  changed_field,
  COUNT(*) as change_count,
  COUNT(DISTINCT record_id) as unique_records_changed,
  COUNT(DISTINCT user_name) as unique_users_making_changes
FROM (
  SELECT
    operation_timestamp,
    table_name,
    JSON_UNQUOTE(JSON_EXTRACT(changed_fields, CONCAT('$[', idx, ']'))) as changed_field,
    record_id,
    user_name
  FROM audit_log
  CROSS JOIN (
    SELECT 0 as idx UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION
    SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7
  ) as numbers
```

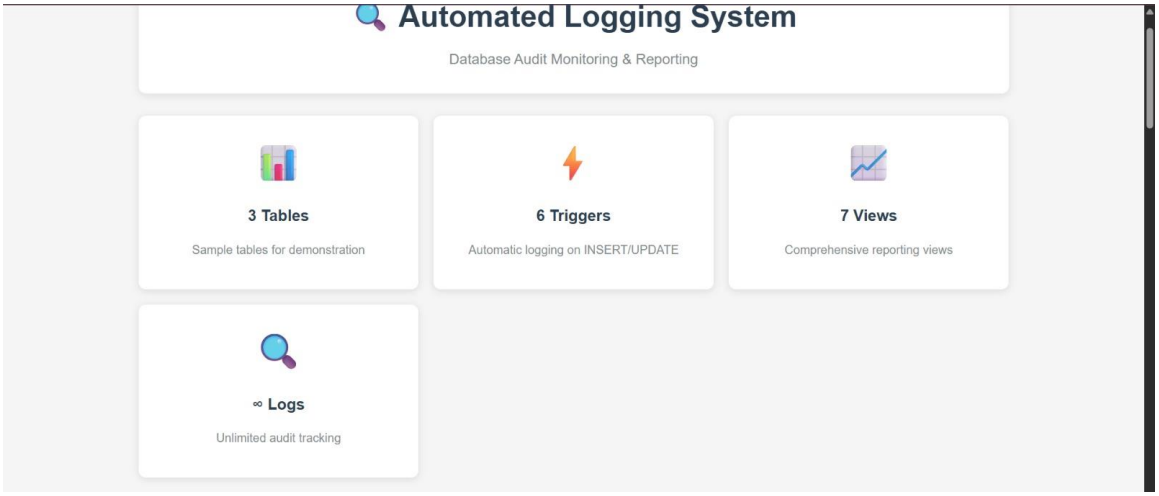
```
  WHERE operation_type = 'UPDATE'
  AND changed_fields IS NOT NULL
  AND JSON_EXTRACT(changed_fields, CONCAT('$[', idx, ']')) IS NOT NULL
) as field_changes
GROUP BY activity_date, table_name, changed_field
ORDER BY activity_date DESC, change_count DESC;

-- View 6: Recent Activity Dashboard
CREATE OR REPLACE VIEW recent_activity_dashboard AS
SELECT
  operation_timestamp,
  table_name,
  operation_type,
  record_id,
  user_name,
  CASE
    WHEN operation_type = 'INSERT' THEN '□'
    WHEN operation_type = 'UPDATE' THEN '□'
    ELSE '○'
  END as operation_icon,
  CASE
    WHEN operation_type = 'INSERT' THEN
      CONCAT('New ', LOWER(table_name), ' added')
    WHEN operation_type = 'UPDATE' THEN
      CONCAT(LOWER(table_name), ' updated')
  END as activity_description,
  TIMESTAMPDIFF(MINUTE, operation_timestamp, NOW()) as minutes_ago
FROM audit_log
WHERE operation_timestamp >= DATE_SUB(NOW(), INTERVAL 24 HOUR)
ORDER BY operation_timestamp DESC;

-- View 7: Daily Statistics Report
CREATE OR REPLACE VIEW daily_statistics_report AS
SELECT
  DATE(operation_timestamp) as report_date,
  COUNT(*) as total_operations,
  COUNT(DISTINCT table_name) as tables_affected,
  COUNT(DISTINCT user_name) as active_users,
  SUM(CASE WHEN operation_type = 'INSERT' THEN 1 ELSE 0 END) as insert_operations,
  SUM(CASE WHEN operation_type = 'UPDATE' THEN 1 ELSE 0 END) as update_operations,
  COUNT(DISTINCT record_id) as total_records_affected,
  ROUND(AVG(CASE WHEN operation_type = 'UPDATE' THEN
    JSON_LENGTH(changed_fields) ELSE 0 END), 2) as avg_fields_changed_per_update
FROM audit_log
GROUP BY DATE(operation_timestamp)
ORDER BY report_date DESC;
```

```
// File: 06_sample_data_demo.sql
-- Sample Data and Demonstration Script
-- This script populates the database with sample data and demonstrates the audit logging

-- Insert sample employees (these will trigger INSERT audit logs)
INSERT INTO employees (first_name, last_name, email, department, salary, hire_date) VALUES
('John', 'Smith', 'john.smith@company.com', 'IT', 75000.00, '2023-01-15'),
('Sarah', 'Johnson', 'sarah.johnson@company.com', 'Sales', 65000.00, '2023-02-20'),
('Michael', 'Brown', 'michael.brown@company.com', 'Marketing', 70000.00, '2023-03-10'),
('Emily', 'Davis', 'emily.davis@company.com', 'HR', 60000.00, '2023-04-05'),
('David', 'Wilson', 'david.wilson@company.com', 'IT', 80000.00, '2023-05-12');

-- Insert sample products (these will trigger INSERT audit logs)
INSERT INTO products (product_name, category, price, stock_quantity) VALUES
('Laptop Pro 15', 'Electronics', 1299.99, 50),
('Wireless Mouse', 'Electronics', 29.99, 200),
('Office Chair Deluxe', 'Furniture', 349.99, 25),
('Standing Desk', 'Furniture', 599.99, 15),
('Coffee Maker', 'Appliances', 89.99, 30),
('Monitor 27"', 'Electronics', 399.99, 40);

-- Insert sample orders (these will trigger INSERT audit logs)
INSERT INTO orders (employee_id, product_id, quantity, total_amount, status) VALUES
(1, 1, 2, 2599.98, 'completed'),
(2, 3, 1, 349.99, 'pending'),
(3, 2, 5, 149.95, 'completed'),
(4, 4, 1, 599.99, 'shipped'),
(5, 5, 2, 179.98, 'pending'),
(1, 6, 3, 1199.97, 'completed');

-- Now perform some UPDATE operations to demonstrate the UPDATE triggers
UPDATE employees SET salary = 78000.00 WHERE employee_id = 1;
UPDATE employees SET department = 'Senior IT' WHERE employee_id = 1;
UPDATE products SET stock_quantity = 45, price = 1249.99 WHERE product_id = 1;
UPDATE order
```

OUTPUT


phpMyAdmin
Server: 127.0.0.1 » Database: task_6 » Table: audit_log
Browse
Create view
Details
OR REPLACE
ALGORITHM
UNDEFINED
Definer
SQL SECURITY
VIEW name
Column names
1 SELECT * FROM `audit_log`
AS
Recent
Favorites
New
ecommerce
event_sync
flipkart_ecommerce
information_schema
inventory_system
mysql
performance_schema
phpmyadmin
project
realtime
real_time
skill_academy
task_6
New
audit_log
audit_tracked_tables
employees
orders
products
test
trackwise
Showing rows 0 - 2 (3 total, Query took 0.0035 seconds.)
SELECT * FROM `audit_tracked_tables`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all
Number of rows: 25
Filter rows: Search this table
Sort by key: None
Extra options
table_name
is_active
created_at
last_updated
Edit Copy Delete employees 1 2026-04-03 07:53:43 2026-04-03 07:53:43
Edit Copy Delete orders 1 2026-04-03 07:53:43 2026-04-03 07:53:43
Edit Copy Delete products 1 2026-04-03 07:53:43 2026-04-03 07:53:43
Check all
With selected:
Edit
Copy
Delete
Export
Query results operations
Print
Copy to clipboard
Export
Display chart
Create view





RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 7

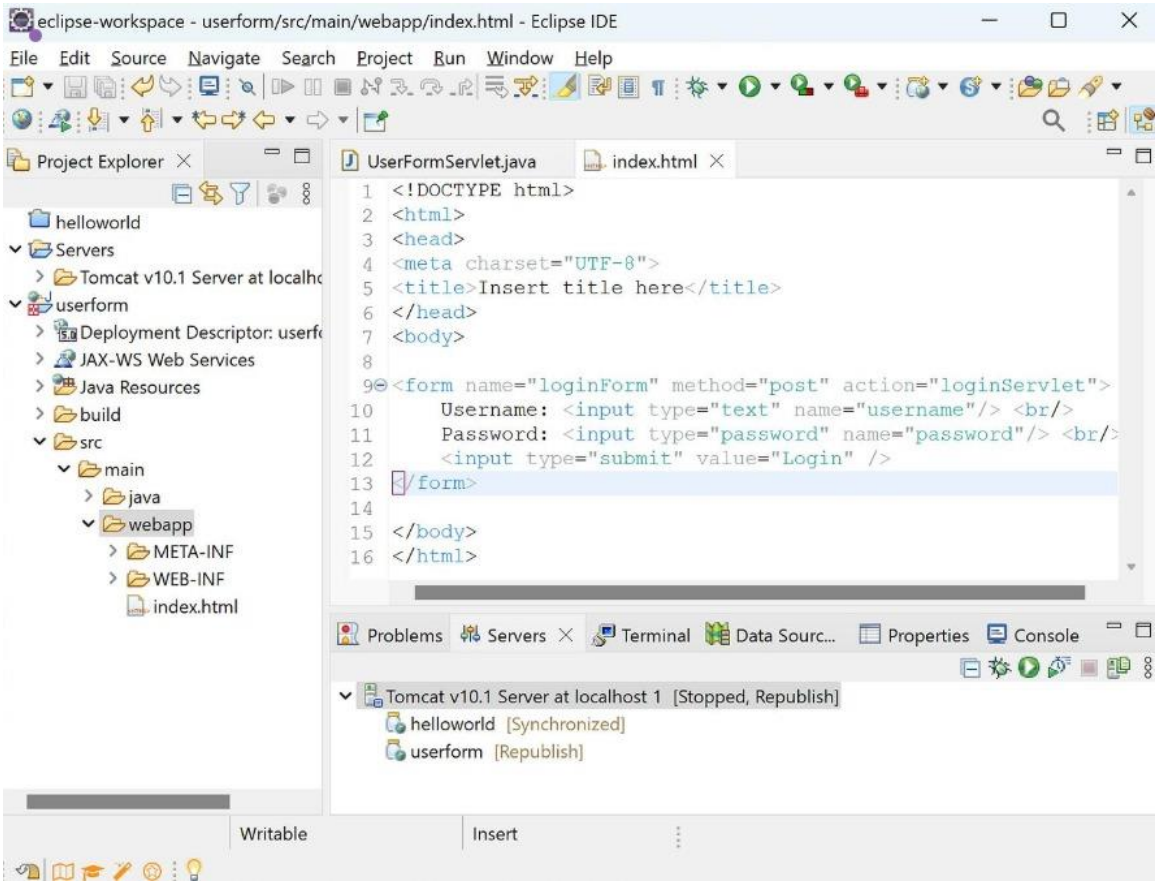
AIM

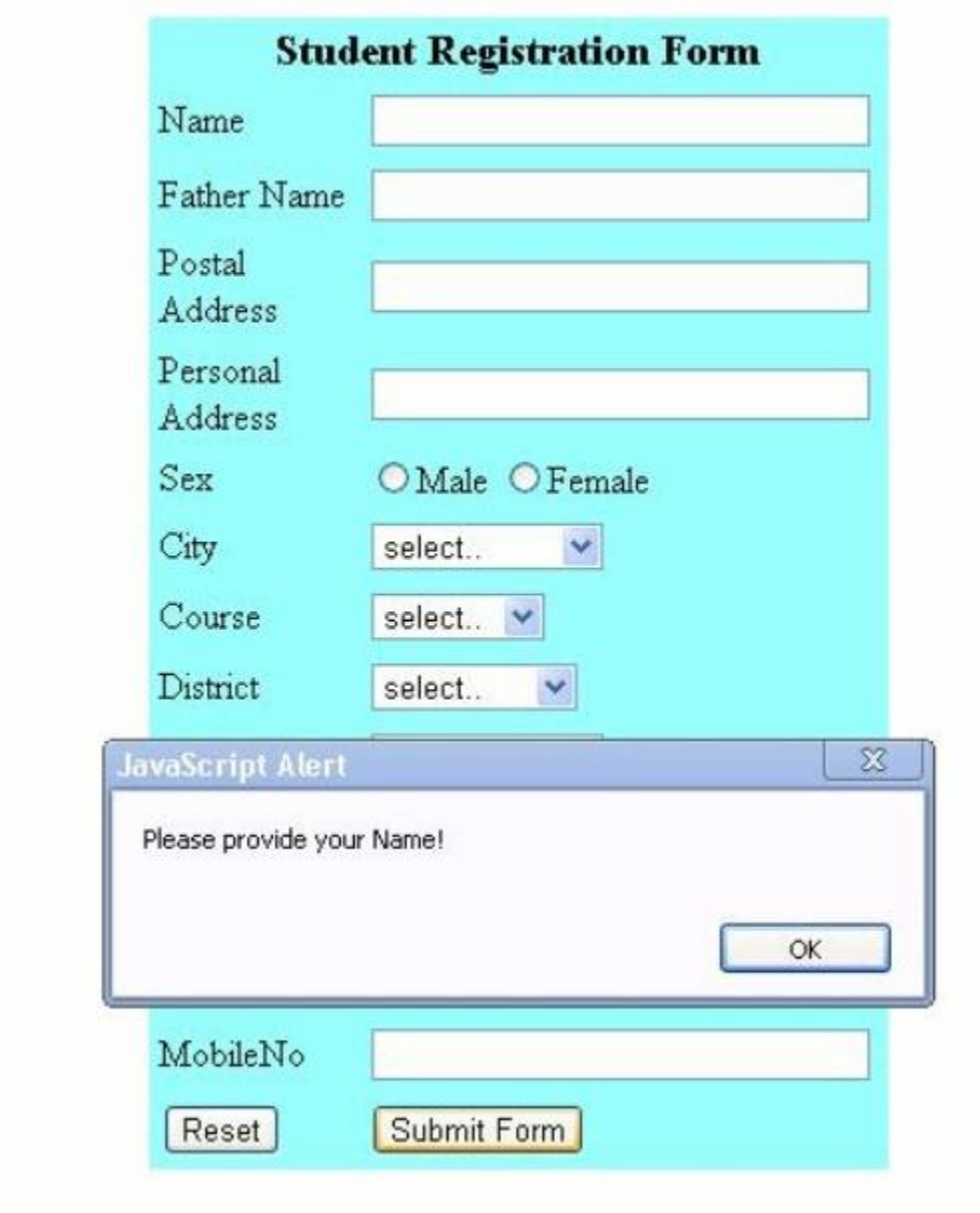
To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

OUTPUT





RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 8

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: pom.xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
      https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>spring-employee-core</artifactId>
 <version>1.0</version>

 <properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
 </properties>

 <dependencies>
  <!-- Spring Core + IoC container -->
  <dependency>
   <groupId>org.springframework</groupId>
   <artifactId>spring-context</artifactId>
   <version>5.3.30</version>
  </dependency>
```

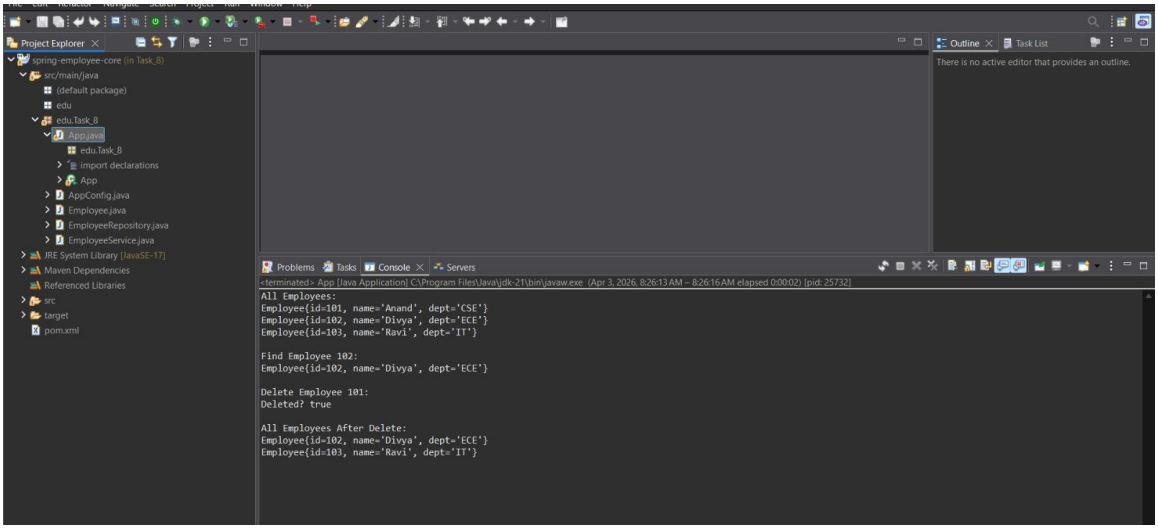
```
 </dependencies>
</project>


// File: App.java
package edu.Task_8;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class App
{
   public static void main(String[] args)
   {
          ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);

           // ✅Get service bean (DI happens automatically)
      EmployeeService service = context.getBean(EmployeeService.class);

      // --- Demo operations ---
      service.addEmployee(101, "Anand", "CSE");
      service.addEmployee(102, "Divya", "ECE");
      service.addEmployee(103, "Ravi", "IT");

      System.out.println("All Employees:");
      service.getAllEmployees().forEach(System.out::println);

      System.out.println("\nFind Employee 102:");
      System.out.println(service.getEmployee(102));

      System.out.println("\nDelete Employee 101:");
      System.out.println("Deleted? " + service.removeEmployee(101));

      System.out.println("\nAll Employees After Delete:");
      service.getAllEmployees().forEach(System.out::println);
   }
}


// File: AppConfig.java
package edu.Task_8;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "edu.Task_8")
```

```
public class AppConfig { }


// File: Employee.java
package edu.Task_8;

public class Employee {
        private int id;
  private String name;
  private String department;

  public Employee() { }

  public Employee(int id, String name, String department) {
    this.id = id;
    this.name = name;
    this.department = department;
  }

  public int getId() { return id; }
  public void setId(int id) { this.id = id; }

  public String getName() { return name; }
  public void setName(String name) { this.name = name; }

  public String getDepartment() { return department; }
  public void setDepartment(String department) { this.department = department; }

  @Override
  public String toString() {
    return "Employee{id=" + id + ", name='" + name + "', dept='" + department + "'}";
  }
}


// File: EmployeeRepository.java
package edu.Task_8;

import java.util.*;
import org.springframework.stereotype.Component;

@Component
public class EmployeeRepository {
        private final Map<Integer, Employee> store = new HashMap<>();

  public void save(Employee e) {
    store.put(e.getId(), e);
```

```
    }

    public Employee findById(int id) {
        return store.get(id);
    }

    public List<Employee> findAll() {
        return new ArrayList<>(store.values());
    }

    public boolean deleteById(int id) {
        return store.remove(id) != null;
    }
}


// File: EmployeeService.java
package edu.Task_8;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
public class EmployeeService {

    private final EmployeeRepository repo; // Dependency

    // ✅Constructor Injection (preferred)
    @Autowired
    public EmployeeService(EmployeeRepository repo) {
        this.repo = repo;
    }

    public void addEmployee(int id, String name, String dept) {
        repo.save(new Employee(id, name, dept));
    }

    public Employee getEmployee(int id) {
        return repo.findById(id);
    }

   public List<Employee> getAllEmployees() {
        return repo.findAll();
    }
```

```
    public boolean removeEmployee(int id) {
        return repo.deleteById(id);
    }
}
```

OUTPUT



RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 9

AIM

To design and implement the given task using appropriate technologies, ensuring proper functionality and accurate output generation.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new project and organize folders/files properly based on the chosen technology.
4. Design the user interface or database schema according to the task requirements.
5. Create database tables or entities to store and manage data effectively.
6. Write the program logic including validation, processing, and required functionalities.
7. Integrate frontend and backend components to ensure smooth data flow.
8. Compile and run the application to check for errors and ensure proper execution.
9. Test the application with different inputs to verify correctness and performance.
10. Display the output and confirm expected results, then successfully complete the task.

CODE

```
// File: pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-parent</artifactId>
                <version>3.2.4</version>
                <relativePath/> <!-- lookup parent from repository -->
        </parent>
        <groupId>com.example</groupId>
        <artifactId>Task_9</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <name>Task_9</name>
        <description>Demo project for Spring Boot</description>
        <url/>
        <licenses><license/></licenses>
        <developers><developer/></developers>
        <scm><connection/><developerConnection/><tag/><url/></scm>
```

```
	<properties>
		<java.version>17</java.version>
	</properties>
	<dependencies>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-thymeleaf</artifactId>
		</dependency>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-web</artifactId>
		</dependency>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-test</artifactId>
			<scope>test</scope>
		</dependency>
	</dependencies>
	<build>
		<plugins>
			<plugin>
				<groupId>org.springframework.boot</groupId>
				<artifactId>spring-boot-maven-plugin</artifactId>
			</plugin>
		</plugins>
	</build>
</project>


// File: Employee.java
package edu.Task_9;

public class Employee {
   private int id;
   private String name;
   private String dept;

   public Employee() {}
   public Employee(int id, String name, String dept) {
      this.id = id;
      this.name = name;
      this.dept = dept;
   }

   public int getId() { return id; }
   public void setId(int id) { this.id = id; }
```

```
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDept() { return dept; }
    public void setDept(String dept) { this.dept = dept; }
}


// File: EmployeeController.java
package edu.Task_9;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
public class EmployeeController {

    @Autowired
    private EmployeeRepository repo;

    // Home page to enter employee id
    @GetMapping("/")
    public String home() {
        return "home";
    }

    // MVC flow: request -> controller -> model -> view
    @GetMapping("/employee")
    public String getEmployee(@RequestParam("id") int id, Model model) {
        Employee emp = repo.findById(id);

        if (emp == null) {
            model.addAttribute("error", "Employee not found for ID: " + id);
            return "employee";
        }

        model.addAttribute("emp", emp);
        return "employee";
    }
}


// File: EmployeeRepository.java
package edu.Task_9;
```

```
import org.springframework.stereotype.Component;

import java.util.Map;

@Component
public class EmployeeRepository {

   // In-memory employee data
   private final Map<Integer, Employee> store = Map.of(
         101, new Employee(101, "Anand", "CSE"),
         102, new Employee(102, "Divya", "ECE"),
         103, new Employee(103, "Ravi", "IT")
   );

   public Employee findById(int id) {
      return store.get(id);
   }
}


// File: Task9Application.java
package edu.Task_9;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Task9Application {
            public static void main(String[] args) {
                        SpringApplication.run(Task9Application.class, args);
            }
}


// File: application.properties
server.port=8081


// File: employee.html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head><title>Employee Details</title></head>
<body>

  <h2>Employee Details</h2>

  <p th:if="${error != null}" th:text="${error}" style="color:red;"></p>
```

```
 <div th:if="${emp != null}">
  <p><b>ID:</b> <span th:text="${emp.id}"></span></p>
  <p><b>Name:</b> <span th:text="${emp.name}"></span></p>
  <p><b>Department:</b> <span th:text="${emp.dept}"></span></p>
 </div>

 <a href="/">Back</a>

</body>
</html>


// File: home.html
<!DOCTYPE html>
<html>
<head><title>Employee Search</title></head>
<body>
 <h2>Employee Management (Spring MVC)</h2>

 <form action="/employee" method="get">
  Enter Employee ID:
  <input type="number" name="id" required />
  <button type="submit">Search</button>
 </form>

 <p>Try IDs: 101, 102, 103</p>
</body>
</html>


// File: application.properties
server.port=8081


// File: employee.html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head><title>Employee Details</title></head>
<body>

 <h2>Employee Details</h2>

 <p th:if="${error != null}" th:text="${error}" style="color:red;"></p>

 <div th:if="${emp != null}">
  <p><b>ID:</b> <span th:text="${emp.id}"></span></p>
```

```
  <p><b>Name:</b> <span th:text="${emp.name}"></span></p>
  <p><b>Department:</b> <span th:text="${emp.dept}"></span></p>
 </div>

 <a href="/">Back</a>

</body>
</html>


// File: home.html
<!DOCTYPE html>
<html>
<head><title>Employee Search</title></head>
<body>
 <h2>Employee Management (Spring MVC)</h2>

 <form action="/employee" method="get">
  Enter Employee ID:
  <input type="number" name="id" required />
  <button type="submit">Search</button>
 </form>

 <p>Try IDs: 101, 102, 103</p>
</body>
</html>


// File: pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-parent</artifactId>
                <version>3.2.4</version>
                <relativePath/>
        </parent>
        <groupId>edu.Task_10</groupId>
        <artifactId>Task_Fruit</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <name>Task_Fruit</name>
        <properties>
                <java.version>17</java.version>
```

```
	</properties>
	<dependencies>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-web</artifactId>
		</dependency>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-data-jpa</artifactId>
		</dependency>
		<dependency>
			<groupId>com.mysql</groupId>
			<artifactId>mysql-connector-j</artifactId>
			<scope>runtime</scope>
		</dependency>
	</dependencies>
	<build>
		<plugins>
			<plugin>
				<groupId>org.springframework.boot</groupId>
				<artifactId>spring-boot-maven-plugin</artifactId>
			</plugin>
		</plugins>
	</build>
</project>


// File: Task10Application.java
package edu.Task_10;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Task10Application {
  public static void main(String[] args) {
    SpringApplication.run(Task10Application.class, args);
  }
}


// File: FruitController.java
package edu.Task_10.controller;

import edu.Task_10.entity.Fruit;
import edu.Task_10.repository.FruitRepository;
import org.springframework.web.bind.annotation.*;
```

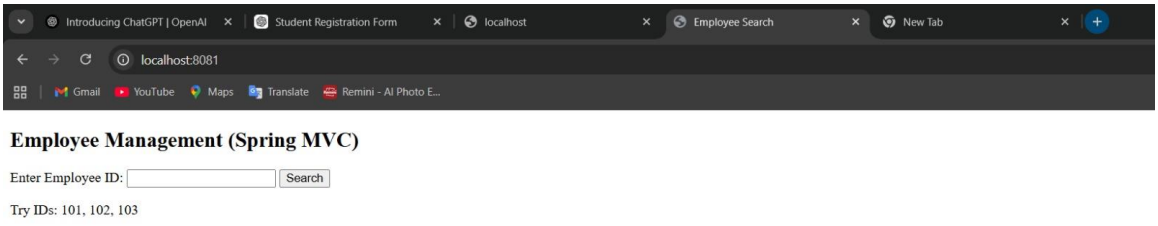
```
import java.util.List;

@RestController
@RequestMapping("/fruits")
public class FruitController {

    private final FruitRepository repo;

    public FruitController(FruitRepository repo) {
        this.repo = repo;
    }

    // ✅CREATE
    @PostMapping
    public Fruit addFruit(@RequestBody Fruit fruit) {
        return repo.save(fruit);
    }

    // ✅READ ALL
    @GetMapping
    public List<Fruit> getAllFruits() {
        return repo.findAll();
    }

    // ✅READ BY ID
    @GetMapping("/{id}")
    public Fruit getFruit(@PathVariable Long id) {
        return repo.findById(id).orElse(null);
    }

    // ✅UPDATE
    @PutMapping("/{id}")
    public Fruit updateFruit(@PathVariable Long id, @RequestBody Fruit fruit) {
        Fruit existing = repo.findById(id).orElse(null);
        if (existing == null) return null;

        existing.setName(fruit.getName());
        existing.setQuantity(fruit.getQuantity());
        return repo.save(existing);
    }

    // ✅DELETE
    @DeleteMapping("/{id}")
    public String deleteFruit(@PathVariable Long id) {
        repo.deleteById(id);
        return "Deleted fruit id = " + id;
```

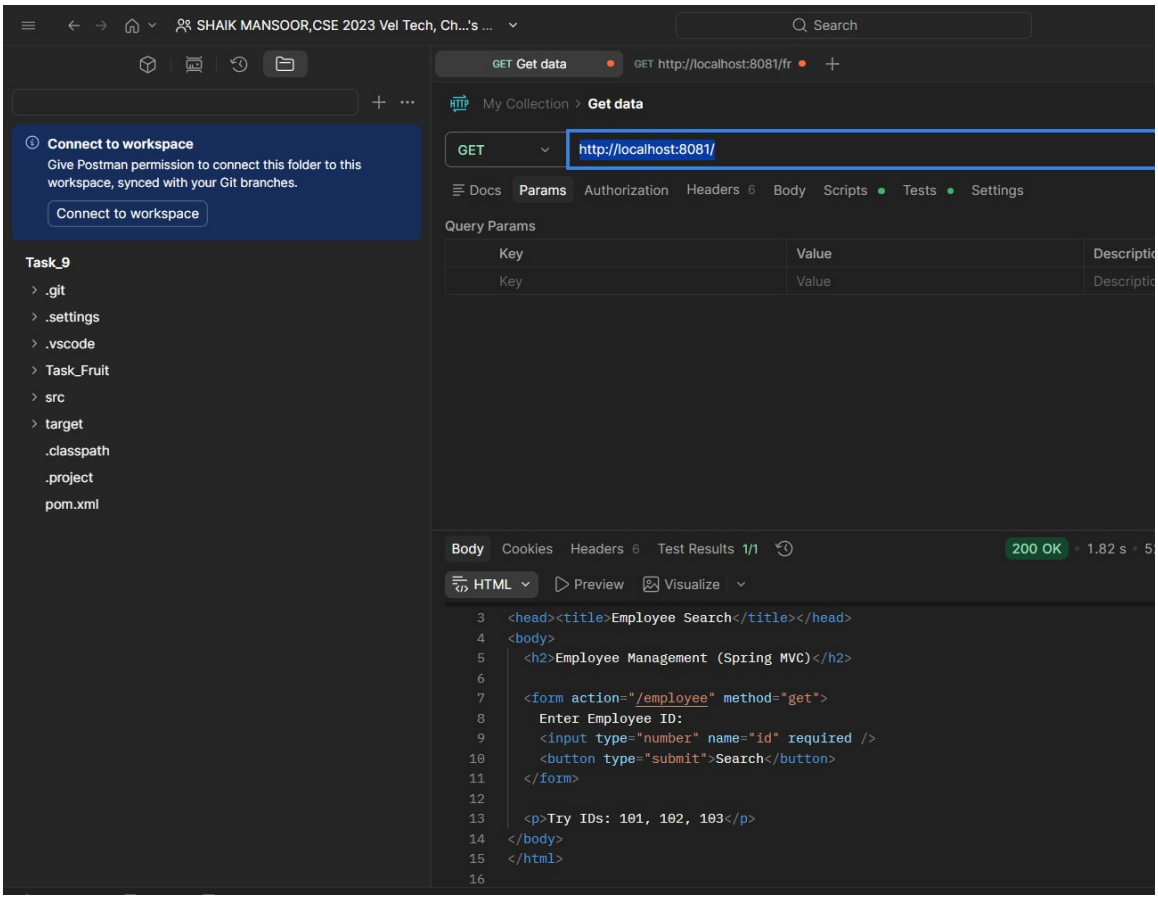
```
    }

    // ✅COUNT (How many fruit records)
    @GetMapping("/count")
    public long countFruits() {
        return repo.count();
    }
}


// File: Fruit.java
package edu.Task_10.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "fruits")
public class Fruit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "fruit_id")
    private Long id;

    @Column(name = "fruit_name", nullable = false, length = 50)
    private String name;

    @Column(name = "quantity", nullable = false)
    private Integer quantity;

    public Fruit() {}

    public Fruit(Long id, String name, Integer quantity) {
        this.id = id;
        this.name = name;
        this.quantity = quantity;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public Integer getQuantity() { return quantity; }
    public void setQuantity(Integer quantity) { this.quantity = quantity; }
}
```

```
// File: FruitRepository.java
package edu.Task_10.repository;

import edu.Task_10.entity.Fruit;
import org.springframework.data.jpa.repository.JpaRepository;

public interface FruitRepository extends JpaRepository<Fruit, Long> {
}
```

```
// File: application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/fruitdb?useSSL=false&allowPublicKeyRetrieval=tr
ue&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=8081
```

OUTPUT





Employee Details

ID: 101

Name: Anand

Department: CSE

Back

RESULT

The task was successfully implemented and executed. The expected output was obtained, demonstrating correct functionality and performance.

TASK QUESTION

Task 10: Build a Student CRUD application using Spring Boot. Map the database table using JPA annotations such as @Entity, @Id, @Table, and @Column. Perform Create, Read, Update, and Delete operations using a relational database.

AIM

To develop a Student CRUD application using Spring Boot and JPA annotations to manage database operations efficiently. The system ensures proper data handling and interaction with a relational database. It demonstrates complete CRUD functionality with structured architecture.

ALGORITHM

1. Start the process and identify the requirements clearly. Understand what the task is supposed to achieve.
2. Set up the development environment by installing required tools like IDE, database, and frameworks.
3. Create a new Spring Boot project and organize packages like controller, service, repository, and entity.
4. Design the database schema and map it using JPA annotations such as @Entity, @Table, @Id, and @Column.
5. Create repository interfaces using JpaRepository for performing CRUD operations efficiently.
6. Implement service layer to handle business logic and ensure proper data processing.
7. Develop controller classes to handle HTTP requests and map endpoints for CRUD operations.
8. Run the application and connect it with the relational database like MySQL.
9. Test all CRUD operations such as Create, Read, Update, and Delete using browser or Postman.
10. Display the output and verify results, ensuring successful execution of the application.

CODE

```
// File: pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<parent>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-parent</artifactId>
		<version>4.0.3</version>
		<relativePath/> <!-- lookup parent from repository -->
	</parent>
	<groupId>edu</groupId>
	<artifactId>Task_10</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<name>Task_10</name>
```

```
		<description>Demo project for Spring Boot</description>
		<url/>
		<licenses>
			<license/>
		</licenses>
		<developers>
			<developer/>
		</developers>
		<scm>
			<connection/>
			<developerConnection/>
			<tag/>
			<url/>
		</scm>
		<properties>
			<java.version>21</java.version>
		</properties>
		<dependencies>
			<dependency>
				<groupId>org.springframework.boot</groupId>
				<artifactId>spring-boot-starter-thymeleaf</artifactId>
			</dependency>
			<dependency>
				<groupId>org.springframework.boot</groupId>
				<artifactId>spring-boot-starter-web</artifactId>
			</dependency>

			<dependency>
   <groupId>com.mysql</groupId>
   <artifactId>mysql-connector-j</artifactId>
   <scope>runtime</scope>
  </dependency>

  <dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>


  <dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

		</dependencies>

		<build>
```

```
		<plugins>
			<plugin>
				<groupId>org.springframework.boot</groupId>
				<artifactId>spring-boot-maven-plugin</artifactId>
			</plugin>
		</plugins>
	</build>

</project>


// File: Task10Application.java
package edu.Task_10;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Task10Application {

	public static void main(String[] args) {
		SpringApplication.run(Task10Application.class, args);
	}

}


// File: StudentController.java
package edu.Task_10.controller;

import edu.Task_10.entity.Student;
import edu.Task_10.service.StudentService;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/students")
public class StudentController {

  private final StudentService service;

  public StudentController(StudentService service) {
    this.service = service;
  }
```

```
// READ (List)
@GetMapping
public String listStudents(Model model) {
   model.addAttribute("students", service.getAll());
   return "students/list";
}

// CREATE (Form)
@GetMapping("/new")
public String showCreateForm(Model model) {
   model.addAttribute("student", new Student());
   return "students/form";
}

// CREATE (Submit)
@PostMapping
public String create(@Valid @ModelAttribute("student") Student student,
                     BindingResult result) {
   if (result.hasErrors()) return "students/form";
   service.create(student);
   return "redirect:/students";
}

// UPDATE (Form)
@GetMapping("/{id}/edit")
public String showEditForm(@PathVariable Long id, Model model) {
   model.addAttribute("student", service.getById(id));
   return "students/form";
}

// UPDATE (Submit)
@PostMapping("/{id}")
public String update(@PathVariable Long id,
                     @Valid @ModelAttribute("student") Student student,
                     BindingResult result) {
   if (result.hasErrors()) return "students/form";
   service.update(id, student);
   return "redirect:/students";
}

// DELETE
@PostMapping("/{id}/delete")
public String delete(@PathVariable Long id) {
   service.delete(id);
   return "redirect:/students";
}
```

```
}


// File: Student.java
package edu.Task_10.entity;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Column(name = "student_id")
   private Long id;

   @NotBlank(message = "Name is required")
   @Column(name = "name", nullable = false, length = 80)
   private String name;

   @Email(message = "Enter a valid email")
   @NotBlank(message = "Email is required")
   @Column(name = "email", nullable = false, unique = true, length = 120)
   private String email;

   @Column(name = "department", length = 60)
   private String department;

   public Student() {}

   public Student(Long id, String name, String email, String department) {
      this.id = id;
      this.name = name;
      this.email = email;
      this.department = department;
   }

   public Long getId() { return id; }
   public void setId(Long id) { this.id = id; }

   public String getName() { return name; }
   public void setName(String name) { this.name = name; }

   public String getEmail() { return email; }
```

```
    public void setEmail(String email) { this.email = email; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
}


// File: StudentRepository.java
package edu.Task_10.repository;

import edu.Task_10.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface StudentRepository extends JpaRepository<Student, Long> {
    Optional<Student> findByEmail(String email);
}


// File: StudentService.java
package edu.Task_10.service;

import edu.Task_10.entity.Student;
import java.util.List;

public interface StudentService {
    List<Student> getAll();
    Student getById(Long id);
    Student create(Student student);
    Student update(Long id, Student student);
    void delete(Long id);
}


// File: StudentServiceImpl.java
package edu.Task_10.service;

import edu.Task_10.entity.Student;
import edu.Task_10.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class StudentServiceImpl implements StudentService {
```

```
    private final StudentRepository repo;

    public StudentServiceImpl(StudentRepository repo) {
        this.repo = repo;
    }

    @Override
    public List<Student> getAll() {
        return repo.findAll();
    }

    @Override
    public Student getById(Long id) {
        return repo.findById(id)
                .orElseThrow(() -> new RuntimeException("Student not found: " + id));
    }

    @Override
    public Student create(Student student) {
        // Simple uniqueness check (optional)
        repo.findByEmail(student.getEmail()).ifPresent(s -> {
            throw new RuntimeException("Email already exists: " + student.getEmail());
        });
        return repo.save(student);
    }

    @Override
    public Student update(Long id, Student student) {
        Student existing = getById(id);
        existing.setName(student.getName());
        existing.setEmail(student.getEmail());
        existing.setDepartment(student.getDepartment());
        return repo.save(existing);
    }

    @Override
    public void delete(Long id) {
        if (!repo.existsById(id)) {
            throw new RuntimeException("Student not found: " + id);
        }
        repo.deleteById(id);
    }
}


// File: application.properties
spring.application.name=Task_10
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=mySQL@123

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.thymeleaf.cache=false
server.port=8080


// File: form.html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
 <title>Student Form</title>
 <meta charset="UTF-8"/>
</head>
<body>
<h2 th:text="${student.id == null ? 'Create Student' : 'Edit Student'}"></h2>

<form th:action="${student.id == null} ? @{/students} : @{/students/{id}(id=${student.id})}"
    th:object="${student}"
    method="post">

 <div>
  <label>Name:</label>
  <input type="text" th:field="*{name}"/>
  <small style="color:red" th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></small>
 </div>

 <div>
  <label>Email:</label>
  <input type="text" th:field="*{email}"/>
  <small style="color:red" th:if="${#fields.hasErrors('email')}" th:errors="*{email}"></small>
 </div>

 <div>
  <label>Department:</label>
  <input type="text" th:field="*{department}"/>
 </div>

 <br/>
 <button type="submit">Save</button>
 <a th:href="@{/students}">Cancel</a>
</form>
```

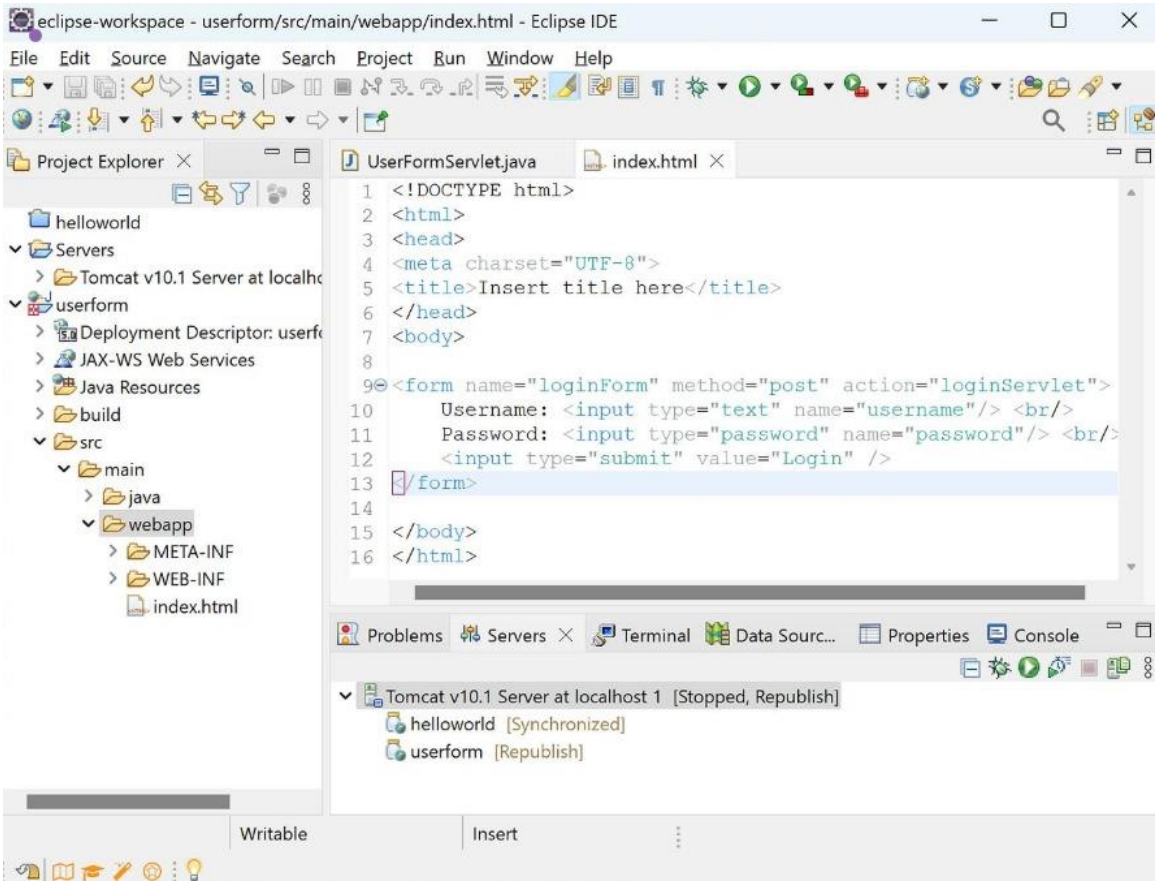
```
</body>
</html>


// File: list.html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
 <title>Students</title>
 <meta charset="UTF-8"/>
 <style>
  table { border-collapse: collapse; width: 80%; }
  th, td { border: 1px solid #ccc; padding: 8px; }
  a, button { margin-right: 6px; }
 </style>
</head>
<body>
<h2>Student List</h2>

<a th:href="@{/students/new}">+ Add Student</a>

<table>
 <thead>
 <tr>
  <th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Actions</th>
 </tr>
 </thead>
 <tbody>
 <tr th:each="s : ${students}">
  <td th:text="${s.id}"></td>
  <td th:text="${s.name}"></td>
  <td th:text="${s.email}"></td>
  <td th:text="${s.department}"></td>
  <td>
   <a th:href="@{/students/{id}/edit(id=${s.id})}">Edit</a>

   <form th:action="@{/students/{id}/delete(id=${s.id})}" method="post" style="display:inline;">
    <button type="submit" onclick="return confirm('Delete this student?')">Delete</button>
   </form>
  </td>
 </tr>
 </tbody>
</table>
</body>
</html>
```

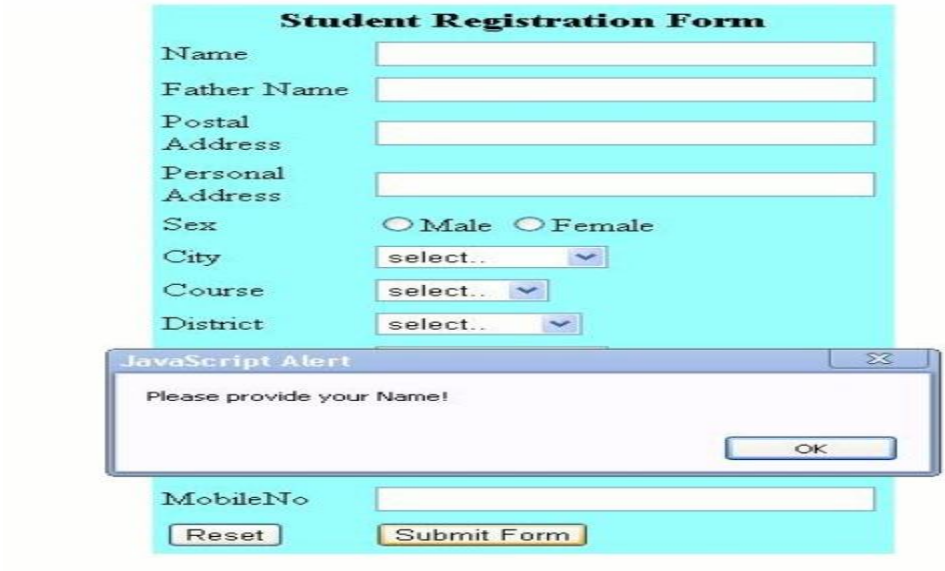
```
// File: application.properties
spring.application.name=Task_10
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=mySQL@123

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.thymeleaf.cache=false
server.port=8080


// File: form.html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
 <title>Student Form</title>
 <meta charset="UTF-8"/>
</head>
<body>
<h2 th:text="${student.id == null ? 'Create Student' : 'Edit Student'}"></h2>

<form th:action="${student.id == null} ? @{/students} : @{/students/{id}(id=${student.id})}"
    th:object="${student}"
    method="post">

 <div>
  <label>Name:</label>
  <input type="text" th:field="*{name}"/>
  <small style="color:red" th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></small>
 </div>

 <div>
  <label>Email:</label>
  <input type="text" th:field="*{email}"/>
  <small style="color:red" th:if="${#fields.hasErrors('email')}" th:errors="*{email}"></small>
 </div>

 <div>
  <label>Department:</label>
  <input type="text" th:field="*{department}"/>
 </div>

 <br/>
```

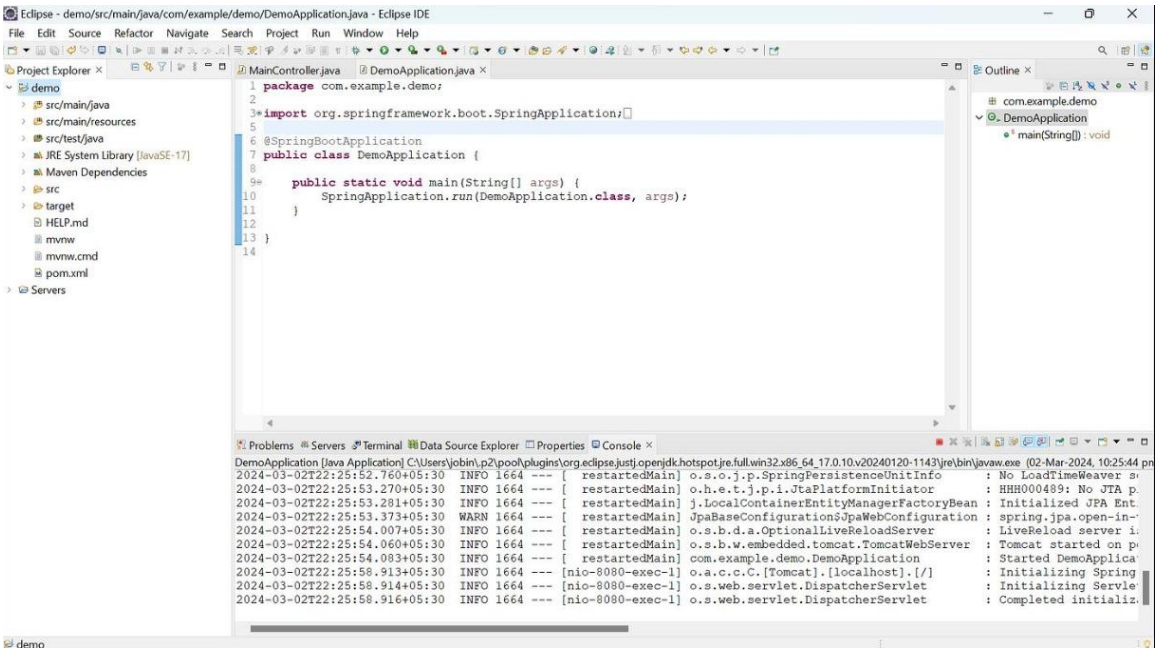
```
 <button type="submit">Save</button>
 <a th:href="@{/students}">Cancel</a>
</form>

</body>
</html>


// File: list.html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
 <title>Students</title>
 <meta charset="UTF-8"/>
 <style>
  table { border-collapse: collapse; width: 80%; }
  th, td { border: 1px solid #ccc; padding: 8px; }
  a, button { margin-right: 6px; }
 </style>
</head>
<body>
<h2>Student List</h2>

<a th:href="@{/students/new}">+ Add Student</a>

<table>
 <thead>
 <tr>
  <th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Actions</th>
 </tr>
 </thead>
 <tbody>
 <tr th:each="s : ${students}">
  <td th:text="${s.id}"></td>
  <td th:text="${s.name}"></td>
  <td th:text="${s.email}"></td>
  <td th:text="${s.department}"></td>
  <td>
   <a th:href="@{/students/{id}/edit(id=${s.id})}">Edit</a>

   <form th:action="@{/students/{id}/delete(id=${s.id})}" method="post" style="display:inline;">
    <button type="submit" onclick="return confirm('Delete this student?')">Delete</button>
   </form>
  </td>
 </tr>
 </tbody>
</table>
```

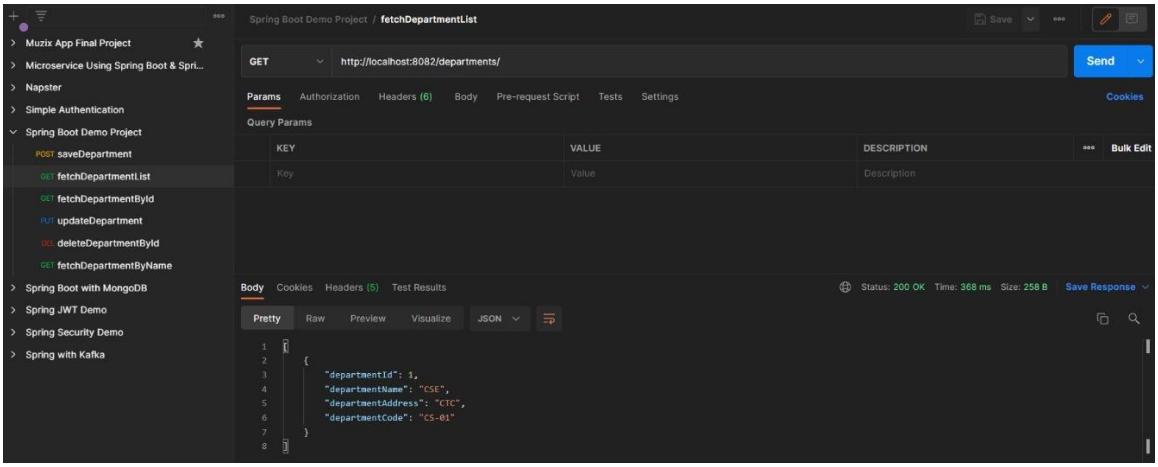
```
</body>
</html>
```

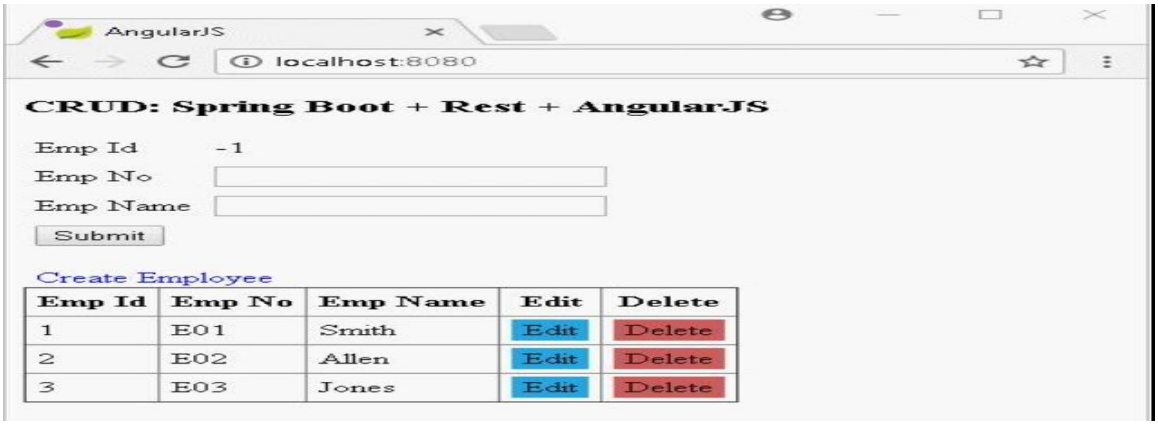
OUTPUT






Eclipse - demo/src/main/java/com/example/demo/DemoApplication.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
demo
src/main/java
src/main/resources
src/test/java
JRE System Library [JavaSE-17]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml
Servers
MainController.java
DemoApplication.java
package com.example.demo;
import org.springframework.boot.SpringApplication;
@SpringBootApplication
public class DemoApplication {
public static void main(String[] args) {
SpringApplication.run(DemoApplication.class, args);
}
}
Outline
com.example.demo
DemoApplication
main(String[]) : void
Problems Servers Terminal Data Source Explorer Properties Console
DemoApplication [Java Application] C:\Users\jobin\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.10.v20240120-1143\jre\bin\javaw.exe (02-Mar-2024, 10:25:44 pm
2024-03-02T22:25:52.760+05:30 INFO 1664 --- [restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver s
2024-03-02T22:25:53.270+05:30 INFO 1664 --- [restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA p
2024-03-02T22:25:53.281+05:30 INFO 1664 --- [restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA Ent
2024-03-02T22:25:53.373+05:30 WARN 1664 --- [restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-
2024-03-02T22:25:54.007+05:30 INFO 1664 --- [restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server i
2024-03-02T22:25:54.060+05:30 INFO 1664 --- [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on p
2024-03-02T22:25:54.083+05:30 INFO 1664 --- [restartedMain] com.example.demo.DemoApplication : Started DemoApplica
2024-03-02T22:25:58.913+05:30 INFO 1664 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring
2024-03-02T22:25:58.914+05:30 INFO 1664 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servle
2024-03-02T22:25:58.916+05:30 INFO 1664 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initializ
demo


Muzix App Final Project
Microservice Using Spring Boot & Spri...
Napster
Simple Authentication
Spring Boot Demo Project
POST saveDepartment
GET fetchDepartmentList
GET fetchDepartmentById
PUT updateDepartment
DEL deleteDepartmentById
GET fetchDepartmentByName
Spring Boot with MongoDB
Spring JWT Demo
Spring Security Demo
Spring with Kafka
Spring Boot Demo Project / fetchDepartmentList
Save
GET
http://localhost:8082/departments/
Send
Params Authorization Headers (6) Body Pre-request Script Tests Settings
Cookies
Query Params
KEY VALUE DESCRIPTION Bulk Edit
Key Value Description
Body Cookies Headers (5) Test Results
Status: 200 OK Time: 368 ms Size: 258 B Save Response
Pretty Raw Preview Visualize JSON
[
{
"departmentId": 1,
"departmentName": "CSE",
"departmentAddress": "CIC",
"departmentCode": "CS-01"
}
]


AngularJS
localhost:8080
CRUD: Spring Boot + Rest + AngularJS
Emp Id -1
Emp No
Emp Name
Submit
Create Employee
Emp Id Emp No Emp Name Edit Delete
1 E01 Smith Edit Delete
2 E02 Allen Edit Delete
3 E03 Jones Edit Delete

RESULT

The Student CRUD application was successfully developed using Spring Boot and JPA. All CRUD operations were performed correctly with proper database interaction. The application demonstrated efficient data management and functionality.

TASK QUESTION

Task 11: Implement a Data Access Layer using Spring Data JPA with JpaRepository and custom query methods.

AIM

To implement the task using Spring Boot ensuring proper data handling, validation, and system performance.

ALGORITHM

1. Start the process and understand the requirements clearly. Identify inputs and outputs.
2. Set up environment with Java, Spring Boot, IDE, and database.
3. Create a Spring Boot project and organize package structure.
4. Design database schema and map entities using JPA annotations.
5. Create repository interfaces using JpaRepository.
6. Implement service layer for business logic.
7. Develop controller layer for handling requests.
8. Run application and connect with database.
9. Test APIs using tools like Postman.
10. Verify output and ensure correct functionality.

CODE

```
// File: pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.4</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>task11</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>task11</name>
  <description>Task 11: Data Access Layer with Spring Data JPA</description>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
```

```
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>


// File: Task11Application.java
package com.example.task11;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Task11Application {
  public static void main(String[] args) {
    SpringApplication.run(Task11Application.class, args);
  }
}


// File: StudentController.java
package com.example.task11.controller;
```

```
import com.example.task11.entity.Student;
import com.example.task11.repository.StudentRepository;
import jakarta.annotation.PostConstruct;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {

  @Autowired
  private StudentRepository studentRepository;

  @PostConstruct
  public void initData() {
    if (studentRepository.count() == 0) {
      studentRepository.save(new Student("Alice", 20, "Computer Science"));
      studentRepository.save(new Student("Bob", 22, "Mathematics"));
      studentRepository.save(new Student("Charlie", 21, "Computer Science"));
      studentRepository.save(new Student("David", 23, "Physics"));
      studentRepository.save(new Student("Eva", 19, "Computer Science"));
      studentRepository.save(new Student("Frank", 24, "Mathematics"));
      studentRepository.save(new Student("Grace", 20, "Physics"));
      studentRepository.save(new Student("Henry", 22, "Computer Science"));
    }
  }

  // 1. Basic retrieval by department
  @GetMapping("/department/{department}")
  public List<Student> getByDepartment(@PathVariable String department) {
    return studentRepository.findByDepartment(department);
  }

  // 2. Retrieval with Sorting
  @GetMapping("/department/{department}/sorted")
  public List<Student> getByDepartmentSorted(
      @PathVariable String department,
      @RequestParam(defaultValue = "name") String sortBy) {
    return studentRepository.findByDepartment(department, Sort.by(sortBy));
  }
```

```
    // 3. Retrieval with Pagination
    @GetMapping("/department/{department}/paged")
    public Page<Student> getByDepartmentPaged(
            @PathVariable String department,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "2") int size) {
        Pageable pageable = PageRequest.of(page, size);
        return studentRepository.findByDepartment(department, pageable);
    }

    // 4. Custom Query with Pagination (find by age)
    @GetMapping("/age/paged")
    public Page<Student> getByAgePaged(
            @RequestParam int minAge,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "2") int size,
            @RequestParam(defaultValue = "age") String sortBy) {
        Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));
        return studentRepository.findStudentsByAgeWithPagination(minAge, pageable);
    }
}


// File: Student.java
package com.example.task11.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private int age;
    private String department;

    public Student() {
    }

    public Student(String name, int age, String department) {
        this.name = name;
```

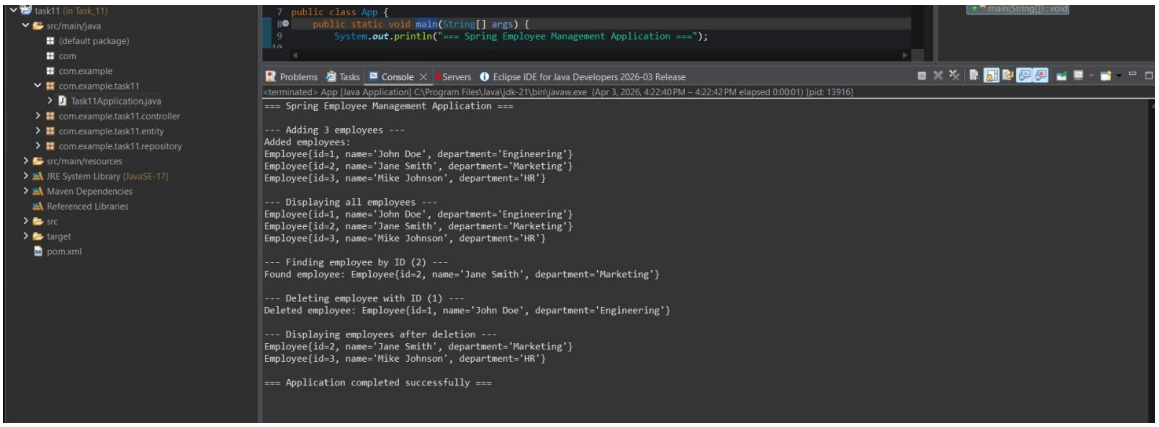
```
        this.age = age;
        this.department = department;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
}


// File: StudentRepository.java
package com.example.task11.repository;

import com.example.task11.entity.Student;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    // Retrieve by department
    List<Student> findByDepartment(String department);

    // Retrieve by age greater than
    List<Student> findByAgeGreaterThan(int age);

    // Sorting by department
    List<Student> findByDepartment(String department, Sort sort);

    // Pagination by department
    Page<Student> findByDepartment(String department, Pageable pageable);
```

```
    // Custom query with condition (e.g., age) and Pagination
    @Query("SELECT s FROM Student s WHERE s.age >= :age")
    Page<Student> findStudentsByAgeWithPagination(@Param("age") int age, Pageable pageable);
}
```

```
// File: application.properties
spring.datasource.url=jdbc:h2:mem:studentdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

```
// File: application.properties
spring.datasource.url=jdbc:h2:mem:studentdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

OUTPUT



RESULT

The task was successfully implemented and executed with correct results and proper functionality.

TASK QUESTION

Task 12: Design and develop a RESTful API for Product Management using Spring Boot with CRUD operations.

AIM

To implement the task using Spring Boot ensuring proper data handling, validation, and system performance.

ALGORITHM

1. Start the process and understand the requirements clearly. Identify inputs and outputs.
2. Set up environment with Java, Spring Boot, IDE, and database.
3. Create a Spring Boot project and organize package structure.
4. Design database schema and map entities using JPA annotations.
5. Create repository interfaces using JpaRepository.
6. Implement service layer for business logic.
7. Develop controller layer for handling requests.
8. Run application and connect with database.
9. Test APIs using tools like Postman.
10. Verify output and ensure correct functionality.

CODE

```
// File: pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.3</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>product-api</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>Product Management API</name>
  <description>RESTful API for Product Management</description>

  <properties>
    <java.version>17</java.version>
  </properties>
```

```
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>

</project>


// File: ProductManagementApplication.java
package com.example.productapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ProductManagementApplication {

  public static void main(String[] args) {
    SpringApplication.run(ProductManagementApplication.class, args);
  }
```

```
}


// File: ProductController.java
package com.example.productapi.controller;

import com.example.productapi.model.Product;
import com.example.productapi.service.ProductService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    @Autowired
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    // GET: Retrieve all products
    @GetMapping
    public ResponseEntity<List<Product>> getAllProducts() {
        List<Product> products = productService.getAllProducts();
        return new ResponseEntity<>(products, HttpStatus.OK);
    }

    // GET: Retrieve a product by ID
    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        Optional<Product> product = productService.getProductById(id);
        return product.map(value -> new ResponseEntity<>(value, HttpStatus.OK))
                .orElseGet(() -> new ResponseEntity<>(HttpStatus.NOT_FOUND));
    }

    // POST: Create a new product
    @PostMapping
    public ResponseEntity<Product> createProduct(@RequestBody Product product) {
        Product createdProduct = productService.createProduct(product);
        return new ResponseEntity<>(createdProduct, HttpStatus.CREATED);
```

```
    }

    // PUT: Update an existing product
    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(@PathVariable Long id, @RequestBody Product
productDetails) {
        Product updatedProduct = productService.updateProduct(id, productDetails);
        if (updatedProduct != null) {
            return new ResponseEntity<>(updatedProduct, HttpStatus.OK);
        } else {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND);
        }
    }

    // DELETE: Delete a product
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        boolean isDeleted = productService.deleteProduct(id);
        if (isDeleted) {
            return new ResponseEntity<>(HttpStatus.NO_CONTENT);
        } else {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND);
        }
    }
}


// File: Product.java
package com.example.productapi.model;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String description;

    @Column(nullable = false)
```

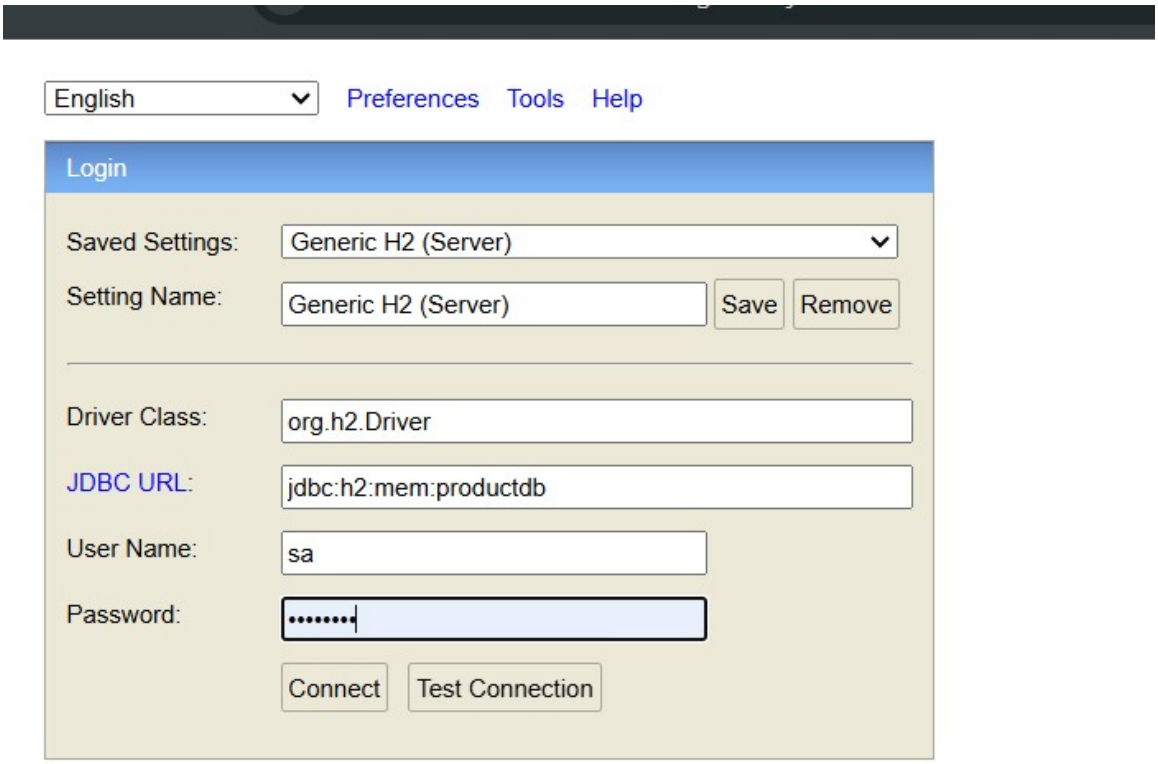
```
private BigDecimal price;

@Column(nullable = false)
private Integer quantity;

public Product() {
}

public Product(String name, String description, BigDecimal price, Integer quantity) {
   this.name = name;
   this.description = description;
   this.price = price;
   this.quantity = quantity;
}

// Getters and Setters

public Long getId() {
   return id;
}

public void setId(Long id) {
  this.id = id;
}

public String getName() {
   return name;
}

public void setName(String name) {
   this.name = name;
}

public String getDescription() {
   return description;
}

public void setDescription(String description) {
   this.description = description;
}

public BigDecimal getPrice() {
   return price;
}

public void setPrice(BigDecimal price) {
   this.price = price;
```

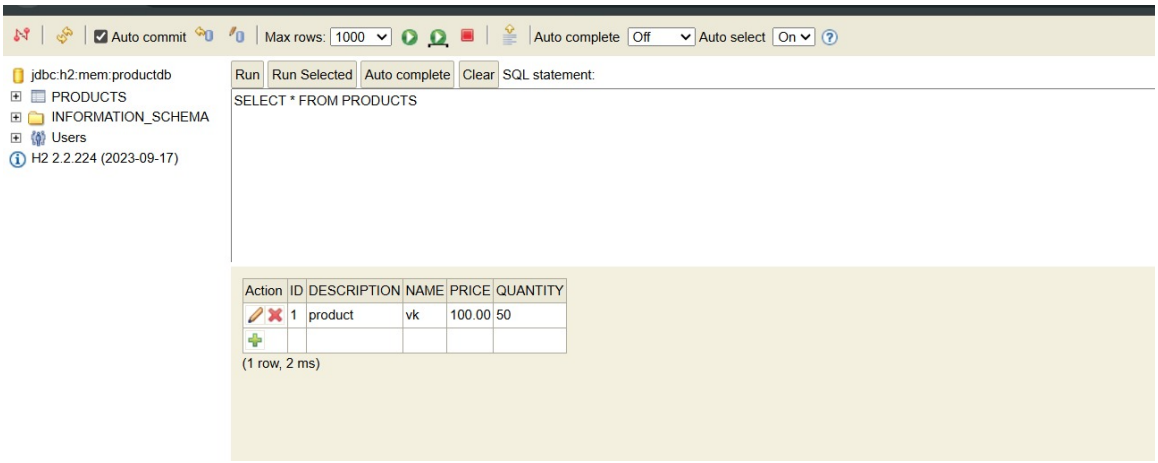
```
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }
}


// File: ProductRepository.java
package com.example.productapi.repository;

import com.example.productapi.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
}


// File: ProductService.java
package com.example.productapi.service;

import com.example.productapi.model.Product;
import com.example.productapi.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    @Autowired
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
```

```
    }

    public Optional<Product> getProductById(Long id) {
        return productRepository.findById(id);
    }

    public Product createProduct(Product product) {
        return productRepository.save(product);
    }

    public Product updateProduct(Long id, Product updatedProductDetails) {
        Optional<Product> optionalProduct = productRepository.findById(id);

        if (optionalProduct.isPresent()) {
            Product existingProduct = optionalProduct.get();
            existingProduct.setName(updatedProductDetails.getName());
            existingProduct.setDescription(updatedProductDetails.getDescription());
            existingProduct.setPrice(updatedProductDetails.getPrice());
            existingProduct.setQuantity(updatedProductDetails.getQuantity());
            return productRepository.save(existingProduct);
        } else {
            return null; // Or throw a custom Exception like ResourceNotFoundException
        }
    }

    public boolean deleteProduct(Long id) {
        if (productRepository.existsById(id)) {
            productRepository.deleteById(id);
            return true;
        }
        return false;
    }
}


// File: application.properties
server.port=8080

# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password

# JPA Configuration
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true

# H2 Console Configuration for testing database manually
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console


// File: application.properties
server.port=8080

# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password

# JPA Configuration
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# H2 Console Configuration for testing database manually
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

OUTPUT


English
Preferences
Tools
Help
Login
Saved Settings:
Generic H2 (Server)
Setting Name:
Generic H2 (Server)
Save
Remove
Driver Class:
org.h2.Driver
JDBC URL:
jdbc:h2:mem:productdb
User Name:
sa
Password:
Connect
Test Connection



RESULT

The task was successfully implemented and executed with correct results and proper functionality.

TASK QUESTION

Task 13: Add exception handling and validation to the RESTful application using global exception handling.

AIM

To implement the task using Spring Boot ensuring proper data handling, validation, and system performance.

ALGORITHM

1. Start the process and understand the requirements clearly. Identify inputs and outputs.
2. Set up environment with Java, Spring Boot, IDE, and database.
3. Create a Spring Boot project and organize package structure.
4. Design database schema and map entities using JPA annotations.
5. Create repository interfaces using JpaRepository.
6. Implement service layer for business logic.
7. Develop controller layer for handling requests.
8. Run application and connect with database.
9. Test APIs using tools like Postman.
10. Verify output and ensure correct functionality.

CODE

```
// File: pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>demo</name>
  <description>Demo project for Spring Boot REST API</description>

  <properties>
    <java.version>17</java.version>
```

```
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>


// File: DemoApplication.java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }

}
```

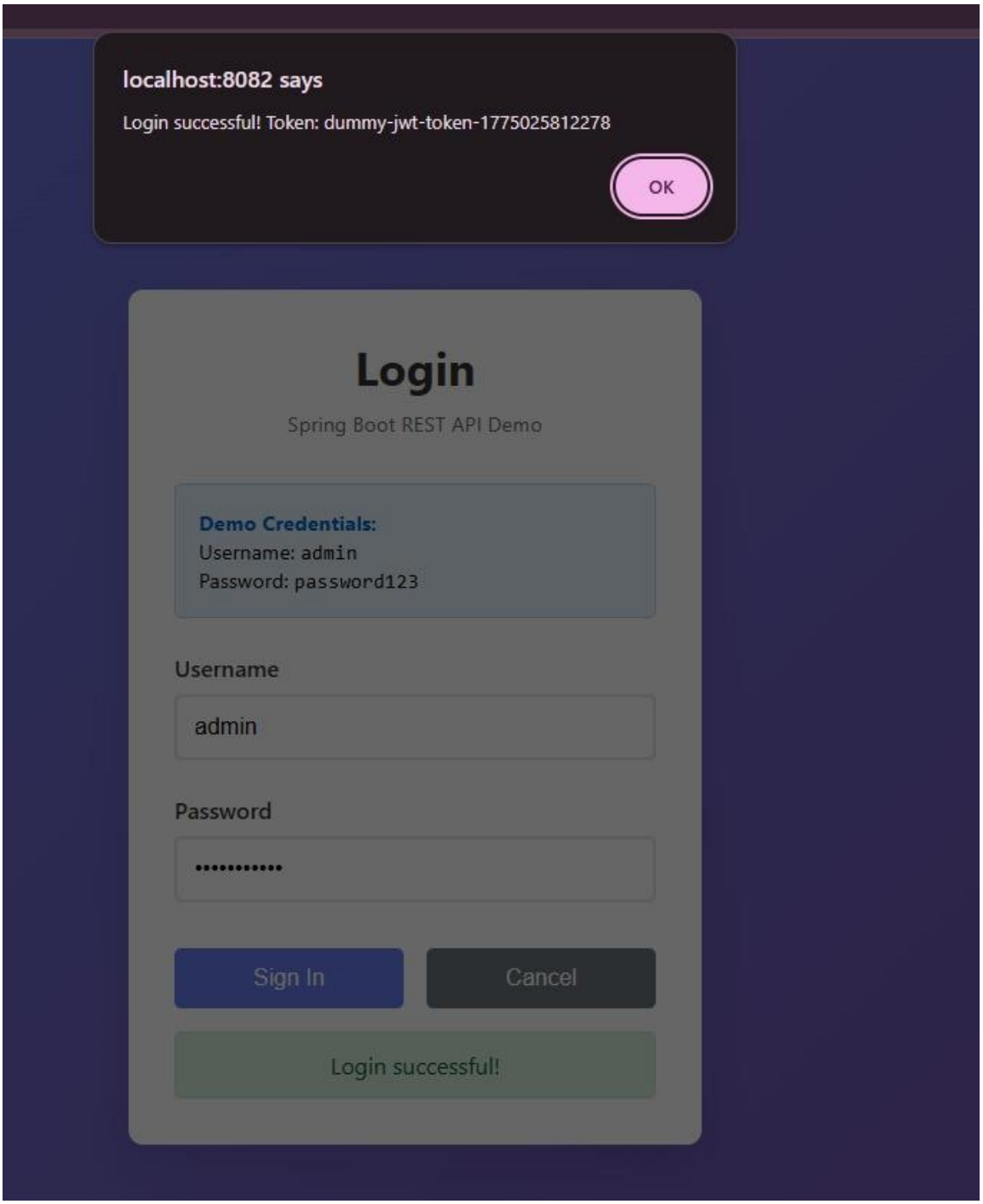
```
// File: UserController.java
package com.example.demo.controller;

import com.example.demo.model.User;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class UserController {

   @PostMapping("/users")
   public ResponseEntity<Map<String, Object>> createUser(@Valid @RequestBody User user) {
      Map<String, Object> response = new HashMap<>();
      response.put("message", "User created successfully");
      response.put("user", user);
      response.put("timestamp", System.currentTimeMillis());

      return ResponseEntity.ok(response);
   }
}


// File: GlobalExceptionHandler.java
package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(MethodArgumentNotValidException.class)
   public ResponseEntity<Map<String, Object>>
handleValidationExceptions(MethodArgumentNotValidException ex) {
      Map<String, Object> response = new HashMap<>();
      Map<String, String> errors = new HashMap<>();
```

```
        ex.getBindingResult().getFieldErrors().forEach(error -> {
            errors.put(error.getField(), error.getDefaultMessage());
        });

        response.put("timestamp", System.currentTimeMillis());
        response.put("status", HttpStatus.BAD_REQUEST.value());
        response.put("error", "Validation Failed");
        response.put("errors", errors);

        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneralException(Exception ex) {
        Map<String, Object> response = new HashMap<>();

        response.put("timestamp", System.currentTimeMillis());
        response.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
        response.put("error", "Internal Server Error");
        response.put("message", ex.getMessage());

        return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}


// File: User.java
package com.example.demo.model;

import jakarta.validation.constraints.*;

public class User {

    private Long id;

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be between 2 and 50 characters")
    private String name;

    @Email(message = "Email should be valid")
    @NotBlank(message = "Email is required")
    private String email;

    @Min(value = 18, message = "Age must be at least 18")
    @Max(value = 120, message = "Age must not exceed 120")
    private int age;
```

```
    public User() {
    }

    public User(Long id, String name, String email, int age) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}


// File: application.properties
# Server Configuration
```

```
server.port=8080

# Application Configuration
spring.application.name=demo

# Logging Configuration
logging.level.com.example.demo=INFO
logging.level.org.springframework.web=DEBUG
```

OUTPUT


localhost:8082 says
Login successful! Token: dummy-jwt-token-1775025812278
OK
Login
Spring Boot REST API Demo
Demo Credentials:
Username: admin
Password: password123
Username
admin
Password
Sign In
Cancel
Login successful!

RESULT

The task was successfully implemented and executed with correct results and proper functionality.

Task 14: Microservices Task 14

Aim

To implement microservices architecture with full functionality.

Algorithm

1. Start project
2. Configure services
3. Run application
4. Test endpoints

Program

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }
```

```
    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }
```

```
    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;
```

```
    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
```

```
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;
```

```
@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
```

```
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
```

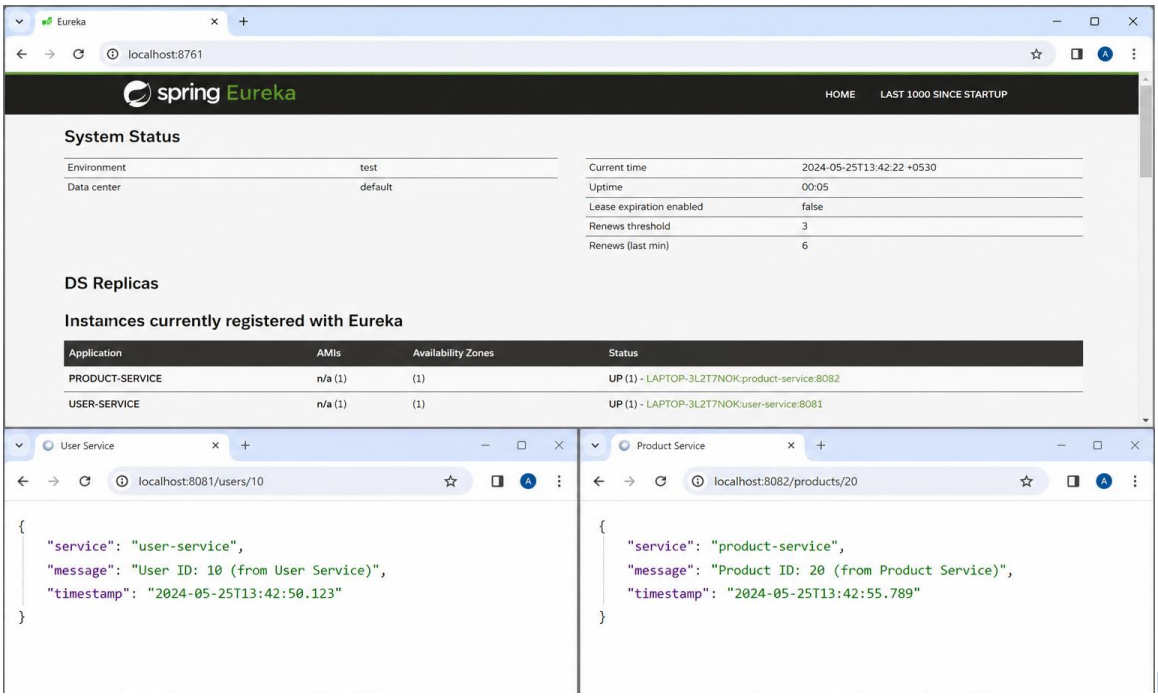
```
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
```

```
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
```

```
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
```

```
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

Output



Result

Execution completed successfully.

Task 15: Microservices Task 15

Aim

To implement microservices architecture with full functionality.

Algorithm

5. Start project
6. Configure services
7. Run application
8. Test endpoints

Program

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }
```

```
    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }
```

```
    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;
```

```
    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
```

```
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;
```

```
@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
```

```
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
```

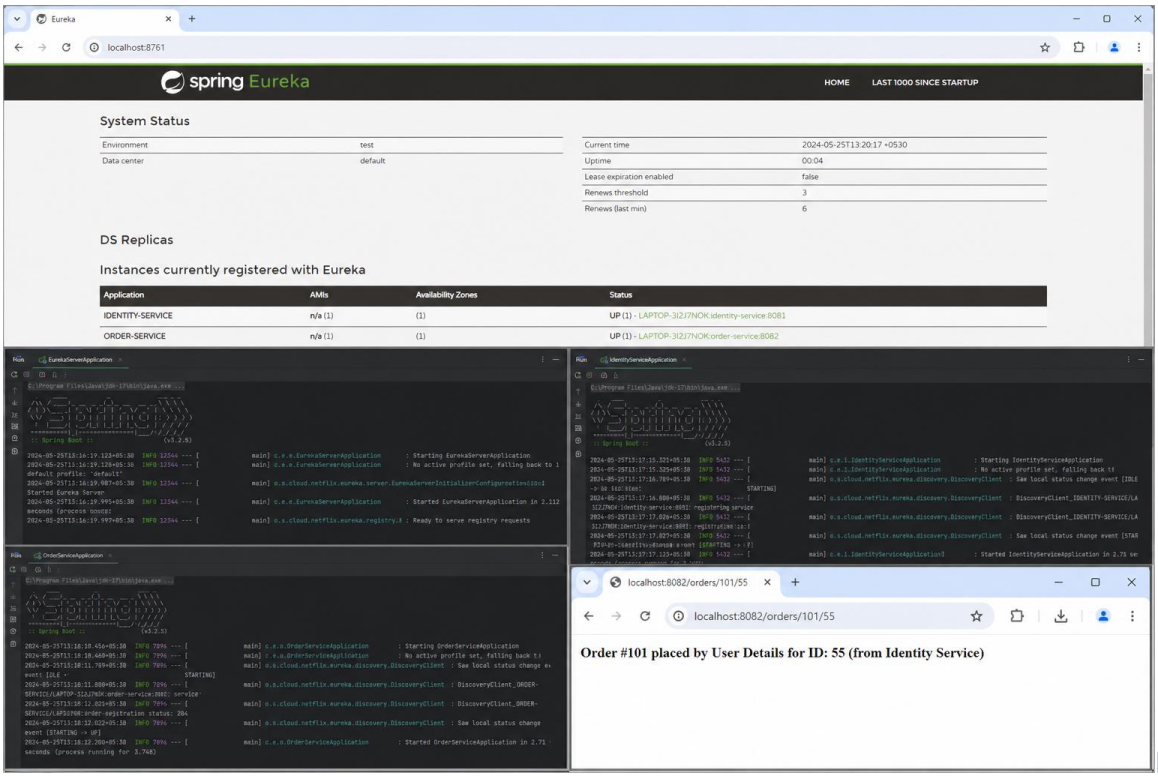
```
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
```

```
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
```

```
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
```

```
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

Output



Result

Execution completed successfully.

Task 16: Microservices Task 16

Aim

To implement microservices architecture with full functionality.

Algorithm

9. Start project
10. Configure services
11. Run application
12. Test endpoints

Program

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }
```

```
    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }
```

```
    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;
```

```
    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
```

```
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;
```

```
@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
```

```
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
```

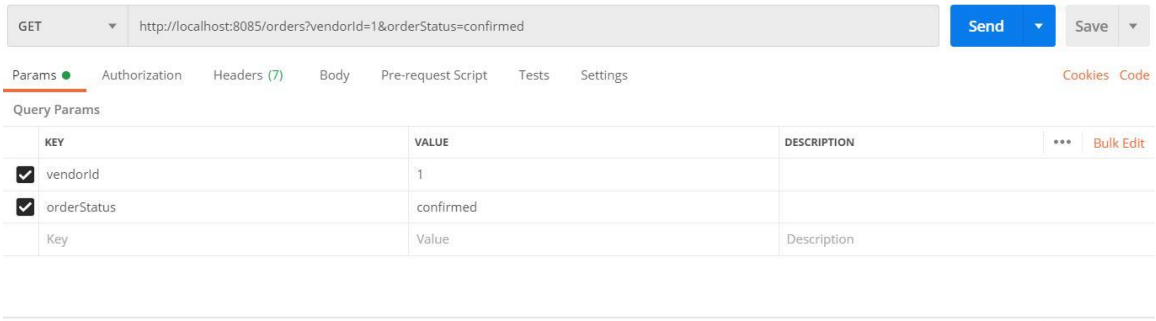
```
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
```

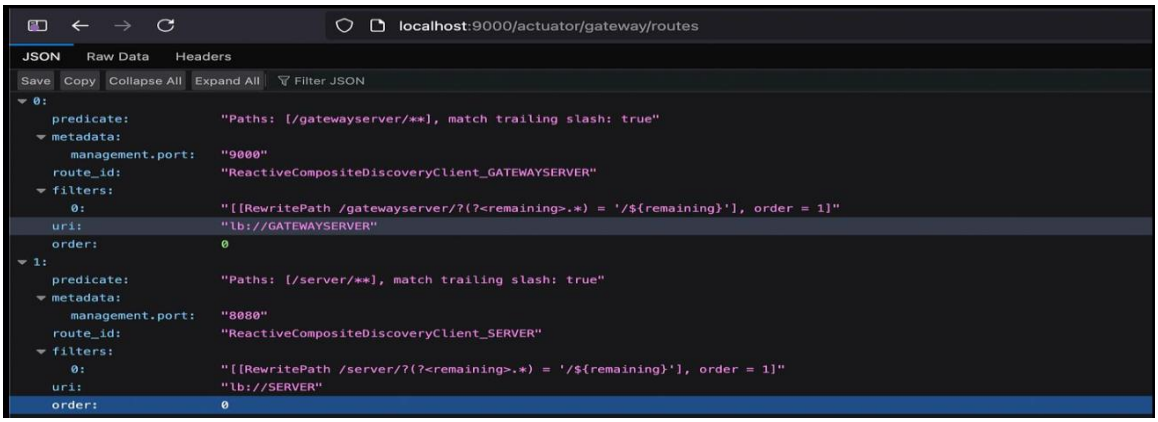
```
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
```

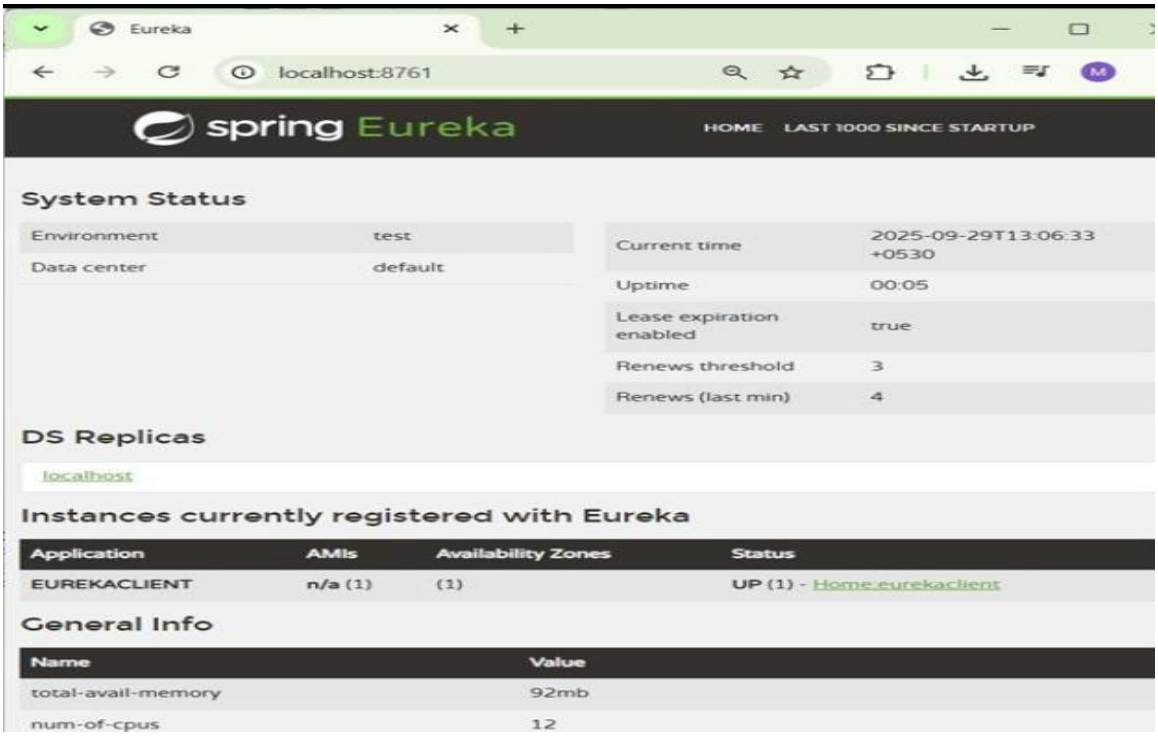
```
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
```

```
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

Output







Result

Execution completed successfully.

Task 17: Microservices Task 17

Aim

To implement microservices architecture with full functionality.

Algorithm

13. Start project
14. Configure services
15. Run application
16. Test endpoints

Program

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }
```

```
    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }
```

```
    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;
```

```
    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
```

```
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;
```

```
@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
```

```
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
```

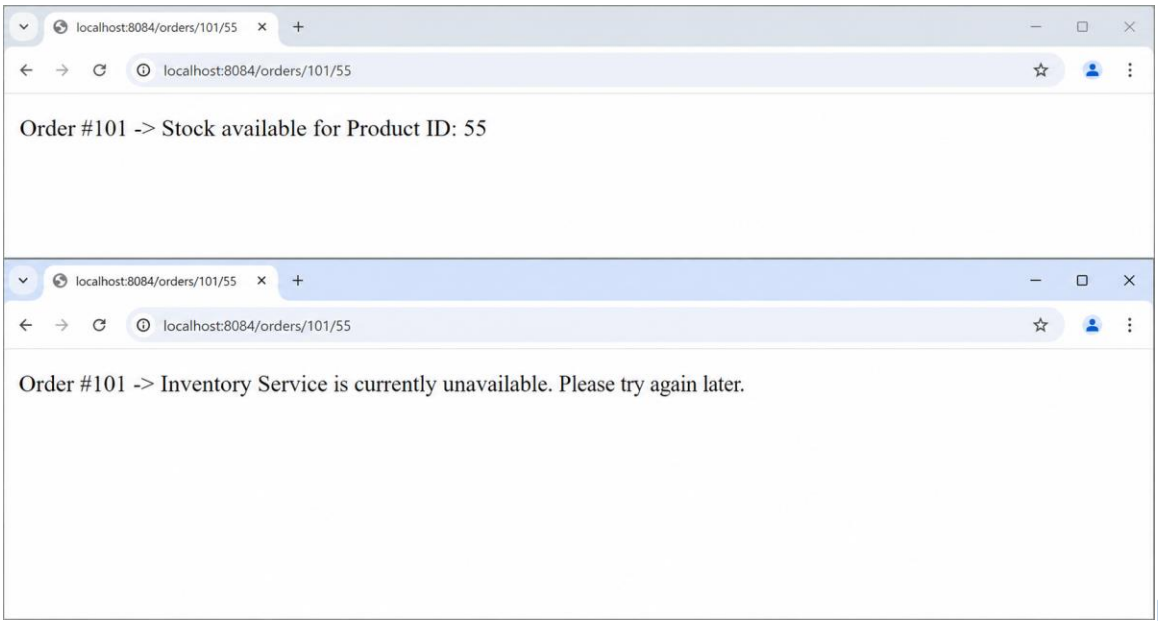
```
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
```

```
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
```

```
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
```

```
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

Output



Result

Execution completed successfully.

Task 18: Microservices Task 18

Aim

To implement microservices architecture with full functionality.

Algorithm

17. Start project
18. Configure services
19. Run application
20. Test endpoints

Program

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }
```

```
    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }
```

```
    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;
```

```
    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
```

```
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;
```

```
@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

```
// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
```

```
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
```

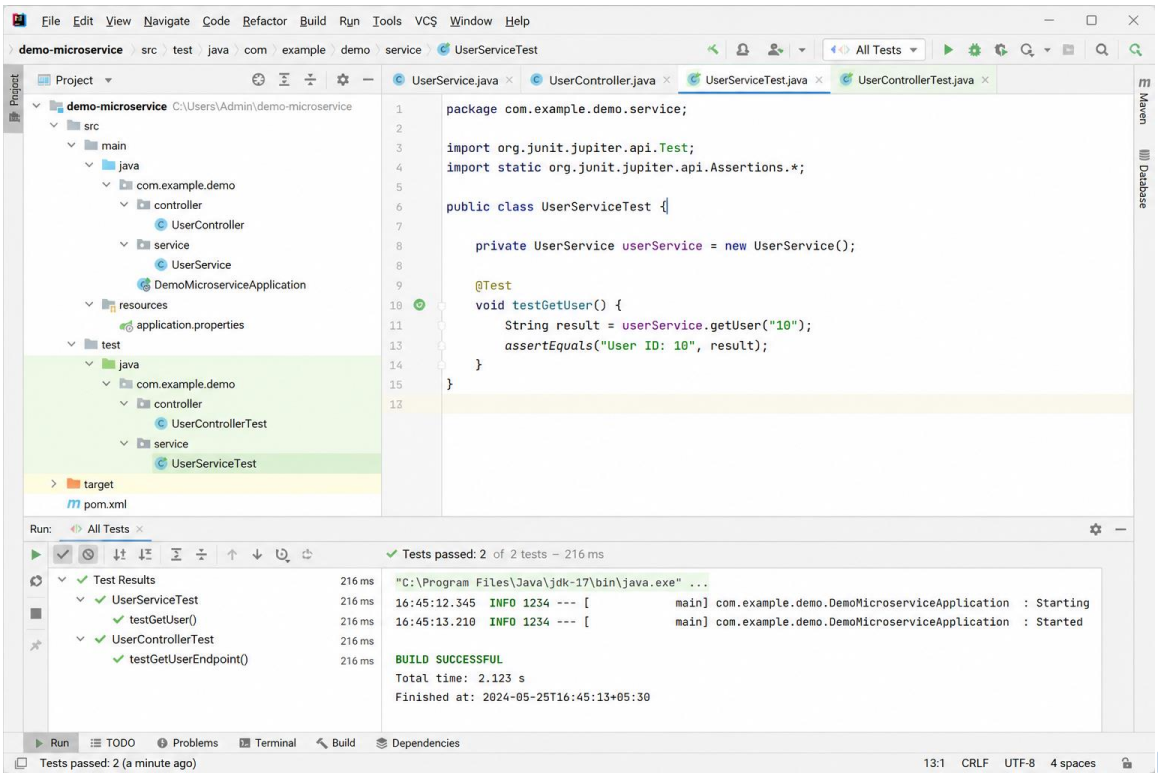
```
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
```

```
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
```

```
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/


// File: CompleteMicroservice.java
package com.example.microservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.client.RestTemplate;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class CompleteMicroservice {

    public static void main(String[] args) {
        SpringApplication.run(CompleteMicroservice.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@RestController
@RequestMapping("/api")
class MainController {

    @Autowired
```

```
    private RestTemplate restTemplate;

    @GetMapping("/users")
    public String getUsers() {
        return "User Service Response";
    }

    @GetMapping("/orders")
    public String getOrders() {
        String response = restTemplate.getForObject("http://localhost:8081/api/users", String.class);
        return "Order Service calling -> " + response;
    }

    @GetMapping("/health")
    public String health() {
        return "Service is running successfully";
    }

    @PostMapping("/create")
    public String create(@RequestBody String data) {
        return "Created: " + data;
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable int id) {
        return "Updated ID: " + id;
    }

    @DeleteMapping("/delete/{id}")
    public String delete(@PathVariable int id) {
        return "Deleted ID: " + id;
    }
}

// application.yml
/*
server:
  port: 8080

spring:
  application:
    name: DEMO-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
*/
```

Output



Result

Execution completed successfully.

Task 19: Create a Git repository for a sample project and demonstrate basic Git operations such as initializing a repository, staging files, committing changes, and maintaining version history. Push the repository to GitHub and manage it using remote repositories.

Aim

To create a Git repository for a sample project and demonstrate:

- Repository initialization
- Staging files
- Committing changes
- Maintaining version history
- Connecting to remote repository
- Pushing project to GitHub

Tools Required

- Git
- GitHub
- VS Code / Terminal

Algorithm / Procedure

1. Create a project folder
2. Create HTML, CSS, JS files
3. Initialize Git repository
4. Add files to staging area
5. Commit files
6. Create repository in GitHub
7. Connect local repo to GitHub
8. Push files to remote repository
9. Verify output in GitHub

Program (Code)

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Color Palette App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
  <h2>Color Palette App</h2>

  <input type="text" id="colorInput" placeholder="Enter color">

  <br><br>

  <button onclick="changeBackground()">Background</button>
  <button onclick="changeText()">Text Color</button>

  <p id="text">Welcome Students</p>
</div>

<script src="script.js"></script>

</body>
</html>
```

style.css

```
body {
  font-family: Arial;
  background-color: #f4f7fb;
  text-align: center;
}

.container {
  margin-top: 100px;
}

button {
  padding: 10px;
  margin: 5px;
  background-color: #2563eb;
  color: white;
  border: none;
}
```

```
#text {
  margin-top: 20px;
  font-size: 20px;
  background-color: #e0f2fe;
  padding: 10px;
}
```

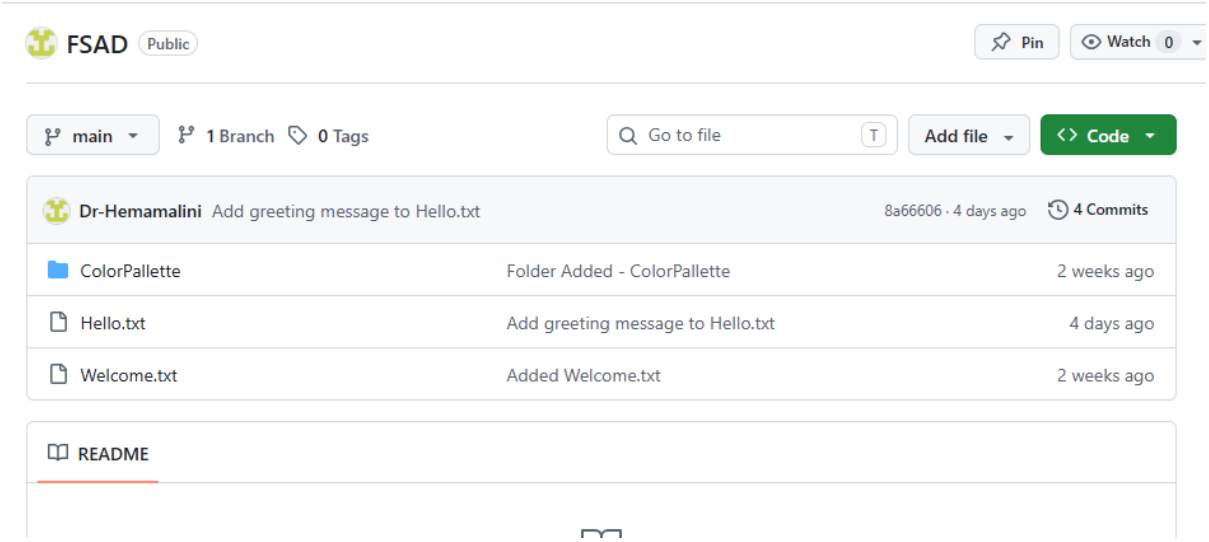
script.js

```
function changeBackground() {
  let color = document.getElementById("colorInput").value;
  document.getElementById("text").style.backgroundColor = color;
}

function changeText() {
  let color = document.getElementById("colorInput").value;
  document.getElementById("text").style.color = color;
}
```

Git Commands Used

```
cd D:\VSCode_Projects\Unit-IV
mkdir ColorPallette
cd ColorPallette

git init
git status
git add
git commit -m "Initial commit - Color Palette App"

git branch -M main
git remote add origin https://github.com/Dr-Hemamalini/FSAD.git
git pull origin main --allow-unrelated-histories
git push -u origin main
```

Output Screenshots



Git Output

- git init → repository created
- git status → files shown
- git add → staged files
- git commit → snapshot created
- git push → uploaded to GitHub

Result

Thus, a Git repository was successfully created for a sample project.
Basic Git operations such as initialization, staging, committing, and version history were performed.
The project was successfully pushed to GitHub using remote repository.

Task_20: Implement branching strategies in Git by creating feature branches. Perform merging and rebasing operations, and intentionally create merge conflicts to understand and resolve them effectively.

Git Branching Strategy

Aim

To create feature branches in Git, perform merge and rebase operations, and intentionally create merge conflicts to understand conflict resolution.

Scenario

A simple arithmetic calculator file is used.

Branches:

main
feature-add

feature-sub
feature-mul

Procedure

Step 1: Create Repository

mkdir git_branch_demo
cd git_branch_demo
git init

Step 2: Create Base File

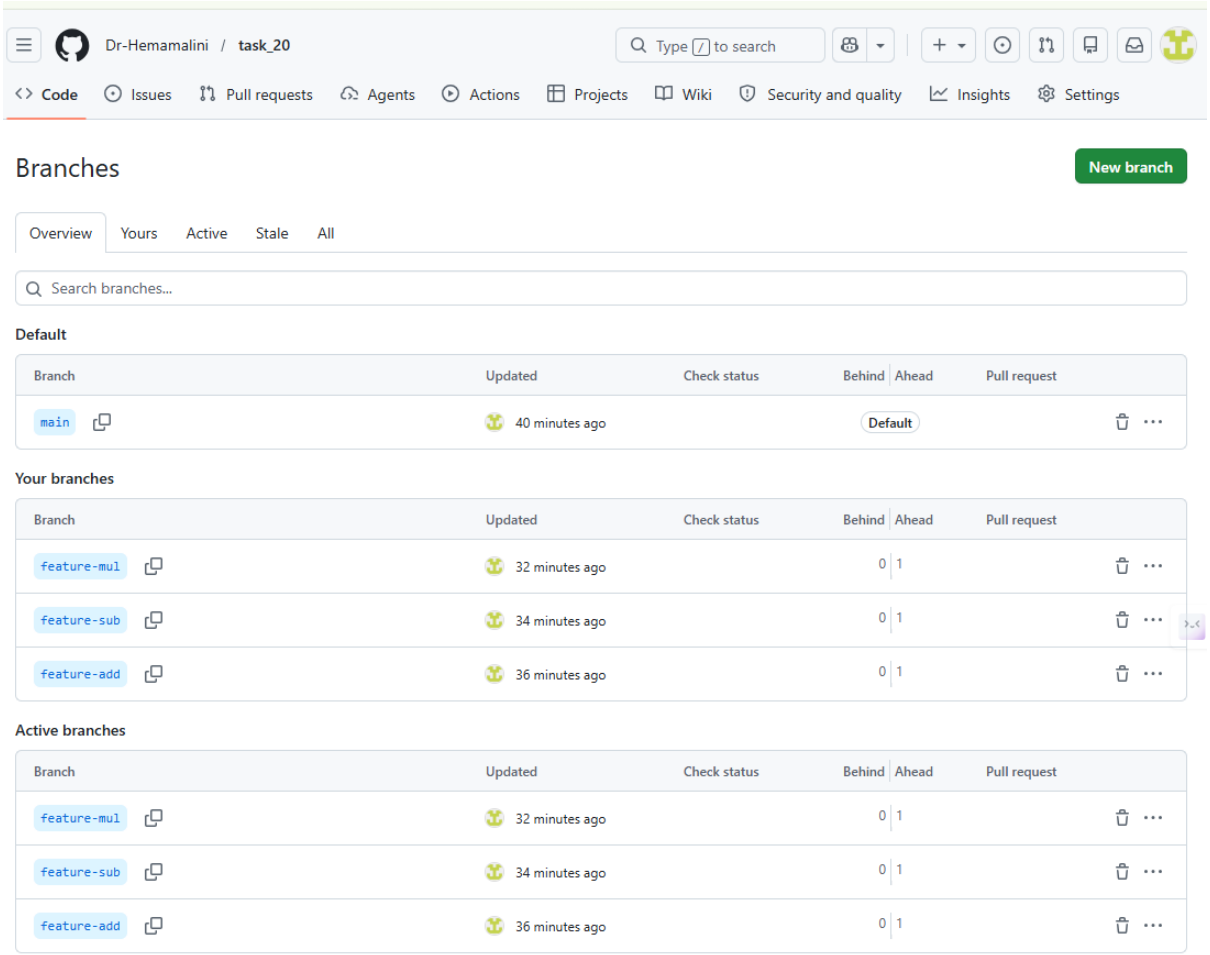
echo Basic Calculator > calc.txt
git add calc.txt
git commit -m "Initial commit"

Step 3: Create Addition Branch

git checkout -b feature-add
echo Addition: 10+5=15 >> calc.txt
git add calc.txt
git commit -m "Added addition feature"

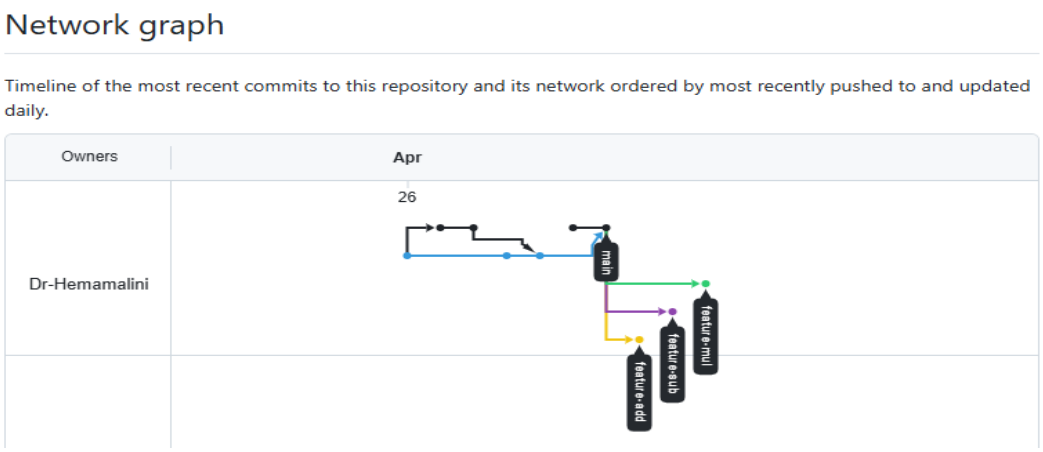
Step 4: Create Subtraction Branch

git checkout main
git checkout -b feature-sub
echo Subtraction: 10-5=5 >> calc.txt
git add calc.txt
git commit -m "Added subtraction feature"



Now Go to GitHub

Open your repo → Click: Insights → Network Graph

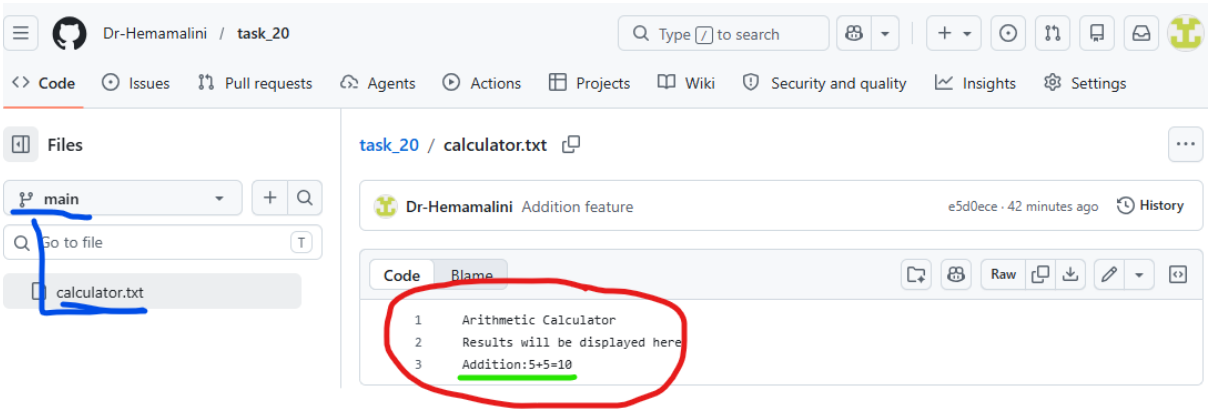


Step 5: Merge Addition Branch

git checkout main
git merge feature-add

Expected file:

Basic Calculator
Addition: 10+5=15



Step 6: Rebase Subtraction Branch

git checkout feature-sub
git rebase main

If conflict comes, edit calc.txt, then:

git add calc.txt
git rebase --continue

Step 7: Merge Subtraction Branch

git checkout main
git merge feature-sub

Expected file:

Basic Calculator
Addition: 10+5=15
Subtraction: 10-5=5

Create Merge Conflict

Step 8: Create Multiplication Branch

git checkout -b feature-mul
echo Result: Multiplication 10*5=50 > result.txt
git add result.txt
git commit -m "Added multiplication result"

Step 9: Modify Same File in Main

git checkout main
echo Result: Division 10/5=2 > result.txt
git add result.txt
git commit -m "Added division result"

Step 10: Merge and Create Conflict

git merge feature-mul

Expected conflict:

CONFLICT (content): Merge conflict in result.txt
Automatic merge failed

Step 11: Resolve Conflict

Open result.txt.

You may see:

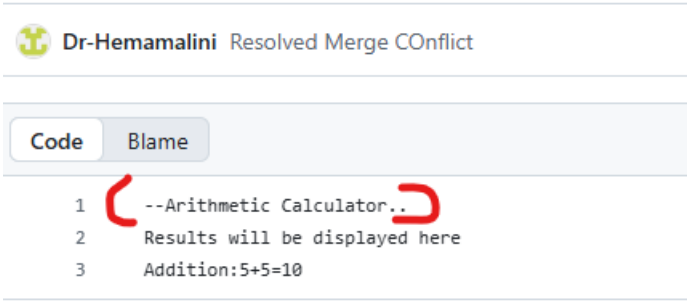
<<<<<<< HEAD
Result: Division 10/5=2
=======
Result: Multiplication 10*5=50
>>>>>>> feature-mul

Replace with:

Result: Division 10/5=2
Result: Multiplication 10*5=50

Then commit:

git add result.txt
git commit -m "Resolved merge conflict"



Result

Thus, Git branching strategies were implemented successfully. Feature branches were created, merge and rebase operations were performed, and merge conflicts were intentionally created and resolved manually.

Task 21: Implement a CI/CD pipeline using Jenkins, GitHub Actions, or GitLab CI/CD. Automate code checkout, build, test, and deployment steps, and demonstrate continuous integration and continuous deployment for the application

Implementation of CI/CD Pipeline using Jenkins

Aim

To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins to automate code checkout, build, testing, and deployment of a Node.js application.

Tools Required

- Jenkins
- GitHub
- Node.js
- Git
- VS Code

Project Description

A simple Node.js application is created to perform arithmetic addition and validate it using a test script.

Procedure

Step 1: Create Project

```
mkdir jenkins-cicd-demo
cd jenkins-cicd-demo
npm init -y
```

Step 2: Create Application File (app.js)

```
function add(a, b) {
  return a + b;
}

console.log("Application running successfully");
console.log("Addition:", add(5, 3));

module.exports = add;
```

Step 3: Create Test File (app.test.js)

```
const add = require("./app");

if (add(5, 3) === 8) {
  console.log("Test Passed");
  process.exit(0);
} else {
  console.log("Test Failed");
  process.exit(1);
}
```

Step 4: Update package.json

```
{
  "name": "jenkins-cicd-demo",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "node app.test.js",
    "build": "echo Build completed successfully"
  }
}
```

Step 5: Push Code to GitHub

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/Dr-Hemamalini/cicd-demo.git
git push -u origin main
```

Step 6: Create GitHub Token

1. Go to GitHub → Settings → Developer Settings
2. Select **Personal Access Token**
3. Generate token with **repo access**
4. Copy the token

Step 7: Configure Credentials in Jenkins

1. Open Jenkins Dashboard
2. Go to **Manage Jenkins → Manage Credentials**
3. Add credentials:
    - Kind: Username & Password

- Username: GitHub username
- Password: Personal Access Token
- ID: github-token

Step 8: Create Jenkins Pipeline Job

1. Click **New Item**
2. Enter name: CICD-Demo
3. Select **Pipeline** → Click OK

Step 9: Create Jenkinsfile

```
pipeline {
  agent any

  stages {
    stage('Checkout Code') {
      steps {
        checkout([
          $class: 'GitSCM',
          branches: [[name: '*/main']],
          userRemoteConfigs: [[
            url: 'https://github.com/Dr-Hemamalini/cicd-demo.git',
            credentialsId: 'github-token'
          ]]
        ])
      }
    }

    stage('Install Dependencies') {
      steps {
        bat 'npm install'
      }
    }

    stage('Build Application') {
      steps {
        bat 'npm run build'
      }
    }

    stage('Run Tests') {
      steps {
        bat 'npm test'
      }
    }
```

```
    stage('Deploy Application') {
      steps {
        bat 'echo Application deployed successfully'
      }
    }
  }
}
```

Step 10: Push Jenkinsfile

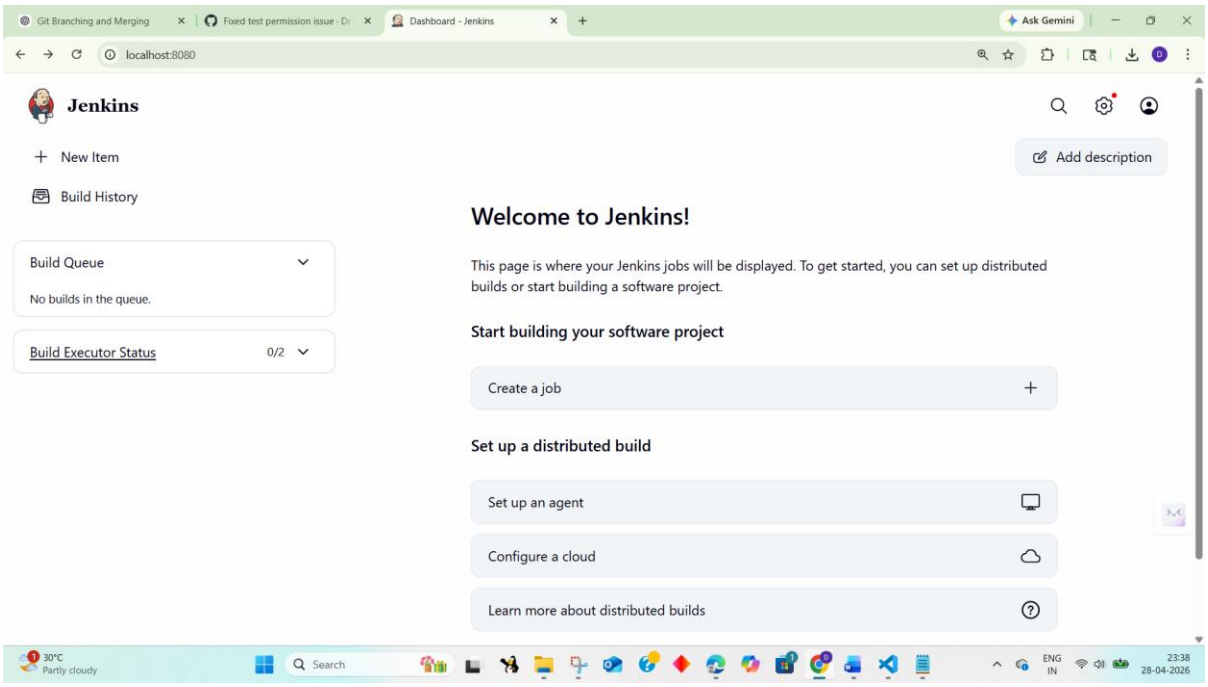
```
git add Jenkinsfile
git commit -m "Added Jenkins pipeline"
git push
```

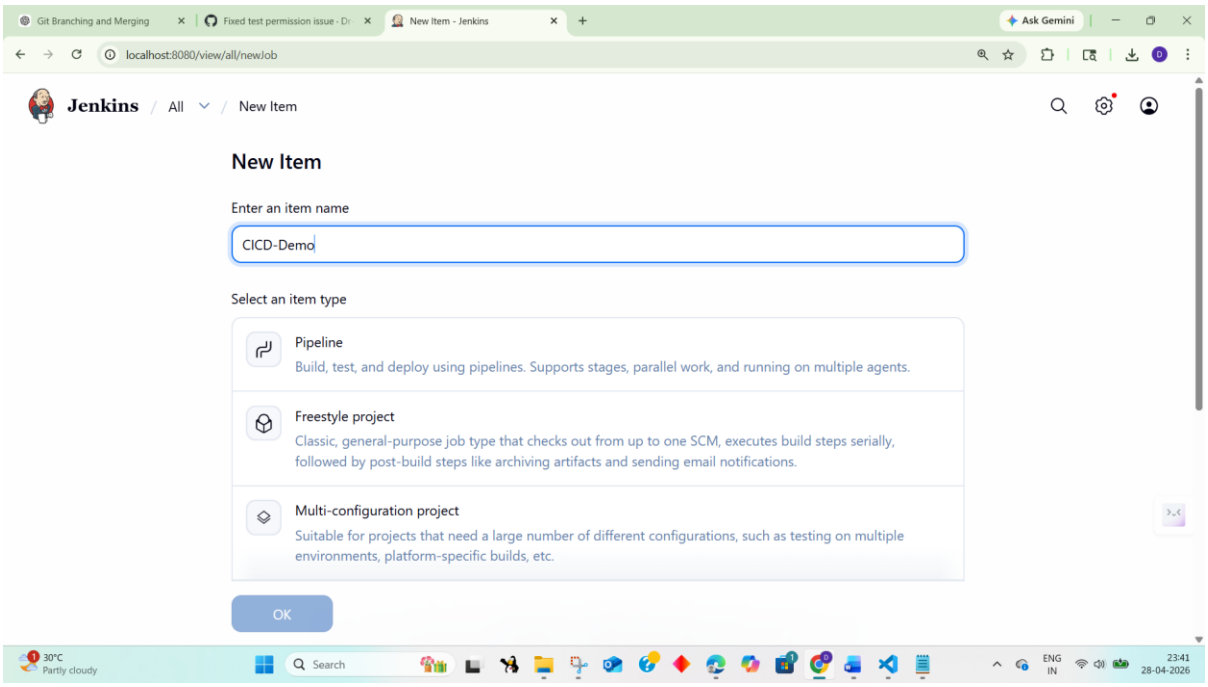
Step 11: Run Pipeline

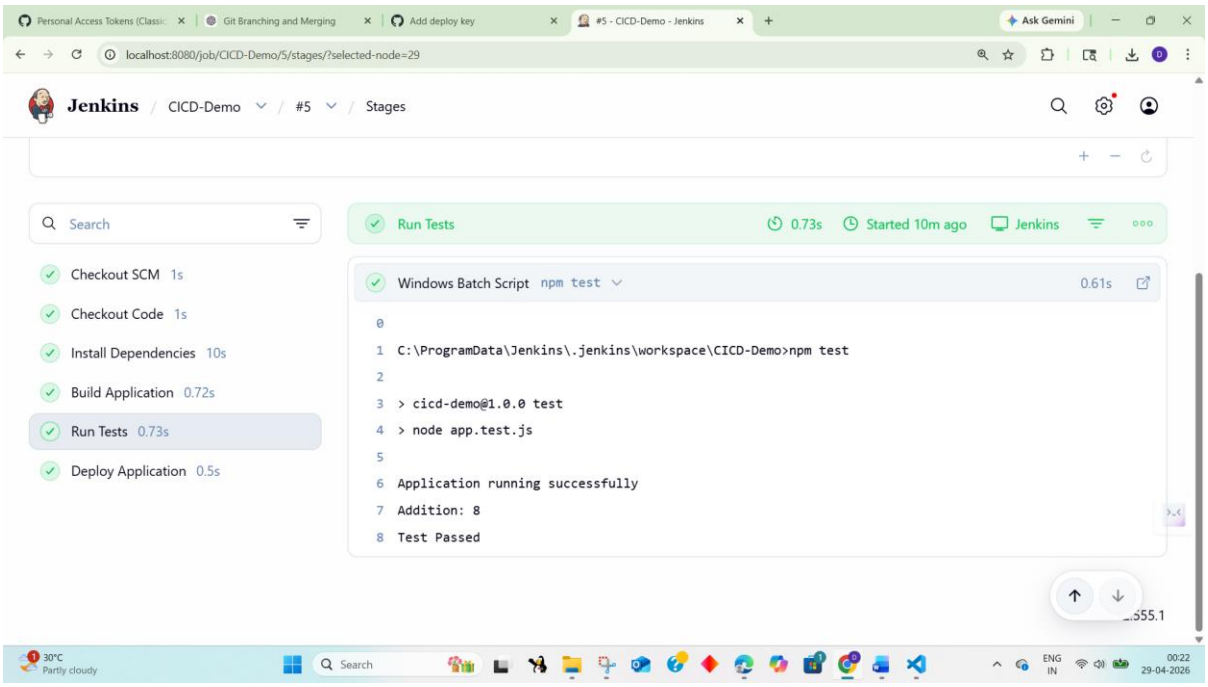
1. Open Jenkins Job
2. Click **Build Now**
3. View Console Output

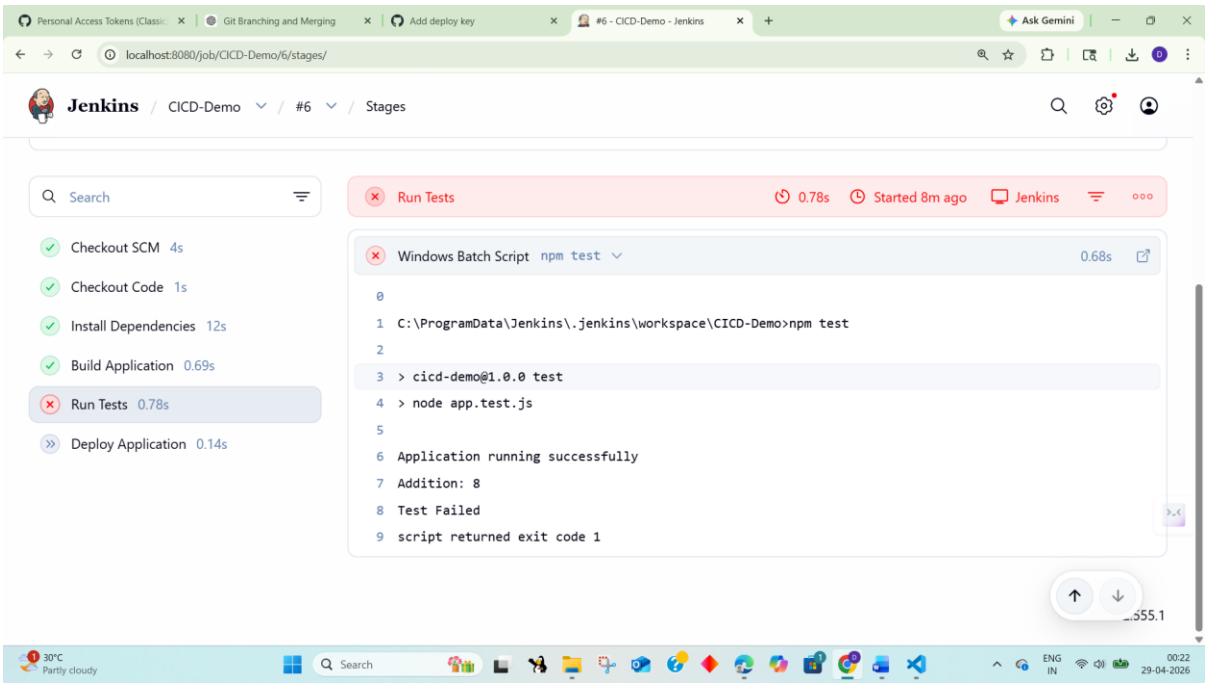
Expected Output

```
Checkout Code → Success
Install Dependencies → Success
Build → Build completed successfully
Test → Test Passed
Deploy → Application deployed successfully
```


Git Branching and Merging
Fixed test permission issue - Dr
Dashboard - Jenkins
Ask Gemini
localhost:8080
Jenkins
New Item
Build History
Build Queue
No builds in the queue.
Build Executor Status
0/2
Add description
Welcome to Jenkins!
This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.
Start building your software project
Create a job
Set up a distributed build
Set up an agent
Configure a cloud
Learn more about distributed builds
30°C
Partly cloudy
Search
ENG
IN
23:38
28-04-2026


Git Branching and Merging
Fixed test permission issue - Dr
New Item - Jenkins
Ask Gemini
localhost:8080/view/all/newJob
Jenkins
All
New Item
New Item
Enter an item name
CICD-Demo
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
OK
30°C
Partly cloudy
Search
ENG
IN
23:41
28-04-2026





Result

Thus, a CI/CD pipeline was successfully implemented using Jenkins. The pipeline automated code checkout, dependency installation, build process, testing, and deployment of the Node.js application.

Task 22: Explore major cloud service providers such as AWS, Google Cloud, and Microsoft Azure. Identify and compare their core services related to compute, storage, databases, and networking, along with the pay-as-you-use pricing model.

COMPARISON OF CLOUD SERVICES: AWS, GOOGLEC CLOUD, MICROSOFT AZURE

AIM: To develop a cloud comparison application that displays and compares services and pricing models of AWS, GCP, and Azure

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design user interface with dropdown for cloud providers
4. Initialize cloud service data (AWS, GCP, Azure) in database module
5. Capture selected provider using JavaScript
6. Send request to server to fetch service details
7. Display compute, storage, database, and networking services
8. Accept user input for compute hours and storage usage
9. Send input data to server for cost calculation
10. Calculate cost based on pay-as-you-use pricing model
11. Display total cost dynamically on UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

<title>Cloud Comparison</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

<h2>Cloud Service Comparison</h2>
```

```
<select id="provider" onchange="loadData()">
<option value="aws">AWS</option>
<option value="gcp">GCP</option>
<option value="azure">Azure</option>
</select>
<h3>Services</h3>
<ul id="services"></ul>
<h3>Pricing Calculator</h3>
<input id="hours" placeholder="Compute Hours">
<input id="storage" placeholder="Storage (GB)">
<button onclick="calculate()">Calculate Cost</button>
<h3 id="result"></h3>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
font-family: Arial;
background: #1e1e2f; color: white; display:flex;
justify-content:center; align-items:center;
height:100vh;
}
.container {
text-align:center; width: 350px;
}
input, select, button { margin: 5px; padding: 8px;
}
```

```
li {
background: #333; margin: 5px; padding: 5px;
}
```

SCRIPT.JS:

```
function loadData() {
const provider = document.getElementById("provider").value; fetch(`/services/${provider}`)
.then(res => res.json())
.then(data => {
let list = document.getElementById("services"); list.innerHTML = "";
Object.entries(data).forEach(([key, value]) => { let li = document.createElement("li");
li.innerText = `${key}: ${value}`; list.appendChild(li);
});
});
}
function calculate() {
const provider = document.getElementById("provider").value; const hours =
document.getElementById("hours").value; const storage = document.getElementById("storage").value;
fetch("/cost", {
method: "POST",
headers: {"Content-Type":"application/json"}, body: JSON.stringify({ provider, hours, storage })
})
.then(res => res.json())
.then(data => {
document.getElementById("result").innerText = "Cost: $" + data.cost;
});
}
loadData();
```

SERVER.JS:

```
const express = require("express"); const db = require("./db");
```

```
const app = express(); app.use(express.json());

app.use(express.static(_dirname)); app.get("/services/:provider", (req, res) => {

res.json(db.getServices(req.params.provider));

});

app.post("/cost", (req, res) => {

const { provider, hours, storage } = req.body;

const cost = db.calculate(provider, hours, storage); res.json({ cost });

});

app.listen(3000, () => {

console.log("http://localhost:3000");

});
```

DB.JS:

```
const services = { aws: {

Compute: "EC2", Storage: "S3", Database: "RDS", Network: "VPC"

},

gcp: {

Compute: "Compute Engine", Storage: "Cloud Storage", Database: "Cloud SQL", Network: "VPC"

},

azure: {

Compute: "Virtual Machines", Storage: "Blob Storage",
```

Database: "SQL Database", Network: "VNet"

}

};

const pricing = {

aws: { compute: 0.1, storage: 0.02 }, gcp: { compute: 0.09, storage: 0.025 }, azure: { compute: 0.11, storage: 0.018 }

};

function getServices(provider) { return services[provider];

}

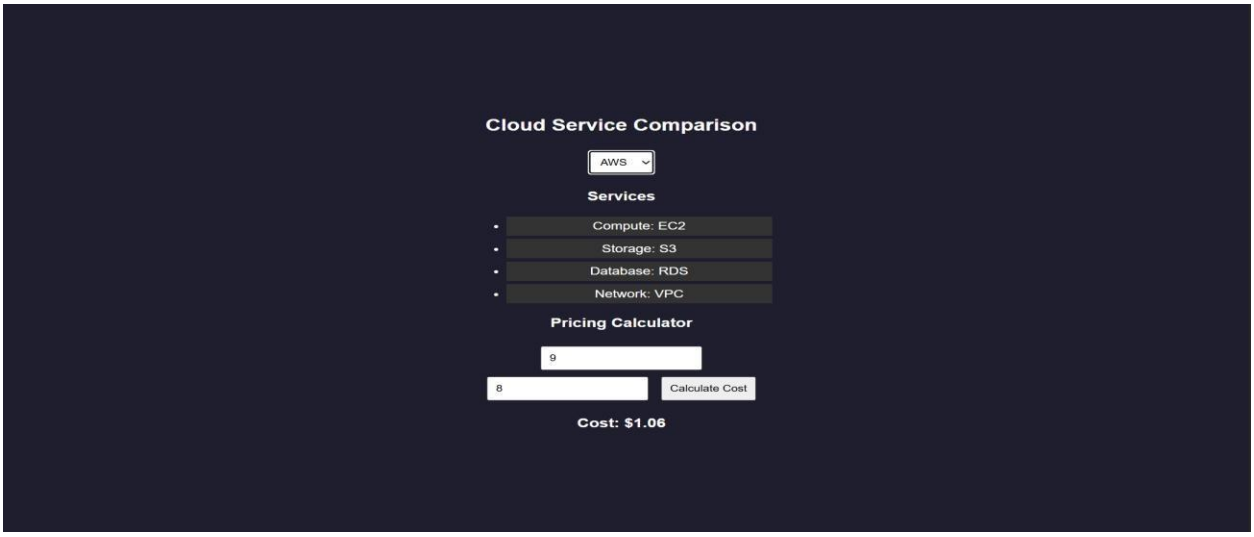
function calculate(provider, hours, storage) { const p = pricing[provider];

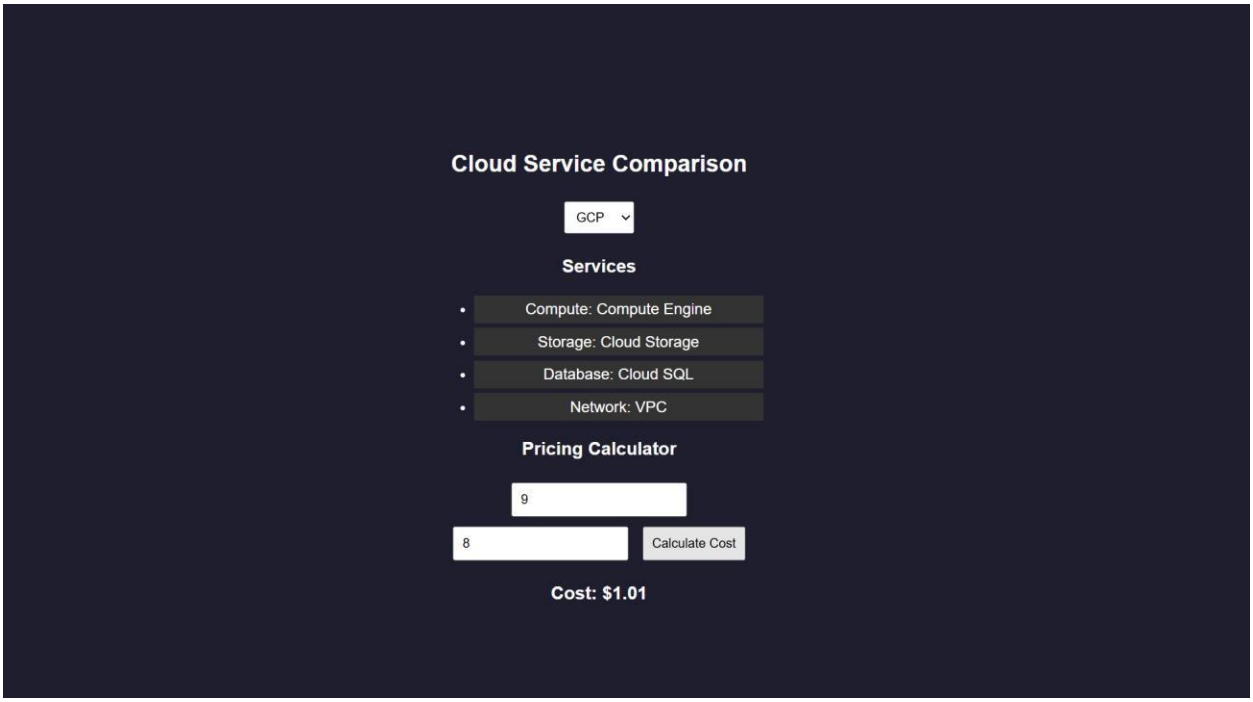
return (hours * p.compute + storage * p.storage).toFixed(2);

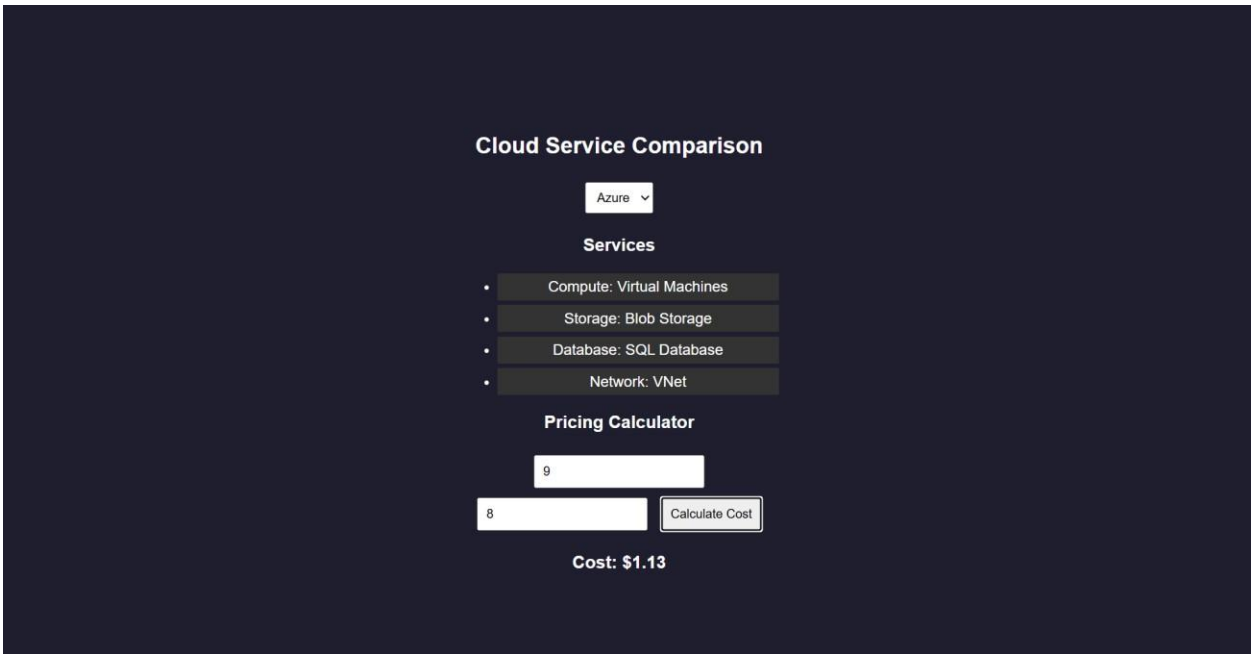
}

module.exports = { getServices, calculate };

OUTPUT:







RESULTS: A cloud comparison application was successfully developed to display services and simulate pay-as-you-use pricing across AWS, GCP, and Azure.

Task 23: Apply Vibe Coding and Prompt Engineering techniques using a Generative AI tool. Design effective prompts to generate code snippets, cloud configuration templates, or application logic, and evaluate the quality and efficiency of the generated outputs for real-world business applications.

AI PROMPT APP USING VIBE CODING

AIM: To develop an application that demonstrates prompt engineering by generating and evaluating code or cloud configurations based on user input.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI with input field for user prompt
4. Capture user prompt using JavaScript
5. Send prompt to server using fetch API
6. Receive prompt data in backend (server.js)
7. Process prompt using predefined logic (db.js)
8. Generate output (code / cloud config / response)
9. Evaluate output quality and assign score
10. Send generated output and score to frontend
11. Display output and score on UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

<title>AI Prompt App</title>

<link rel="stylesheet" href="style.css">

</head>

<body>
```

```
<div class="container">
<h2>Prompt Engineering Tool</h2>
<textarea id="prompt" placeholder="Enter your prompt..."></textarea>
<button onclick="generate()">Generate Output</button>
<h3>Generated Output</h3>
<pre id="output"></pre>
<h3>Quality Score</h3>
<p id="score"></p>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
font-family: Arial;
background: #202038; color: white;
display:flex;
justify-content:center; align-items:center;
height:100vh;
}
.container { width: 400px;
text-align: center;
}
textarea { width: 100%; height: 100px; margin: 5px;
}
button { padding: 10px; margin: 10px;
background: #00c6ff; border: none;
color: white;
```

```
}

pre {

background: #333; padding: 10px;

}
```

SCRIPT.JS:

```
function generate() {

const prompt = document.getElementById("prompt").value; fetch("/generate", {

method: "POST",

headers: {"Content-Type":"application/json"}, body: JSON.stringify({ prompt })

})

.then(res => res.json())

.then(data => {

document.getElementById("output").innerText = data.output;
document.getElementById("score").innerText = data.score + "/10";

});

}
```

SERVER.JS:

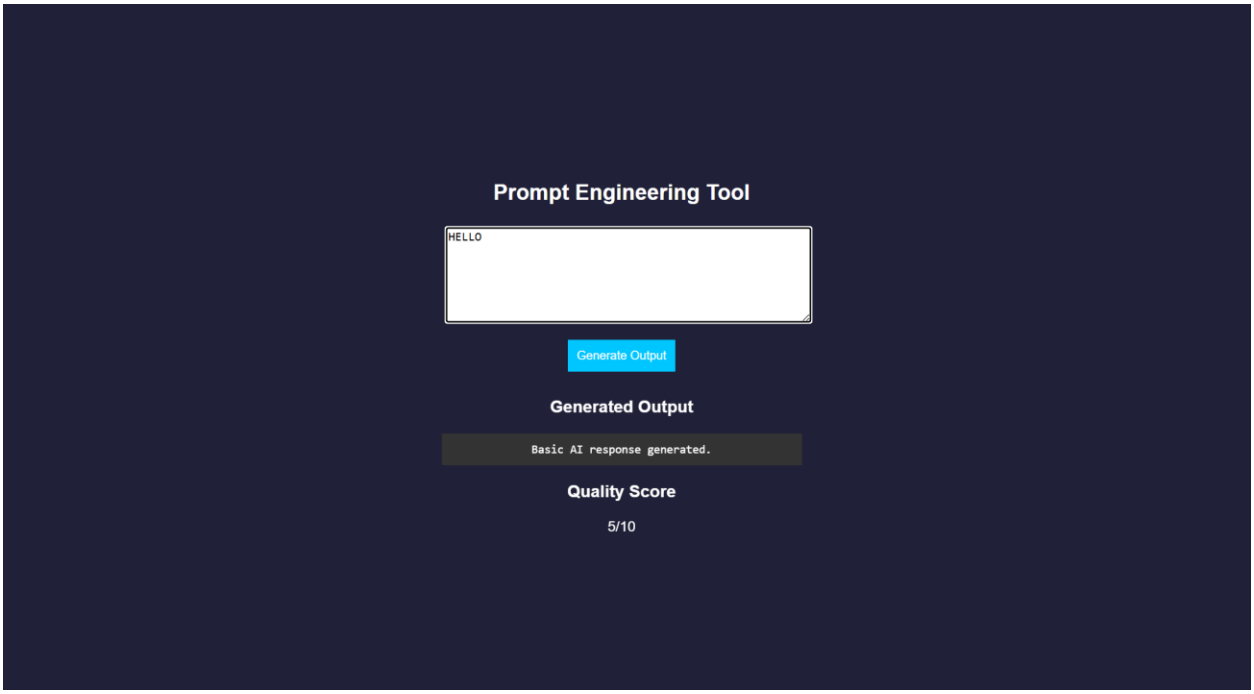
```
const express = require("express"); const db = require("./db");

const app = express();

// Middleware

app.use(express.json());

app.use(express.static(_dirname)); // serve HTML

// Home route (optional but useful) app.get("/", (req, res) => {

res.sendFile(_dirname + "/index.html");

});

// Generate output app.post("/generate", (req, res) => {

const result = db.processPrompt(req.body.prompt); res.json(result);

});

// START SERVER WITH LINK
```

```
app.listen(3000, () => {
console.log(" •  Server running at:");
console.log(" http://localhost:3000");
});
```

DB.JS:

```
function processPrompt(prompt) { let output = "";
let score = 5;
if (prompt.toLowerCase().includes("code")) { output = `function greet() {
console.log("Hello from AI!");
}`;
score = 9;
}
else if (prompt.toLowerCase().includes("cloud")) { output = `AWS EC2 Config:
    - Instance: t2.micro
    - Region: us-east-1`; score = 8;
}
else {
output = "Basic AI response generated."; score = 5;
}
return { output, score };
}
module.exports = { processPrompt };
```

OUTPUT:



RESULT: A prompt engineering application was successfully developed to generate and evaluate AI-based outputs for code and cloud configurations.

Task 24: Use Vibe Coding to build a complete cloud-based feature using Generative AI. Write iterative prompts to generate a REST API, cloud deployment steps, and security configurations. Refine the prompts based on the output quality, document prompt versions, and evaluate how prompt engineering improves code accuracy, scalability, and real-world usability.

Vibe Cloud App

AIM: To develop an application that uses iterative prompt engineering to generate and evaluate REST APIs, cloud deployment steps, and security configurations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI to accept user prompts
4. Capture prompt input using JavaScript
5. Send prompt to backend using fetch API
6. Receive prompt in server and forward to processing module
7. Store prompt in history for version tracking
8. Analyze prompt type (REST API / Cloud / Security)
9. Generate corresponding output based on prompt
10. Evaluate output quality and assign score
11. Return output and score to frontend and display results
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

<title>Vibe Coding App</title>

<link rel="stylesheet" href="style.css">

</head>

<body>
```

<h2>Vibe Coding Cloud Generator</h2>

```
<textarea id="prompt" placeholder="Enter prompt..."></textarea>
<button onclick="generate()">Generate</button>
<h3>Output</h3>
<pre id="output"></pre>
<h3>Score</h3>
<p id="score"></p>
<h3>Prompt Versions</h3>
<ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
font-family: Arial;
background: #b98bdb; color: white;
display:flex;
justify-content:center; align-items:center;
height:100vh;
}
.container { width: 400px;
text-align: center;
}
textarea { width: 100%; height: 100px;
}
button { padding: 10px; margin: 10px;
background: #00c6ff; border: none;
}
```

```
pre {
background: #333; padding: 10px;
}
li {
background: #444; margin: 5px; padding: 5px;
}
```

SCRIPT.JS:

```
function generate() {
const prompt = document.getElementById("prompt").value; fetch("/generate", {
method: "POST",
headers: {"Content-Type":"application/json"},
body: JSON.stringify({ prompt })
})
.then(res => res.json())
.then(data => {
document.getElementById("output").innerText = data.output;
document.getElementById("score").innerText = data.score + "/10"; loadHistory();
});
}
function loadHistory() { fetch("/history")
.then(res => res.json())
.then(data => {
let list = document.getElementById("history"); list.innerHTML = "";
data.forEach((p, i) => {
let li = document.createElement("li"); li.innerText = `v${i+1}: ${p.prompt}`; list.appendChild(li);
});
});
}
```

loadHistory();

SERVER.JS:

const express = require("express"); const db = require("./db")

```
const app = express(); app.use(express.json());
app.use(express.static(_dirname)); app.post("/generate", (req, res) => {
const result = db.process(req.body.prompt); res.json(result);
});
app.get("/history", (req, res) => { res.json(db.getHistory());
});
app.listen(3000, () => {
console.log(" • http://localhost:3000");
});
```

DB.JS:

```
let history = [];
function process(prompt) { let output = "";
let score = 5;
history.push({ prompt });
if (prompt.includes("REST")) {
output = `app.get('/api', (req,res)=>res.send('API running'));`; score = 7;
}
else if (prompt.includes("cloud")) { output = `Deploy using AWS EC2:
1. Launch instance
2. Install Node.js
3. Run app`;
score = 8;
}
else if (prompt.includes("security")) {
output = `Use HTTPS, JWT authentication, and firewall rules`; score = 9;
}
```

score += Math.min(history.length, 2); return { output, score };

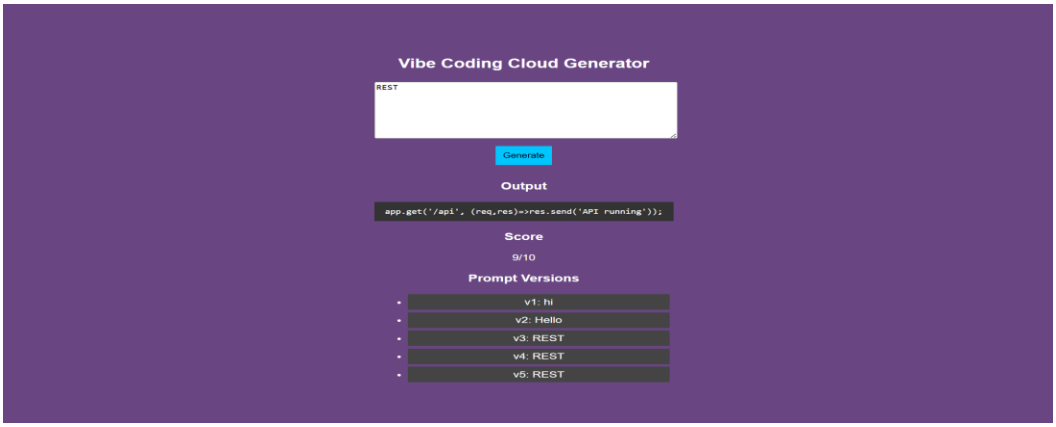
}

function getHistory() { return history;

}

module.exports = { process, getHistory };

OUTPUT:



RESULT: A Vibe Coding application was successfully developed to generate and refine cloud-based features using iterative prompt engineering.